

Corso di Laurea triennale in Ingegneria e Scienze Informatiche

**Analisi comparativa di modelli di Deep Learning per la
riduzione del rumore in immagini digitali**

Elaborato in: Metodi numerici per l'intelligenza artificiale

Relatore:

Prof. Damiana Lazzaro

Candidato:

Matteo Vanni

Matricola: 0000935584

Sessione Unica

Anno accademico 2024-2025

*"I can have a dream, can't I?
You don't have to laugh at it.
A dream is what keeps you going."
- Ichiban Kasuga, Y:LAD*

Indice

Introduzione	11
1 Denoising di immagini	13
1.1 Definizione del problema	13
1.2 Denoising di Immagini tramite Deep Learning	19
1.3 Metriche di Valutazione	19
1.4 Riflessione sulla letteratura	20
2 Analisi di reti neurali per il denosing di immagini	21
2.1 RIDNet	21
2.2 DnCNN	23
2.3 Autoencoder	24
3 Descrizione dei dataset	25
3.1 BSD500	26
3.2 DIV2K	26
3.3 MIT-Adobe FiveK	27
3.4 SIDD	28
3.5 RENOIR	28
3.6 KVASIR	29
4 Preprocessing dei Dataset	31
4.1 Conversione immagini RAW	31
4.2 Padding di immagini con risoluzione inferiore a 512x512	39
4.3 Patching	41
4.4 Aggiunta del rumore	44
4.5 Creazione dei dataset	46
5 Sperimentazione e analisi dei modelli	51
5.1 Implementazione modelli	51
5.2 Allenamento	59
5.3 Valutazioni	66
5.4 Fine Tuning	68
5.5 Ottimizzazione degli iperparametri	72
5.6 Fine tuning con iperparametri ottimizzati	76
5.7 Conclusioni	81
A Confronto visivo tra modelli	83

B Risultati Fine Tuning	95
B.1 Pre Fine Tuning	95
B.2 Post Fine Tuning	99
C Risultati con Iperparametri ottimizzati	105
C.1 Training iniziale con iperparametri ottimizzati	105
C.2 Risultati Fine Tuning con iperparametri ottimizzati	110
D Ringraziamenti	113

Elenco delle figure

1.1	Esempi di rumore sulla stessa immagine	14
1.2	Esempio di rumore speckle in un'immagine ad ultrasuoni.	14
1.3	Esempio di artefatti di compressione in un'immagine JPEG.	15
1.4	Diagramma schematico di un kernel per convoluzione con filtro gaussiano. Mostra come i pesi centrali contribuiscono maggiormente al valore del nuovo pixel. .	16
1.5	Esempio di funzionamento del filtro mediano.	16
2.1	Architettura della rete RIDNet, che mostra i principali blocchi costitutivi e il flusso di dati attraverso la rete.	22
2.2	Architettura della DnCNN, che mostra i principali blocchi costitutivi e il flusso di dati attraverso la rete.	23
2.3	Architettura dell'Autoencoder, che mostra i principali blocchi costitutivi e il flusso di dati attraverso la rete.	24
3.1	Esempi di immagini del dataset BSD500, contenenti scene naturali e urbane ad alta qualità.	26
3.2	Esempi di immagini ad alta risoluzione dal dataset DIV2K. Mostrano la varietà semantica e la qualità visiva tipica del dataset.	27
3.3	Esempi di immagini RAW del dataset MIT-Adobe FiveK.	27
3.4	Esempi di immagini dataset SIDD, acquisite con smartphone in condizioni reali di illuminazione.	28
3.5	Ogni soggetto ha molteplici foto con diverse ISO.	28
3.6	In più per ogni soggetto+iso esiste una coppia gt-noisy.	28
3.7	Esempi dal dataset RENOIR	29
3.8	Esempi di immagini dal dataset KVASIR. Le immagini mostrano polipi e altre anomalie del tratto gastrointestinale, acquisite in contesti clinici reali. (non so se aggiungerle sono molto brutte)	30
4.1	Esempio di padding sul dataset BSD500.	40
4.2	Esempio di layout immagini	44
4.3	Esempio di aggiunta del rumore	45
5.1	Andamento delle curve di addestramento della RIDNet.	62
5.2	Andamento delle curve di addestramento di DnCNN.	63
5.3	Andamento delle curve di addestramento dell'Autoencoder.	63
5.4	Andamento delle curve di fine tuning della Ridnet.	70
5.5	Andamento delle curve di fine tuning della DnCNN.	70
5.6	Andamento delle curve di finetuning dell'Autoencoder.	70
A.1	Dataset renoir	84
A.2	Dataset sidd	85

A.3	Dataset bsd– Livello di rumore $\sigma = 15$.	86
A.4	Dataset bsd– Livello di rumore $\sigma = 25$.	87
A.5	Dataset bsd– Livello di rumore $\sigma = 50$.	88
A.6	Dataset fivek– Livello di rumore $\sigma = 15$.	89
A.7	Dataset fivek– Livello di rumore $\sigma = 25$.	90
A.8	Dataset fivek– Livello di rumore $\sigma = 50$.	91
A.9	Dataset div2k– Livello di rumore $\sigma = 15$.	92
A.10	Dataset div2k– Livello di rumore $\sigma = 25$.	93
A.11	Dataset div2k– Livello di rumore $\sigma = 50$.	94
B.1	Kvasir – Livello di rumore $\sigma = 15$.	96
B.2	Kvasir – Livello di rumore $\sigma = 25$.	97
B.3	Kvasir – Livello di rumore $\sigma = 50$.	98
B.4	Kvasir – Livello di rumore $\sigma = 15$.	100
B.5	Kvasir – Livello di rumore $\sigma = 25$.	101
B.6	Kvasir – Livello di rumore $\sigma = 50$.	102
B.7	Kvasir – Confronto BM3D ai diversi livelli di rumore.	104
C.1	Risultati della Ridnet con iperparametri ottimizzati	106
C.2	Dataset BSD500 – Confronto Ridnet Student ai diversi livelli di rumore.	107
C.3	Dataset MIT Adobe FiveK – Confronto Ridnet Student ai diversi livelli di rumore.	108
C.4	Dataset Div2K – Confronto Ridnet Student ai diversi livelli di rumore.	109
C.5	Dataset Kvasir pre Fine tuning – Confronto Ridnet Student ai diversi livelli di rumore.	111
C.6	Dataset Kvasir post Fine tuning – Confronto Ridnet Student ai diversi livelli di rumore.	112

Elenco dei listing

4.1	Conversione immagini raw a jpg	32
4.2	Estensioni immagini raw	32
4.3	Scansione ricorsiva cartelle	32
4.4	Lettura immagini con OpenCV	32
4.5	Rinomina file	33
4.6	Salvataggio immagine	33
4.7	Caricamento delle immagini di RENOIR	34
4.8	Mean Squared Error	34
4.9	Classificazione batch con MSE	34
4.10	Calcolo di MSE	35
4.11	Elaborazione dei risultati	35
4.12	Salvataggio della classificazione	35
4.13	Funzione principale per la suddivisione e classificazione in batch	36
4.14	Definizione della funzione per processare la directory	36
4.15	Step 1: Gestione Reference	37
4.16	Step 2: Gestione Noisy	37
4.17	Step 3: Aggiunta del prefisso Batch_XXX_	38
4.18	Step 4: Copia nella cartella principale	38
4.19	Utilizzo della funzione process_batch_folder	38
4.20	Costanti con directory per BSD	39
4.21	Funzione di padding	39
4.22	Creazione del padding sui lati dell'immagine	39
4.23	Definizione funzione di elaborazione immagini	40
4.24	Costanti per il patching	41
4.25	Padding per il patching	41
4.26	Funzione di estrazione patch dalle immagini	42
4.27	Calcolo dei passi e creazione patch	42
4.28	Ciclo di elaborazione delle immagini e verifica preliminare	43
4.29	Aggiunta di rumore da lista	44
4.30	Aggiunta del rumore da funzione	45
4.31	Definizione della funzione di creazione dei dataset	46
4.32	Calcolo degli indici di divisione	46
4.33	Definizione sottofunzione per caricare le immagini	47
4.34	Creazione dei dataset TensorFlow	47
4.35	Aggiunta del rumore nei dataset	48
4.36	Creazione del dataset rumoroso	48
5.1	Implementazione del blocco di base BasicBlock	51
5.2	Implementazione del blocco di base con sigmoide BasicBlockSig	52
5.3	Blocco Merge_Run_dual per l'estrazione multiscale	52
5.4	Blocco ResidualBlock con connessione residua	53

5.5	Blocco EResidualBlock con convoluzioni a gruppi	53
5.6	Layer MeanShift per normalizzazione dei valori dei pixel	53
5.7	Layer di attenzione sui canali (CALayer)	54
5.8	Blocco completo che unisce moduli multiscala, residui e attenzione	54
5.9	Costruzione dell'architettura completa RIDNet	54
5.10	Corpo della ridnet	55
5.11	Ultimo blocco della rete	55
5.12	Pezzo finale della rete	55
5.13	Costruzione della RIDNet	55
5.14	Compilazione del modello	56
5.15	Definizione della funzione build_DnCNN	56
5.16	Primo strato convoluzionale	56
5.17	Ciclo con blocchi ripetuti	57
5.18	Strato finale e output	57
5.19	Definizione e primo livello convoluzionale	57
5.20	Secondo livello convolutivo + pooling	58
5.21	Terzo livello convolutivo + upsampling	58
5.22	Strato finale dell'autoencoder	58
5.23	Creazione della Ridnet	59
5.24	Compilazione della Ridnet	59
5.25	Creazione delle callback	60
5.26	Compilazione della Ridnet	61
5.27	Caricamento e nuovo train della ridnet	61
5.28	Creazione, compilazione e training di DnCNN	62
5.29	Creazione, compilazione e training di un Autoencoder	63
5.30	Definizione di BM3D	64
5.31	Stima e denoising delle immagini con BM3D	65
5.32	Definizione di BM3D con rumore definito	65
5.33	Denoising con BM3D	66
5.34	Calcolo di PSNR	66
5.35	Definizione nuovo learning scheduler	68
5.36	Ottimizzatore e compilazione per fine tuning	69
5.37	Callback per ridurre il LR	69
5.38	Fine tuning dei modelli	69
5.39	Definizione della funzione obiettivo	73
5.40	Corpo funzione obiettivo	73
5.41	Lancio dello studio Optuna	74
5.42	Definizione della funzione obiettivo per DnCNN	75
5.43	Definizione della funzione obiettivo per Autoencoder	76
5.44	Definizione della classe Distiller	77
5.45	Compile di Distiller	77
5.46	Definizione della funzione per i cicli di train	77
5.47	Calcolo perdite modelli e bilanciamento	78
5.48	Calcolo gradiente Distiller	78
5.49	Aggiornamento delle metriche dello student	78
5.50	Definizione della funzione di validazione validation/test	79
5.51	Definizione della callback per Distiller	79
5.52	Definizione della callback per Distiller	79

Introduzione

La presente tesi si propone di esplorare l'uso delle reti neurali per il miglioramento della qualità delle immagini digitali, con particolare attenzione alla rimozione del rumore. Il lavoro si inserisce nel contesto dell'elaborazione delle immagini, un campo in continua evoluzione che trova applicazioni in ambiti diversi, dalle immagini naturali a quelle mediche.

Le immagini digitali acquisite sono spesso affette da rumore, ossia da disturbi visivi che degradano la qualità e la nitidezza dei dettagli. Le cause del rumore sono molteplici e legate sia a fattori esterni che tecnici. Ad esempio, in situazioni di scarsa illuminazione, il sensore della fotocamera cattura meno fotoni, aumentando così la variabilità casuale del segnale, che si manifesta come rumore. Per compensare la poca luce, si tende a utilizzare alti valori di ISO, che amplificano anche il rumore elettronico presente nel sensore. Inoltre, lunghe esposizioni possono far surriscaldare il sensore, causando rumore di crominanza, cioè variazioni di colore indesiderate. Anche la dimensione e la tecnologia del sensore hanno un ruolo cruciale: sensori più piccoli o meno avanzati sono più soggetti a generare rumore. Infine, il processo di conversione analogico-digitale e la compressione influenzano ulteriormente la presenza di rumore nel file immagine.

In questa tesi vengono approfonditi tre denoiser basati su reti neurali presenti in letteratura: RIDNet, DnCNN e un Autoencoder. Ciascuna di queste architetture è stata allenata con attenzione all'ottimizzazione degli iperparametri e valutata su diversi dataset. I risultati ottenuti sono stati confrontati per comprendere se sia possibile individuare una rete neurale migliore per il denoising o se la scelta dipenda dal tipo di immagini considerate..

La tesi è così organizzata:

Il **Capitolo 1** fornisce una panoramica generale sul problema del rumore nelle immagini digitali. Vengono introdotte le tipologie e le cause principali del rumore, le origini dei disturbi e le metriche comunemente utilizzate per valutarne l'impatto. Successivamente, si presentano i metodi tradizionali di denoising, come i filtri spaziali e le tecniche nel dominio della frequenza, per poi discutere le innovazioni più recenti basate sulle reti neurali e altri approcci avanzati.

Il **Capitolo 2** analizza le principali architetture di reti neurali utilizzate per il denoising. Vengono discussi i principi generali di funzionamento, i vantaggi e le limitazioni di ciascun approccio, fornendo una base teorica per comprendere le scelte sperimentali dei capitoli successivi.

Il **Capitolo 3** descrive i dataset utilizzati per l'allenamento e la valutazione dei modelli, comprendendo sia immagini naturali sia immagini mediche. Ogni dataset viene presentato con le sue caratteristiche principali e le motivazioni per la sua scelta nell'ambito del denoising.

Il **Capitolo 4** illustra le fasi di elaborazione dei dataset, dalla conversione dei file RAW alla preparazione delle immagini, comprese operazioni di padding, patching e aggiunta controllata del rumore. Questa fase è fondamentale per garantire la qualità e la coerenza dei dati utilizzati durante l'allenamento.

Il **Capitolo 5** riguarda la sperimentazione e l'analisi dei modelli. Qui vengono descritti l'allenamento dei diversi modelli, lo studio degli iperparametri, le valutazioni quantitative e qualitative, e la fase di *fine-tuning* sui dataset medici per adattare i modelli a contesti specifici.

Capitolo 1

Denoising di immagini

In questo capitolo viene fornita una panoramica generale sul problema del rumore nelle immagini digitali. Si introducono le tipologie e le cause principali del rumore, le origini dei disturbi e le metriche comunemente utilizzate per valutarne l'impatto. Successivamente, si presentano i metodi tradizionali di denoising, come i filtri spaziali e le tecniche nel dominio della frequenza, per poi discutere le innovazioni più recenti basate sulle reti neurali e altri approcci avanzati.

1.1 Definizione del problema

Il denoising delle immagini rappresenta una problematica centrale nell'ambito dell'elaborazione digitale delle immagini. Si tratta di un processo volto a rimuovere il rumore che si introduce durante la cattura, la trasmissione o la memorizzazione di un'immagine digitale, con l'obiettivo di recuperare una rappresentazione più fedele e pulita del contenuto visivo originale.

La presenza di rumore può compromettere significativamente la qualità visiva e l'accuratezza di analisi successive, come il riconoscimento di oggetti o la segmentazione. Formalmente, il problema può essere modellato come:

$$y = x + n$$

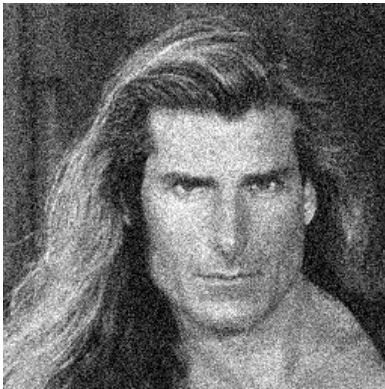
dove y è l'immagine osservata contenente rumore, x è l'immagine pulita da recuperare e n rappresenta il rumore additivo.

Tipologie e Cause di Rumore nelle Immagini Digitali

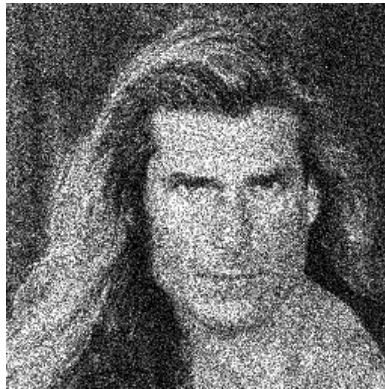
Il rumore presente nelle immagini può assumere diverse forme, a seconda della sorgente e delle condizioni di acquisizione. I principali modelli e le relative cause sono:

- **Rumore gaussiano additivo (AWGN):** spesso usato come modello ideale per rumori sommatoria di più sorgenti e per molte analisi teoriche. Nelle immagini acquisite, una componente a distribuzione approssimativamente gaussiana deriva da disturbi elettronici del circuito di acquisizione (read noise, thermal noise). Per il rumore reale dei sensori si usa frequentemente il modello Poisson–Gaussiano (shot noise + rumore elettronico) e metodi specifici per la sua stima. (Janesick 2007; Foi et al. 2008)
- **Rumore di tipo Poisson (shot noise):** dovuto alla natura discreta dei fotoni rilevati dal sensore; la varianza è proporzionale al segnale stesso. Tecniche come la trasformazione di Anscombe e i suoi sviluppi (e.g. inversioni ottimali) sono largamente usate per stabilizzare la varianza prima del denoising. (Mäkitalo e Foi 2013; Foi et al. 2008)

- **Rumore impulsivo ("salt-and-pepper"):** si manifesta come singoli pixel con valori estremi (0 o massimo) e tipicamente è causato da errori di trasmissione, cellule pixel difettose o errori A/D; esistono filtri switch, mediani adattivi e approcci più recenti (SVM, reti profonde) dedicati alla sua rimozione. (Gómez-Moreno et al. 2014)



(a) Rumore Gaussiano additivo (AWGN)



(b) Poisson (Shot Noise)



(c) Rumore impulsivo (Salt & Pepper)

Figura 1.1: Esempi di rumore sulla stessa immagine.

- **Rumore speckle (moltiplicativo):** caratteristico dei sistemi coerenti (SAR, ultrasuoni, laser), ha natura moltiplicativa e statistiche particolari; richiede metodi di despeckling specifici che tengano conto della natura non additiva del disturbo. (Goodman 1976; Rajamani e Krishnaveni 2014)

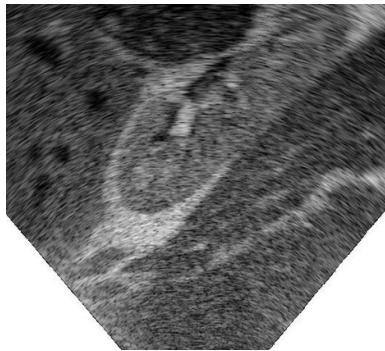


Figura 1.2: Esempio di rumore speckle in un'immagine ad ultrasuoni.

- **Artefatti di compressione (lossy):** la compressione con perdita (e.g. JPEG) introduce blocchi, ringing e perdita di dettaglio ad alta frequenza; questi disturbi non sono "rumore" stocastico puro ma effetti deterministici dell'analisi quantizzazione/trasformata che richiedono correzione mirata. (Wallace 1991; Eschbach 1997)

Origini del rumore

Le principali origini fisiche e operative del rumore nelle immagini sono:

- **Fenomeni fotonici e statistica di conteggio (shot noise):** il processo di conteggio fotonico genera rumore Poissoniano la cui intensità dipende dal livello di illuminazione; nei sistemi a basso conteggio questo è dominante. (Foi et al. 2008; Janesick 2007)



Figura 1.3: Esempio di artefatti di compressione in un'immagine JPEG.

- **Rumore elettronico del sensore (read noise, dark current, quantizzazione):** deriva dai circuiti di lettura (amplificatori, ADC), dal dark current termico e da imperfezioni di fabbricazione; la loro combinazione spesso giustifica il modello Poisson-Gaussiano per i dati raw. (Janesick 2007; Foi et al. 2008)
- **Errori di trasmissione e memorizzazione:** perdite o bit-flip durante la trasmissione e settori di memoria corrotti possono introdurre rumore impulsivo o artefatti localizzati (salt-and-pepper). (Gómez-Moreno et al. 2014)
- **Interferenze ambientali e condizioni di acquisizione:** scarsa illuminazione, elevata temperatura, luce dispersa e vibrazioni incrementano il rumore e possono cambiare la sua natura (es. aumento di dark current o rumore termico). (Janesick 2007)
- **Processi di compressione e post-elaborazione:** quantizzazione e compressione con perdita introducono artefatti strutturati (blocchi, ringing) che spesso vengono trattati con tecniche di post-processing specifiche, non con semplici filtri statistici. (Wallace 1991; Eschbach 1997)

Metodi classici

I primi approcci al denoising delle immagini si basavano su filtri spaziali, che agiscono direttamente sui pixel nell'intorno locale dell'immagine.

Tra i filtri lineari, i più comuni sono il filtro medio e il filtro gaussiano, che agiscono medianando i valori dei pixel in un intorno locale. Sebbene semplici, tendono a sfocare i dettagli fini dell'immagine (Gonzalez e Woods 2018). Il filtro gaussiano, in particolare, utilizza un kernel a forma di campana per pesare i pixel vicini, preservando meglio i bordi rispetto al filtro medio.

- **Filtro medio:** sostituisce ogni pixel con la media dei pixel circostanti, semplice ma tende a sfocare i bordi (Gonzalez e Woods 2018).

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 8 & 6 \\ 7 & 5 & 9 \end{bmatrix} \rightarrow 1 + 2 + 3 + 4 + 6 + 7 + 5 + 9 = 37 \rightarrow \frac{37}{8} = 4.625 \rightarrow \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 5 & 9 \end{bmatrix}$$

- **Filtro gaussiano:** convoluzione con kernel gaussiano pesato, preserva meglio i bordi (Gonzalez e Woods 2018).

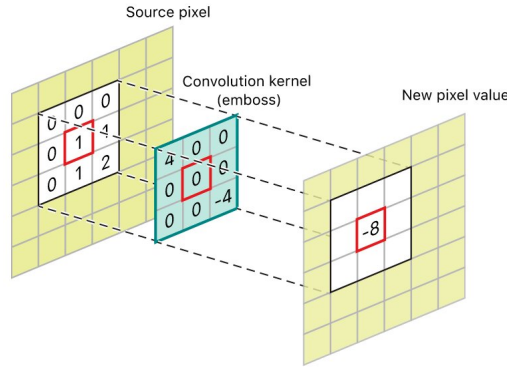


Figura 1.4: Diagramma schematico di un kernel per convoluzione con filtro gaussiano. Mostra come i pesi centrali contribuiscono maggiormente al valore del nuovo pixel.

Per ovviare a questo problema, sono stati sviluppati filtri non lineari, come il filtro mediano, efficace nel rimuovere rumore impulsivo (è dimostrato che mantiene i bordi meglio dei filtri lineari per rumore moderato (Arias-Castro e Donoho 2009)), e il filtro bilaterale, che preserva i bordi mediante un'azione locale-spaziale e di intensità (Tomasi e Manduchi 1998; Paris et al. 2008).

- **Filtro mediano:** robusto al rumore impulsivo, efficace soprattutto con rumore sale e pepe (T. S. Huang e Tang 1979).

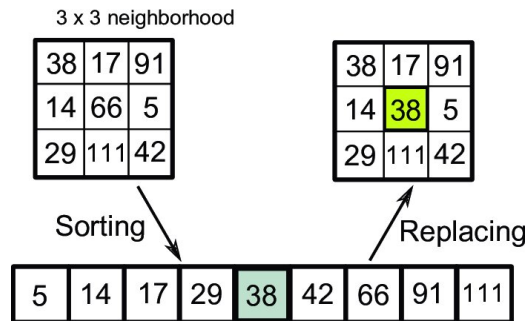


Figura 1.5: Esempio di funzionamento del filtro mediano.

Miglioramenti ulteriori si sono ottenuti con la trasformata wavelet: Donoho e Johnstone hanno introdotto il denoising via soglia nei coefficienti wavelet ("wavelet shrinkage") (Donoho 1995), che costituisce una pietra miliare. Ulteriori lavori ne hanno studiato variazioni e parametri di soglia ottimali (Chambolle et al. 2003; Denoising 2020).

Modelli Statistici e Filtri Non Locali

Un progresso significativo è stato portato dai modelli statistici e dai filtri basati sulla similarità delle patch:

- **Markov Random Fields (MRF)**: modelli probabilistici che catturano la dipendenza tra pixel vicini, permettendo di formulare il denoising come un problema di inferenza bayesiana (S. Geman e D. Geman 1984).
- **Non-Local Means (NLM)** (Buades, Coll e J.-M. Morel 2005): sfrutta la ridondanza strutturale delle immagini, mediando patch simili non necessariamente vicine nello spazio. Questo approccio ha dimostrato di preservare meglio i dettagli rispetto ai filtri locali (Buades, Coll e J. Morel 2005).
- **BM3D (Block Matching 3D)** (Dabov et al. 2007): raggruppa patch simili in domini tridimensionali ed applica trasformate collaborative, con risultati allo stato dell'arte per lungo tempo (H. Wang et al. 2020).

Metodi come **Non-Local Means** (NLM) e **BM3D** si basano sul concetto di similitudine tra porzioni (patch) di immagine. Questi algoritmi ricercano blocchi simili all'interno dell'immagine per effettuare una media ponderata più efficace.

BM3D, in particolare, è considerato uno dei migliori metodi non-apprendenti ed è tuttora in uso.

Sparse Coding e Dictionary Learning

La rappresentazione sparsa ha introdotto un paradigma potente per il denoising delle immagini, fondato sull'assunzione che le patch possano essere rappresentate come combinazioni lineari di pochi atomi selezionati da un dizionario (over-complete). Il modello standard si esprime attraverso:

$$\hat{\alpha} = \arg \min_{\alpha} \|y - D\alpha\|_2^2 + \lambda \|\alpha\|_1$$

dove D è il dizionario e α il vettore dei coefficienti sparsi (Fan et al. 2019; H. Wang et al. 2020).

- **K-SVD**: algoritmo iterativo che alterna sparse coding e aggiornamento di atomi attraverso SVD, apprendendo un dizionario ottimale per il denoising (Lebrun e Leclaire 2012; Wikipedia contributors 2025).
- **Migliorie recenti**: versioni avanzate come il $K\text{-SVD}_P$, che applica algoritmi primal-dual per migliorare efficacia e velocità con rumore elevato (Xiao et al. 2020).
- **Approcci basati su rete neurale**: reti come SRNet integrano il framework di sparse coding in reti CNN multiscala, accelerando l'ottimizzazione e migliorando la qualità, soprattutto nei dettagli testurizzati (Sheng et al. 2022).
- **Sparse coding convoluzionale**: estende il modello a dizionari convoluzionali adatti a strutture locali ripetute; ad oggi è all'origine di architetture on-line più efficienti (Garcia-Cardona e Wohlberg 2018; Y. Zhang et al. 2022).
- **Denoising cooperativo distribuito**: proposte come *Cloud K-SVD* permettono l'apprendimento di dizionari condivisi tra nodi, preservando qualità e generalizzazione (Lillelund, Jensen e Pedersen 2022).

Questi approcci modellano ogni patch come una combinazione sparsa di elementi di un dizionario. Algoritmi come **K-SVD** apprendono un dizionario adattivo dai dati per una rappresentazione più efficiente. Sebbene più costosi computazionalmente, offrono buoni risultati su immagini naturali e sono particolarmente efficaci nel preservare dettagli locali rispetto ai filtri classici.

La rappresentazione sparsa ha rappresentato un passo fondamentale nell'evoluzione dei metodi di denoising, introducendo concetti di apprendimento adattivo e modellazione statistica dei dati, aprendo la strada ai successivi approcci basati su reti neurali.

Denoising nel dominio della frequenza

Mediante la trasformata di Fourier è possibile separare le componenti ad alta frequenza (spesso associate al rumore) da quelle a bassa frequenza (contenuto strutturale) (Oppenheim e Schaffer 1999). Questo approccio è utile in presenza di rumore periodico, ma tende a sacrificare dettagli fini.

Tra i suoi principali limiti vi sono la limitata adattabilità al contenuto specifico dell'immagine, il compromesso necessario tra la riduzione del rumore e la conservazione dei dettagli, e la ridotta efficacia nel trattamento di rumori non uniformi o strutturati.

Metodi Basati su Apprendimento Automatico

L'avvento del machine learning ha rivoluzionato il denoising delle immagini:

- **Reti neurali convoluzionali (CNN):** modelli come DnCNN (K. Zhang, Zuo, Chen et al. 2017) e FFDNet (K. Zhang, Zuo e L. Zhang 2018) hanno mostrato notevoli miglioramenti nella rimozione del rumore.
- **Metodi auto-supervisionati:** Noise2Noise (Lehtinen et al. 2018) permette di addestrare modelli senza immagini pulite di riferimento.
- **Real Image Denoising:** metodi come RIDNet (Anwar e Barnes 2019) introducono meccanismi di attenzione per gestire meglio il rumore reale catturato dalle fotocamere.

Più recentemente, modelli basati su *transformer* hanno ampliato la capacità di catturare relazioni non locali e migliorato la generalizzazione.

Metodi basati sull'ottimizzazione di modelli variazionali

I modelli variazionali per il problema del denoising delle immagini rappresentano un approccio matematico e computazionale basato sull'ottimizzazione di funzionali definiti per stimare un'immagine pulita a partire da una versione rumorosa. Tali modelli sono spesso formulati come problemi di minimizzazione che bilanciano due termini principali: un termine di fedeltà ai dati osservati, che vincola la soluzione affinché resti simile all'immagine rumorosa di input, e un termine regolarizzante, che favorisce soluzioni "lisce" o con proprietà desiderate, come la preservazione dei bordi.

Uno dei metodi più noti è la regolarizzazione a variazione totale (**Total Variation denoising**, TV denoising), introdotta da Rudin, Osher e Fatemi nel 1992 (modello ROF) (Rudin, Osher e Fatemi 1992). Questo modello riduce il rumore minimizzando la variazione totale dell'immagine, ovvero la somma delle variazioni assolute dei pixel, mantenendo però la somiglianza con l'immagine originale. Tale approccio è efficace perché elimina il rumore preservando

simultaneamente i contorni netti e gli spigoli dell'immagine, a differenza di filtraggi più semplici che tendono a sfocare i dettagli.

1.2 Denoising di Immagini tramite Deep Learning

Dalla metà degli anni 2010, l'adozione di modelli convoluzionali ha portato a un significativo salto di qualità nel campo del denoising delle immagini. Le reti neurali convoluzionali (**CNN**) sono state tra le prime architetture profonde ad essere utilizzate con successo in questo ambito, grazie alla loro capacità di apprendere direttamente la mappatura da immagini rumorose a immagini pulite. Un esempio è **DnCNN**(K. Zhang, Zuo, Chen et al. 2017), che ha introdotto il concetto di residual learning, permettendo alla rete di concentrarsi specificamente sulla componente di rumore da rimuovere.

Successivamente, modelli più flessibili come **FFDNet**(K. Zhang, Zuo e L. Zhang 2018) hanno esteso queste capacità introducendo la possibilità di gestire livelli variabili di rumore in ingresso, migliorando così l'efficienza e la generalizzazione del modello. Contestualmente, sono cominciati a emergere approcci che integravano meccanismi di attenzione per focalizzarsi in modo selettivo sulle aree più degradate dell'immagine.

Parallelamente, l'approccio auto-supervisionato ha rivoluzionato il settore con modelli come **Noise2Noise**(Lehtinen et al. 2018), che hanno dimostrato come fosse possibile addestrare reti capaci di ripulire immagini senza la necessità di avere immagini pulite di riferimento, affidandosi invece a molteplici immagini rumorose della stessa scena.

Più recentemente, architetture basate su U-Net, reti generative avversarie (**GAN**) e transformer hanno ulteriormente migliorato le prestazioni, anche in presenza di rumori più complessi e in scenari diversi(Anwar e Barnes 2019). Questi modelli riescono a generalizzare meglio e ad apprendere relazioni non locali grazie alla loro struttura e ai meccanismi di attenzione incorporati.

1.3 Metriche di Valutazione

Per misurare l'efficacia dei metodi di denoising si considerano:

- **PSNR (Peak Signal-to-Noise Ratio)**: rappresenta il rapporto tra il valore massimo possibile del segnale e il rumore presente(Hore e Ziou 2010), espresso tramite errore quadratico medio (MSE).

Definito come:

$$PSNR = 20 \log_{10} \left(\frac{MAX_f}{\sqrt{MSE}} \right)$$

dove MAX_f è il valore massimo del pixel dell'immagine

MSE (Mean Square Error) è l'errore quadratico medio tra l'immagine originale e quella rumorosa.

$$MSE = \frac{1}{mn} \sum_0^{m-1} \sum_0^{n-1} ||f(i, j) - g(i, j)||^2$$

- **SSIM (Structural Similarity Index)**: valuta la similarità strutturale percepita, considerando luminanza, contrasto e struttura (Z. Wang et al. 2004). Calcolato come

$$SSIM(x, y) = \frac{(2\mu_x\mu_y + C_1)(2\sigma_{xy} + C_2)}{(\mu_x^2 + \mu_y^2 + C_1)(\sigma_x^2 + \sigma_y^2 + C_2)}$$

dove μ e σ sono media e deviazione standard, e C_1, C_2 sono costanti di stabilizzazione.

La valutazione qualitativa visiva rimane cruciale in contesti come quello medico o artistico (Buades, Coll e J.-M. Morel 2005; Dabov et al. 2007).

1.4 Riflessione sulla letteratura

Nonostante i risultati eccellenti ottenuti dai modelli di deep learning, permangono diverse sfide aperte. Tra queste, vi è l'adattamento a rumori complessi o misti che non corrispondono ai modelli sintetici utilizzati durante il training (H. Wang et al. 2020; H. Wang et al. 2020), l'addestramento in assenza di dati puliti o con dataset limitati, e l'applicazione in tempo reale a video e flussi di immagini, dove la complessità computazionale dei modelli può rappresentare un limite. Inoltre, l'integrazione di metodi di denoising con altri task di miglioramento delle immagini, come la super-risoluzione o il deblurring, costituisce una direzione di ricerca promettente.

L'analisi della letteratura evidenzia come il denoising delle immagini abbia conosciuto un'evoluzione significativa: dai filtri spaziali e trasformate classiche ai complessi modelli neurali. Ogni approccio presenta vantaggi e limiti specifici, e le ricerche più recenti tendono a combinare metodi classici e deep learning, oppure ad incorporare conoscenza specifica del dominio, per ottenere soluzioni più robuste e generalizzabili.

Capitolo 2

Analisi di reti neurali per il denoising di immagini

In questo capitolo vengono descritte le principali architetture di reti neurali utilizzate per il problema della rimozione del rumore dalle immagini (*image denoising*).

La scelta delle reti è stata effettuata in base alla loro diffusione nella letteratura scientifica, alla semplicità di implementazione e alla capacità di rappresentare approcci diversi allo stesso problema.

Negli ultimi decenni sono stati proposti sia metodi tradizionali di denoising, come il *Non-Local Means* (Buades, Coll e J.-M. Morel 2005; Buades, Coll e J. Morel 2005) e il celebre algoritmo BM3D (Dabov et al. 2007), sia approcci di tipo *learning-based*, che hanno rapidamente superato i limiti delle tecniche classiche.

In particolare, le reti convoluzionali hanno introdotto una vera e propria rivoluzione, garantendo migliori prestazioni sia su rumori sintetici che su scenari più realistici (K. Zhang, Zuo, Chen et al. 2017; K. Zhang, Zuo e L. Zhang 2018; Lehtinen et al. 2018; Batson e Royer 2019).

Le tre architetture analizzate in questa tesi sono:

- **RIDNet** - una rete moderna e avanzata, specificamente progettata per il denoising di immagini reali;(Anwar e Barnes 2019)
- **DnCNN** - un modello pionieristico che ha introdotto l'idea del denoising come stima del residuo;(K. Zhang, Zuo, Chen et al. 2017)
- **Autoencoder** - un'architettura generica, versatile e adattabile a diversi scenari di apprendimento non supervisionato.(Vincent et al. 2008)

Di tutte le reti è stato utilizzato il codice originale, implementato in Python utilizzando Keras con backend Tensor-Flow fornito dagli autori. Esso è stato oggetto di un'attenta analisi approfondita per comprenderne dettagli e funzionamento.

2.1 RIDNet

La **Residual Image Denoising Network** (RIDNet) rappresenta uno dei modelli più moderni e robusti per la riduzione del rumore in immagini naturali (Abdelhamed, Timofte, Brown et al. 2019).

Il suo principale punto di forza è la capacità di gestire rumori **reali**, come quelli provenienti dai sensori fotografici digitali, che non seguono distribuzioni statistiche ideali come il rumore gaussiano additivo (Janesick 2007; Foi et al. 2008; Mäkitalo e Foi 2013).

L'architettura è basata su *blocchi residui* e meccanismi di attenzione che consentono di bilanciare in maniera ottimale la riduzione del rumore e la preservazione dei dettagli fini. In particolare:

- l'uso di **connessioni residue** facilita il flusso dell'informazione, riducendo la perdita di dettaglio e migliorando la stabilità dell'addestramento, sulla scia delle reti ResNet (He et al. 2016);
- l'integrazione di **attenzione spaziale e di canale** permette di focalizzare l'elaborazione sulle regioni e caratteristiche più rilevanti per il denoising, riducendo al minimo la rimozione di dettagli utili (Hu, Shen e Sun 2018);
- la progettazione modulare rende RIDNet scalabile e adattabile a immagini di diversa risoluzione e qualità.

Grazie a queste caratteristiche, RIDNet è considerata uno standard di riferimento per il denoising in scenari realistici, superando molti dei limiti dei metodi classici basati su rumori sintetici (Dabov et al. 2007; Buades, Coll e J.-M. Morel 2005).

RIDNET ha un'architettura modulare e si articola in tre componenti principali: estrazione delle feature, moduli di apprendimento residuo e ricostruzione dell'immagine.

L'input iniziale passa attraverso uno strato di normalizzazione (**MeanShift**) che centra i dati, seguito da un blocco di convoluzioni base e da quattro blocchi principali, ognuno costituito da diversi sottoblocchi specializzati: **Merge Run Dual**, **ResidualBlock**, **EResidualBlock** e **CALayer** (Channel Attention Layer). Questi moduli lavorano sinergicamente per estrarre e affinare caratteristiche a diverse scale e frequenze spaziali.

Tutte le convoluzioni sono di dimensione 3×3 con 64 feature map, garantendo un bilanciamento ottimale tra profondità rappresentazionale e complessità computazionale. L'ultimo strato convoluzionale utilizza 3 filtri con padding "same" per preservare la dimensione spaziale dell'output. A questo segue un'operazione di normalizzazione inversa e la somma residua con l'immagine di input, implementando così un meccanismo di residual learning che facilita la convergenza del modello e riduce la perdita di informazioni.

La funzione di loss adottata è l'errore quadratico medio (MSE), calibrata per minimizzare la differenza pixel-wise tra l'immagine denoisata e l'immagine pulita di riferimento durante la fase di training.

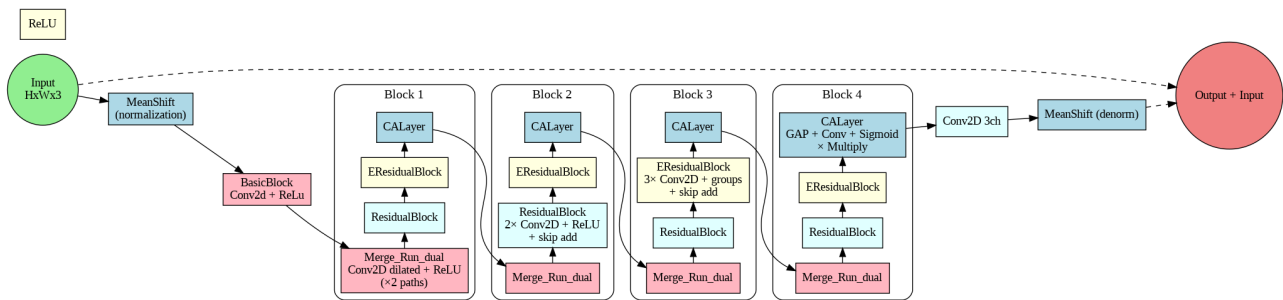


Figura 2.1: Architettura della rete RIDNet, che mostra i principali blocchi costitutivi e il flusso di dati attraverso la rete.

2.2 DnCNN

La **Denoising Convolutional Neural Network** (DnCNN) è stata introdotta da K. Zhang, Zuo, Chen et al. 2017 come una delle prime reti profonde dedicate al denoising di immagini corrotte da rumore gaussiano additivo. La sua innovazione principale è quella di formulare il problema non come una mappatura diretta tra immagine rumorosa e immagine pulita, ma come la stima del **residuo** (ossia il rumore stesso).

Il funzionamento di DnCNN può essere riassunto in tre fasi:

1. riceve in input un'immagine rumorosa;
2. apprende a predire la componente di rumore associata a quell'immagine;
3. sottrae il rumore stimato dall'immagine originale, ottenendo un risultato denoised.

L'architettura utilizza più strati convoluzionali seguiti da *batch normalization* e funzioni di attivazione ReLU, che permettono di estrarre rappresentazioni sempre più complesse. Un aspetto chiave è la capacità di generalizzare a diversi livelli di rumore, senza dover essere riaddestrata per ogni condizione.

Per queste ragioni, DnCNN è ampiamente utilizzata come **baseline sperimentale** in numerosi studi successivi (K. Zhang, Zuo e L. Zhang 2018; Lehtinen et al. 2018; Batson e Royer 2019), e ha posto le basi per una nuova generazione di metodi di denoising basati su reti neurali profonde.

La rete DnCNN è stata implementata in Python utilizzando Keras con backend TensorFlow ed è composta da 20 strati convoluzionali: il primo strato è seguito da un'attivazione ReLU, i 17 strati successivi sono ciascuno seguiti da normalizzazione batch e attivazione ReLU, e infine un ultimo strato convoluzionale per la stima del rumore. Tutte le convoluzioni utilizzano filtri 3×3 e 64 feature map, mentre l'ultimo strato ha 3 filtri senza funzione di attivazione, finalizzati a stimare il rumore presente nell'immagine.

L'output della rete è ottenuto sottraendo la stima del rumore dall'immagine originale, implementando così una strategia di residual learning volta a facilitare la convergenza e migliorare la qualità del denoising. Anche in questo caso la funzione loss è MSE tra immagine pulita e immagine ricostruita dalla rete.

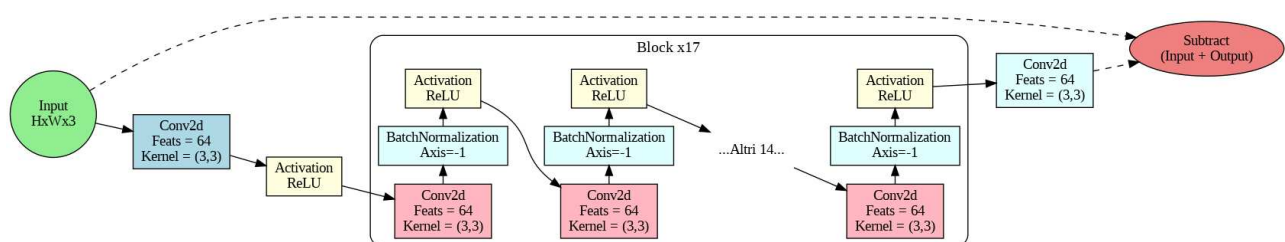


Figura 2.2: Architettura della DnCNN, che mostra i principali blocchi costitutivi e il flusso di dati attraverso la rete.

2.3 Autoencoder

Gli **autoencoder** rappresentano un'architettura più generica, non progettata specificamente per il denoising, ma ampiamente utilizzata a tale scopo grazie alla loro capacità di apprendere rappresentazioni compatte dei dati (Goodfellow, Bengio e Courville 2016). Un autoencoder è composto da due blocchi principali:

- un **encoder**, che riduce progressivamente la dimensione dell'immagine di input, comprimendola in uno spazio latente a bassa dimensionalità;
- un **decoder**, che tenta di ricostruire l'immagine originale a partire dalla rappresentazione latente.

Nel contesto del denoising, l'autoencoder sfrutta il fatto che il processo di compressione tende a eliminare le variazioni casuali non rilevanti (come il rumore), mentre il decoder è addestrato a ricostruire le strutture coerenti dell'immagine (Vincent et al. 2008).

Esistono due principali approcci:

1. **Denoising Autoencoder (DAE)** – addestrati esplicitamente fornendo in input immagini rumorose e in output le corrispondenti immagini pulite (Vincent et al. 2008);
2. **Variational Autoencoder (VAE)** – che sfruttano una formulazione probabilistica per modellare la distribuzione delle immagini (Kingma e Welling 2013), con la possibilità di generare nuove istanze coerenti e prive di rumore.

Sebbene le prestazioni di base degli autoencoder siano inferiori rispetto a modelli più specializzati come RIDNet o DnCNN, essi costituiscono un approccio estremamente flessibile, utile per la didattica e come punto di partenza per architetture più complesse (es. GAN (Goodfellow, Pouget-Abadie et al. 2014) e modelli basati su Transformer (Dosovitskiy et al. 2021)).

L'autoencoder proposto è stato implementato in Python utilizzando Keras con backend TensorFlow. La rete è costituita da cinque strati convoluzionali principali, intervallati da due operazioni di MaxPooling e due di UpSampling, per un totale di cinque livelli convoluzionali. Ogni convoluzione utilizza filtri di dimensione 3×3 con 64 feature map tranne l'ultimo strato, che ha 3 filtri per ricostruire l'immagine finale. L'ultimo strato applica una convoluzione con attivazione ReLU e padding same. La funzione di perdita utilizzata è l'errore quadratico medio (MSE).

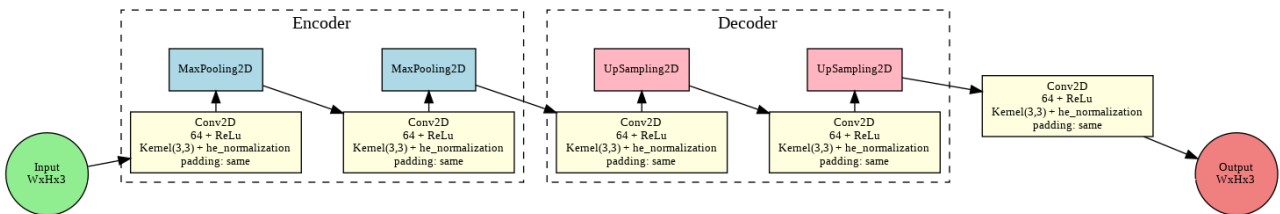


Figura 2.3: Architettura dell'Autoencoder, che mostra i principali blocchi costitutivi e il flusso di dati attraverso la rete.

Capitolo 3

Descrizione dei dataset

L'efficacia di qualunque modello di visione artificiale dipende in larga misura dalla qualità, dalla varietà e dalla rappresentatività dei dati su cui viene addestrato. Per questo motivo, in questo lavoro sono stati impiegati cinque dataset pubblici ampiamente adottati nella comunità scientifica. I dataset selezionati sono stati suddivisi in due gruppi: il primo comprende tre dataset **BSD500** (Martin et al. 2001), **DIV2K** (Ignatov, Timofte et al. 2019) e **MIT-Adobe FiveK** (Bychkovsky et al. 2011), a cui è stato applicato rumore artificiale; il secondo gruppo include due dataset con rumore reale, **SIDD** (Abdelhamed, Timofte, Brown et al. 2019) e **RENOIR** (Anaya e Barbu 2014).

Nello specifico, i dataset sono stati selezionati per coprire scenari diversi:

- **BSD500** – immagini naturali utilizzate per segmentazione e denoising;
Sito: <https://www.kaggle.com/datasets/kavithak1388/bsd500original>;
- **DIV2K** – immagini ad alta risoluzione per il task di super-risoluzione;
Sito: <https://data.vision.ee.ethz.ch/cvl/DIV2K>;
- **MIT-Adobe FiveK** – dataset per il miglioramento automatico delle immagini (*image enhancement*);
Sito: <https://data.csail.mit.edu/graphics/fivek/>;
- **SIDD** – immagini reali da smartphone per denoising in condizioni di acquisizione realistiche;
Sito: <https://www.eecs.yorku.ca/~kamel/sidd/>;
- **RENOIR** – immagini con rumore reale in scenari a bassa illuminazione, acquisite con DSLR e smartphone.
Sito: <https://ani.stat.fsu.edu/~abarbu/Renoir.html>.
- **KVASIR** – immagini mediche endoscopiche per la rimozione del rumore in ambito sanitario.
[urlhttps://datasets.simula.no/kvasir/](https://datasets.simula.no/kvasir/).

Come dataset per il training, la validazione e il test delle reti neurali sono stati utilizzati BSD500, DIV2K e MIT-Adobe FiveK, SIDD e Renoir unificati in un unico dataset utilizzando Python e TensorFlow. Mentre Kvasir è stato utilizzato per il fine tuning su immagini mediche.

3.1 BSD500

Il **Berkeley Segmentation Dataset 500** (BSD500) è un benchmark classico nell'ambito della visione artificiale, introdotto originariamente per la valutazione di algoritmi di segmentazione semantica (Martin et al. 2001). Contiene un totale di 500 immagini naturali ad alta qualità, suddivise in tre insiemi: 200 per l'addestramento, 100 per la validazione e 200 per il test.

Ogni immagine è accompagnata da annotazioni di segmentazione manuali, fornite da più utenti, permettendo di confrontare segmentazioni automatiche con la percezione umana. Nonostante il suo scopo originario non fosse il denoising o la super-risoluzione, la qualità visiva e la varietà semantica hanno reso BSD500 un dataset ampiamente impiegato anche in questi task (K. Zhang, Zuo, Gu et al. 2017).

Il dataset è caratterizzato da immagini naturali di alta qualità, con una diversità semantica che comprende scene urbane, naturali e ambienti interni ed esterni, annotazioni dettagliate di segmentazione manuale e un suo largo impiego in compiti come il denoising, la super-risoluzione e la compressione delle immagini.

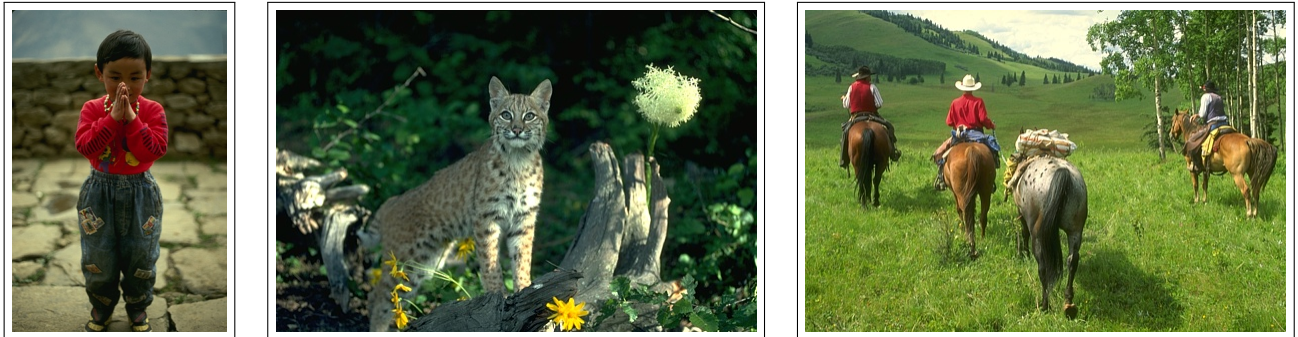


Figura 3.1: Esempi di immagini del dataset BSD500, contenenti scene naturali e urbane ad alta qualità.

3.2 DIV2K

Il dataset **DIV2K** (DIVERse 2K resolution images) è uno dei benchmark principali per la valutazione di algoritmi di super-risoluzione (Ignatov, Timofte et al. 2019). Contiene 1000 immagini ad alta definizione (risoluzione 2K), suddivise in 800 immagini per training, 100 per validazione e 100 per test.

Le immagini, selezionate per garantire varietà semantica e alta qualità visiva, includono versioni degradate (LR – Low Resolution) ottenute tramite downscaling con diversi metodi, come bicubico, bicubico + blur e compressione JPEG. Ciò consente lo sviluppo e la comparazione di metodi avanzati di upscaling e denoising.

Il DIV2K si distingue per la disponibilità di immagini ad alta risoluzione 2K e versioni degradate a più scale ($\times 2$, $\times 3$, $\times 4$), ed è ampiamente utilizzato in challenge internazionali come NTIRE (Timofte, Agustsson e Van Gool 2017), grazie anche alla sua elevata diversità semantica e alla qualità complessiva delle immagini.



Figura 3.2: Esempi di immagini ad alta risoluzione dal dataset DIV2K. Mostrano la varietà semantica e la qualità visiva tipica del dataset.

3.3 MIT-Adobe FiveK

Il **MIT-Adobe FiveK** è un dataset progettato per il miglioramento automatico delle immagini (*image enhancement*) (Bychkovsky et al. 2011). Comprende 5000 fotografie in formato RAW scattate da diversi fotografi, ognuna accompagnata da cinque versioni ritoccate da esperti di fotoritocco. Questo fornisce un riferimento multiplo per la qualità “ideale” di un’immagine.

Il dataset è particolarmente utile per addestrare modelli che replicano interventi di editing umano su esposizione, contrasto, bilanciamento del bianco e saturazione, rappresentando uno standard di riferimento per lo sviluppo di algoritmi di image enhancement e personalizzazione dello stile.

Il dataset preso in considerazione è costituito soltanto dalle 5000 immagini RAW originali senza modifiche.

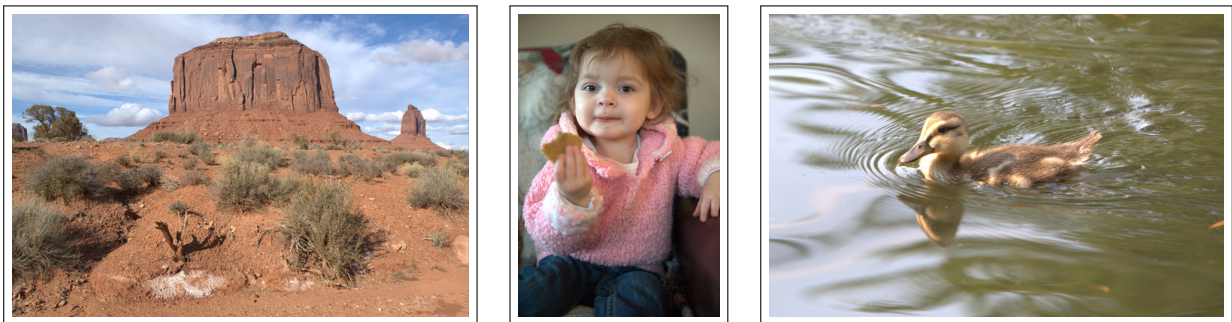


Figura 3.3: Esempi di immagini RAW del dataset MIT-Adobe FiveK.

3.4 SIDD

Il **Smartphone Image Denoising Dataset** (SIDD) è stato sviluppato per colmare il divario tra denoising su immagini sintetiche e denoising in condizioni reali (Abdelhamed, Timofte, Brown et al. 2019). Contiene immagini acquisite con dispositivi mobili che presentano rumore autentico, spesso derivante da condizioni di scarsa luminosità.

Ogni immagine “noisy” è accompagnata dalla corrispondente versione “clean”, ottenuta tramite esposizioni multiple o tecniche hardware di riduzione del rumore. Le immagini coprono diversi ambienti indoor e outdoor, con vari livelli ISO e illuminazione.

Il dataset è caratterizzato da acquisizioni da smartphone in condizioni reali, coppie noisy-clean, diversi livelli di ISO e condizioni di luce. È utile per lo sviluppo di modelli robusti al rumore reale.



Figura 3.4: Esempi di immagini dataset SIDD, acquisite con smartphone in condizioni reali di illuminazione.



Figura 3.5: Ogni soggetto ha molteplici foto con diverse ISO.

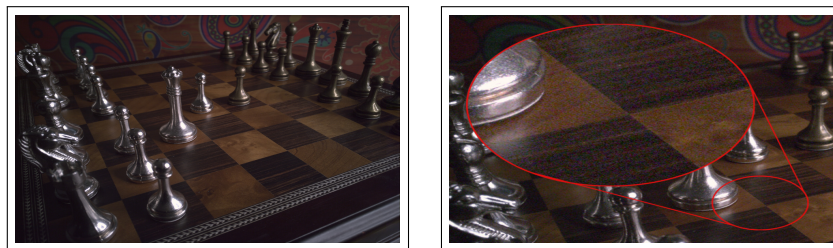


Figura 3.6: In più per ogni soggetto+iso esiste una coppia gt-noisy.

3.5 RENOIR

Il **RENOIR** (Real Noise Image Dataset) è stato progettato per studiare la rimozione di rumore reale in immagini fotografiche acquisite in scenari difficili (Anaya e Barbu 2014). Le immagini sono ottenute con DSLR e smartphone, catturando la stessa scena con esposizione breve (noisy) e lunga (clean), fornendo così una ground truth affidabile.

Una caratteristica distintiva di **RENOIR** è la varietà di rumori presenti (termico, elettronico, ISO elevato, compressione), che rende il dataset particolarmente utile per testare algoritmi di denoising in condizioni realistiche. Ogni soggetto è fotografato con tre dispositivi diversi (Canon EOS Rebel T3i, Canon PowerShot S90 e Xiaomi Mi3), aumentando ulteriormente la diversità del dataset e l'efficacia per valutare l'efficacia di algoritmi di denoising realistici.

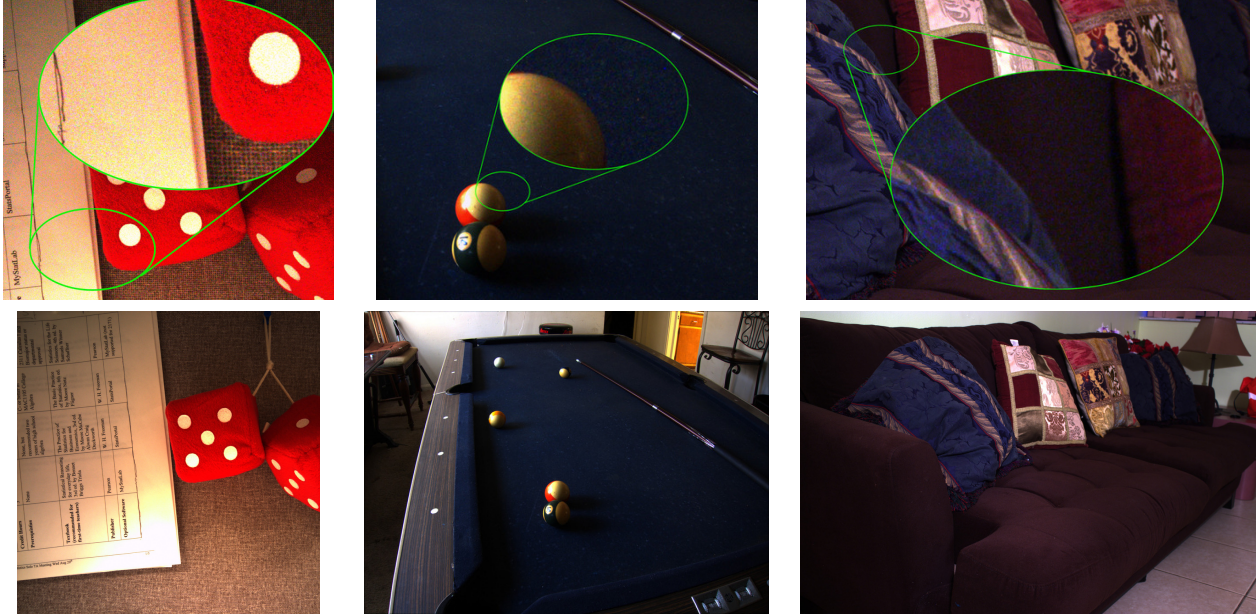


Figura 3.7: Esempi di immagini dal dataset **RENOIR**. La prima riga mostra immagini a breve esposizione (noisy), mentre la seconda riga le corrispondenti immagini a lunga esposizione (clean). Le colonne corrispondono ai tre dispositivi utilizzati nel dataset: **t3i**, **s90** e **mi3**.

3.6 KVASIR

Per il fine tuning dei modelli è stato impiegato il **Kvasir dataset** (Pogorelov et al. 2017), un insieme di dati che rappresenta un punto di riferimento nella ricerca applicata all'analisi automatica di immagini mediche endoscopiche.

Il dataset comprende 1.000 immagini ad alta risoluzione, raccolte in contesti clinici reali, che mostrano polipi, lesioni e altre anomalie del tratto gastrointestinale, caratterizzati da condizioni di illuminazione, angolazione e qualità visiva differenti, riproducendo fedelmente le difficoltà tipiche delle analisi endoscopiche reali.

Adatto per lo sviluppo di algoritmi di *computer-aided diagnosis* (CADx), il Kvasir dataset è stato ampiamente utilizzato per addestrare e valutare modelli di classificazione, rilevamento e segmentazione di patologie gastrointestinali.

Nel contesto di questa tesi, il dataset è stato utilizzato per valutare la capacità dei modelli di adattarsi a un dominio specifico, profondamente diverso rispetto alle immagini naturali comunemente usate per il training. L'obiettivo non è soltanto la riduzione del rumore visivo, ma anche la preservazione di dettagli clinicamente rilevanti, come bordi e texture dei polipi, che risultano fondamentali per una diagnosi accurata.



Figura 3.8: Esempi di immagini dal dataset KVASIR. Le immagini mostrano polipi e altre anomalie del tratto gastrointestinale, acquisite in contesti clinici reali. (non so se aggiungerle sono molto brutte)

Grazie a queste caratteristiche, il Kvasir dataset rappresenta non soltanto un banco di prova per le tecniche di denoising, ma anche un passo cruciale verso lo sviluppo di strumenti di supporto decisionale che possano essere realmente applicabili in ambienti clinici (Pogorelov et al. 2017).

Capitolo 4

Preprocessing dei Dataset

Questo capitolo descrive i dettagli tecnici relativi alla fase di preprocessing ed elaborazione dei dataset descritti nel capitolo precedente.

I vari dataset sono stati però manipolati prima di essere passati alle reti prendendo come riferimento l'esperimento di Real image denoising with feature attention (Anwar e Barnes 2019). Da ciascun dataset con immagini 4K sono state estratte 12 patch di dimensione 512x512, mentre per il dataset BSD500, che presenta immagini di risoluzione inferiore, è stato applicato un padding con specchiamento dei bordi, in modo da ottenere patch sempre di dimensione 512x512.

Il preprocessing delle immagini è un passaggio cruciale per la standardizzazione del dataset, soprattutto in contesti in cui le dimensioni delle immagini devono essere uniformi per essere elaborate da modelli convoluzionali. Le tre modalità implementate coprono i principali scenari applicativi, permettendo flessibilità tra conservazione del contenuto e adattamento geometrico.

- Conversione di immagini RAW in RGB jpg (per MIT-Adobe FiveK);
- Suddivisione in patch da 512x512
- padding fino a 512x512
- Normalizzazione dei valori dei pixel (scaling da $[0, 255]$ a $[0, 1]$ o $[-1, 1]$); <-quando carico i dataset
- Strategie per la gestione di coppie noisy-clean

Questa parte è fondamentale per garantire che i dati in ingresso ai modelli siano coerenti, bilanciati e in grado di migliorare la convergenza durante la fase di addestramento.

4.1 Conversione immagini RAW

Le immagini contenute nei dataset come MIT-Adobe FiveK o RENOIR sono spesso fornite in formato RAW. Per utilizzare tali immagini nei pipeline di addestramento dei modelli di deep learning, è necessario convertirle in un formato più accessibile e standard.

A tal fine, è stato sviluppato uno script Python che sfrutta le librerie `rawpy` e `imageio` per effettuare la conversione in maniera ricorsiva su intere cartelle. Di seguito viene analizzato il codice sorgente, commentandolo nei suoi blocchi funzionali.

Importazione dei Moduli e Definizione della Funzione

Listing 4.1: Conversione immagini raw a jpg

```

1 def convert_raw_to_jpg_recursive(input_folder, output_folder):
2     input_path = Path(input_folder)
3     output_path = Path(output_folder)
4     output_path.mkdir(parents=True, exist_ok=True)

```

In questo primo blocco viene definita una funzione `convert_raw_to_jpg_recursive` che prende in input il percorso della cartella contenente i file RAW e quello della cartella di destinazione; dopodichè i percorsi vengono convertiti in oggetti `Path` della libreria `pathlib`, per semplificare la gestione cross-platform dei file system. Se la cartella di output non esiste, viene creata automaticamente tramite `mkdir(parents=True, exist_ok=True)`.

Estensioni RAW supportate

Listing 4.2: Estensioni immagini raw

```

1 raw_extensions = ['.CR2', '.NEF', '.ARW', '.RW2', '.RAF', '.
    ↪ DNG', '.ORF', '.PEF']

```

Si definisce una lista delle estensioni più comuni per i file RAW prodotti da vari modelli di fotocamere digitali. Queste estensioni verranno utilizzate per filtrare i file da convertire.

Scansione Ricorsiva della Cartella

Listing 4.3: Scansione ricorsiva cartelle

```

1 for raw_file in input_path.rglob('*'):
2     if raw_file.suffix.upper() in raw_extensions and raw_file
    ↪ .is_file():

```

Qui si esegue una scansione ricorsiva all'interno della cartella di input usando `rglob('*')`, che restituisce tutti i file e sottocartelle. Per ogni file si verifica che l'estensione (in maiuscolo) appartenga alla lista di estensioni RAW supportate e che l'elemento sia effettivamente un file (e non una directory).

Lettura e Conversione del File RAW

Listing 4.4: Lettura immagini con OpenCV

```

1     try:
2         with rawpy.imread(str(raw_file)) as raw:
3             rgb = raw.postprocess()

```

All'interno del blocco `try`, viene utilizzata la libreria `rawpy`, che fornisce un'interfaccia per la decodifica dei file RAW. La funzione `postprocess()` converte i dati RAW nel formato RGB, eseguendo demosaicizzazione, bilanciamento del bianco e correzione del colore.

Generazione del Nome File di Output

Listing 4.5: Rinomina file

```
1 parent_name = raw_file.parent.name
2 output_filename = f"{parent_name}_{raw_file.
  ↳ stem}.jpg"
3 output_file = output_path / output_filename
```

Per generare un nome univoco al file JPG risultante viene prelevato il nome della cartella contenente il file RAW (utile per mantenere traccia della struttura). Il nome del file finale è composto da <nome_cartella>_<nome_file>.jpg e viene salvato nella directory di output.

Scrittura del File e gestione delle eccezioni

Listing 4.6: Salvataggio immagine

```
1 imageio.imwrite(str(output_file), rgb)
2 print(f"Convertito: {raw_file} -> {
  ↳ output_file.name}")
3 except Exception as e:
4 print(f"Errore nella conversione di {raw_file.
  ↳ name}: {e}")
```

La libreria `imageio` viene utilizzata per salvare l'immagine RGB come file JPG. Viene anche stampato a console un messaggio di conferma con il nome del file convertito.

Infine, viene catturata qualsiasi eccezione durante il processo di conversione. Questo è utile per non interrompere lo script in caso di file corrotti o non leggibili. L'errore viene riportato a console per un'eventuale analisi successiva.

Questo script permette di gestire automaticamente grandi volumi di immagini RAW provenienti da più sorgenti e formati. La conversione a JPG è un passaggio fondamentale per uniformare i dati in ingresso ai modelli, riducendone lo spazio occupato. Così facendo si possono applicare tecniche di preprocessing standard e visualizzare rapidamente i dati convertiti.

L'esecuzione ricorsiva rende il sistema scalabile per dataset organizzati in sottocartelle, come spesso accade in raccolte fotografiche professionali.

Classificazione immagini GT-Noisy

Il dataset RENOIR contiene, per ogni scena, una serie di 4/5 immagini acquisite con diversi tempi di esposizione. Questa caratteristica introduce un rumore naturale nelle acquisizioni, rendendo necessario distinguere automaticamente le immagini di riferimento (*ground truth*, GT) da quelle rumorose (*noisy*).

Per questa operazione sono stati utilizzati due script principali: uno per classificare le immagini e uno per duplicare l'immagine GT in corrispondenza di ciascuna immagine rumorosa. Di seguito viene illustrato il codice di classificazione e il suo funzionamento.

Listing 4.7: Caricamento delle immagini di RENOIR

```

1 def load_images_by_batch(input_folder):
2     batches = defaultdict(list)
3     for fname in os.listdir(input_folder):
4         if fname.lower().endswith(".jpg") and fname.startswith("
           ↪ Batch_"):
5         parts = fname.split("_", 2)
6         if len(parts) >= 3:
7             batch_id = parts[1]
8             batches[batch_id].append(fname)
9     return batches

```

La funzione `load_images_by_batch` organizza le immagini in gruppi, basandosi sul prefisso del nome file (`Batch_XXX`). In questo modo, ogni batch di immagini viene identificato in maniera univoca ed è possibile elaborarlo separatamente.

Listing 4.8: Mean Squared Error

```

1 def mse(img1, img2):
2     err = np.mean((img1.astype("float32") - img2.astype("float32")
           ↪ )) ** 2)
3     return err

```

La funzione `mse` calcola il *Mean Squared Error*, ovvero la differenza media al quadrato tra due immagini.

Questo indice è molto usato per stimare quanto un'immagine differisca da un'altra: più il valore è basso, maggiore è la somiglianza.

Listing 4.9: Classificazione batch con MSE

```

1 def classify_batch(batch_images, input_folder):
2     loaded = {}
3     for fname in batch_images:
4         path = os.path.join(input_folder, fname)
5         img = cv2.imread(path)
6         if img is not None:
7             img_resized = cv2.resize(img, (256, 256))
8             loaded[fname] = img_resized
9         else:
10            print(f"[!] Immagine non caricata: {fname}")

```

La funzione `classify_batch` carica tutte le immagini del batch e le ridimensiona ad una dimensione fissa (256×256).

Questo passaggio assicura coerenza nelle dimensioni, fondamentale per il calcolo corretto della similarità.

Listing 4.10: Calcolo di MSE

```

1  mse_scores = {}
2  for name1, img1 in loaded.items():
3      total_error = 0
4      for name2, img2 in loaded.items():
5          if name1 != name2:
6              total_error += mse(img1, img2)
7      avg_error = total_error / (len(loaded) - 1)
8      mse_scores[name1] = avg_error
9
10 gt_image = min(mse_scores, key=mse_scores.get)

```

Successivamente, per ogni immagine del batch si calcola l'errore medio rispetto a tutte le altre (escludendo i confronti con se stessa). L'immagine con il valore medio di MSE più basso viene identificata come *ground truth*, poiché è quella che differisce meno dalle altre.

Listing 4.11: Elaborazione dei risultati

```

1  result = {}
2  for fname in batch_images:
3      label = "GT" if fname == gt_image else "NOISE"
4      result[fname] = (label, mse_scores.get(fname, -1)) # -1
5                      ↪ se immagine non caricata
6  return result

```

In questa fase, i risultati vengono raccolti in un dizionario: l'immagine riconosciuta come GT riceve l'etichetta GT, mentre le altre vengono etichettate come NOISE. Per ciascun file viene salvato anche il relativo punteggio di errore.

Listing 4.12: Salvataggio della classificazione

```

1  def save_classification(result_dict, output_file):
2      MSE_MAX = 255.0
3      with open(output_file, "w") as f:
4          for fname, (label, score) in result_dict.items():
5              f.write(f"{fname} -> {label} [MSE: {score:.2f} -
                      ↪ ERROR: {(score/MSE_MAX):.2f}]\n")

```

La funzione `save_classification` scrive i risultati su un file di testo, indicando per ciascuna immagine: il nome del file, la classificazione (GT/NOISE) e il relativo valore di errore normalizzato rispetto al massimo MSE possibile.

Listing 4.13: Funzione principale per la suddivisione e classificazione in batch

```

1 def main(input_folder, output_file):
2     batches = load_images_by_batch(input_folder)
3     all_results = {}
4     for batch_id, images in batches.items():
5         if len(images) < 2:
6             print(f"[!] Batch {batch_id} ha meno di 2 immagini,
7                 ↳ ignorato.")
8             continue
9
10            print(f"[+] Elaborazione Batch {batch_id} ({len(images)}
11                ↳ immagini)")
12            batch_result = classify_batch(images, input_folder)
13            all_results.update(batch_result)
14            save_classification(all_results, output_file)
15            print(f" Classificazione completata. Risultati salvati in: {
16                ↳ output_file}")

```

Infine, la funzione `main` coordina l'intero processo suddividendo le immagini in batch, applica la classificazione GT/NOISE ad ogni gruppo, raccoglie i risultati complessivi e li salva su file. Questo garantisce una pipeline automatica e ripetibile per la classificazione delle immagini del dataset RENOIR.

Partendo dal caricamento delle batch di immagini con `load_images_by_batch`^{4.7}, vengo-
no ignorate le batch con 2 immagini e si procede con la classificazione con `classify_batch`^{4.9} e
l'aggiunta dei risultati ai risultati totali che sono poi scritti su file attraverso `save_classification`^{4.12}

Duplicazione immagini

Una volta classificata l'immagine come *ground truth* (gt), essa viene duplicata e associata per ognuna di quelle classificate come *noisy*. Il seguente script Python automatizza il processo, organizzando le immagini e garantendo la coerenza della nomenclatura.

Listing 4.14: Definizione della funzione per processare la directory

```

1 def process_batch_folder(batch_folder, parent_dir):
2     batch_name = os.path.basename(batch_folder)
3     print(f"[+] Processing {batch_name}")
4     copied_files = []

```

La funzione `process_batch_folder` riceve in ingresso una cartella di batch e la cartella principale. Inizia recuperando il nome del batch e prepara una lista per memorizzare i file duplicati o rinominati.

Listing 4.15: Step 1: Gestione Reference

```

1  reference_files = glob.glob(os.path.join(batch_folder, '*
    ↳ Reference.bmp'))
2  if not reference_files:
3      print(f"Nessun file Reference trovato in {batch_name}")
4  else:
5      ref = reference_files[0]
6      base = os.path.splitext(os.path.basename(ref))[0]
7      dest1_name = f"{base}_1.bmp"
8      dest2_name = f"{base}_2.bmp"
9      dest1 = os.path.join(batch_folder, dest1_name)
10     dest2 = os.path.join(batch_folder, dest2_name)
11     shutil.copy(ref, dest1)
12     shutil.copy(ref, dest2)
13     os.remove(ref)
14     copied_files += [dest1, dest2]

```

Nel Primo Step si gestiscono le immagini di riferimento (Reference). Se presenti, vengono duplicate e rinominate con suffissi numerici (_1, _2). L'originale viene poi eliminato, così da mantenere solo i duplicati coerenti.

Listing 4.16: Step 2: Gestione Noisy

```

1  noisy_files = sorted(glob.glob(os.path.join(batch_folder, '*
    ↳ Noisy.bmp')))
2  new_noisy_paths = []
3  for idx, path in enumerate(noisy_files, start=1):
4      base = os.path.splitext(os.path.basename(path))[0]
5      new_name = f"{base}_{idx}.bmp"
6      new_path = os.path.join(batch_folder, new_name)
7      os.rename(path, new_path)
8      copied_files.append(new_path)

```

Nel Secondo Step vengono gestite le immagini rumorose (Noisy). Ognuna viene rinominata in modo incrementale, aggiungendo un indice alla fine del nome, così da distinguerle chiaramente e mantenerle ordinate.

Listing 4.17: Step 3: Aggiunta del prefisso Batch_XXX_

```

1  renamed_paths = []
2  for fname in os.listdir(batch_folder):
3      if fname.endswith(".bmp") and not fname.startswith(
4          ↪ batch_name + "_"):
5          old_path = os.path.join(batch_folder, fname)
6          new_name = f"{batch_name}_{fname}"
7          new_path = os.path.join(batch_folder, new_name)
8          os.rename(old_path, new_path)
9          # Aggiorna anche la lista dei file da copiare
10         if old_path in copied_files:
11             renamed_paths.append(new_path)
12     elif fname.endswith(".bmp") and fname.startswith(
13         ↪ batch_name + "_"):
14         renamed_paths.append(os.path.join(batch_folder, fname
15         ↪ ))

```

Il Terzo Step si occupa di uniformare i nomi dei file aggiungendo il prefisso Batch_XXX_, dove XXX è il nome della cartella batch. In questo modo, tutte le immagini risultano facilmente tracciabili rispetto al batch di origine.

Listing 4.18: Step 4: Copia nella cartella principale

```

1  for file_path in renamed_paths:
2      dest_path = os.path.join(parent_dir, os.path.basename(
3          ↪ file_path))
4      shutil.copy(file_path, dest_path)
5      print(f"Copiato {os.path.basename(file_path)} -> {
6          ↪ parent_dir}")

```

Nel Quarto Step le immagini rinominate vengono copiate nella cartella principale, così da raccogliere i dati elaborati da più batch in un unico luogo centralizzato.

Listing 4.19: Utilizzo della funzione process_batch_folder

```

1  def main(parent_dir):
2      for folder in sorted(os.listdir(parent_dir)):
3          full_path = os.path.join(parent_dir, folder)
4          if folder.startswith("Batch_") and os.path.isdir(
5              ↪ full_path):
6              process_batch_folder(full_path, parent_dir)

```

La funzione main scorre tutte le cartelle nella directory principale, seleziona quelle che iniziano con Batch_ e applica la funzione process_batch_folder per elaborarle.

4.2 Padding di immagini con risoluzione inferiore a 512x512

Nel contesto dell'addestramento dei modelli di deep learning, è spesso necessario uniformare la dimensione delle immagini in input. Questo script Python permette di adattare immagini a una risoluzione di 512×512 pixel.

Definizione delle Costanti

Il dataset BSD 500 è composto da tre cartelle con immagini non RAW e dimensioni variabili inferiori alla risoluzione 512x512

Listing 4.20: Costanti con direcotry per BSD

```

1 TRAIN_512_DIR = "BSD_512/train"
2 TEST_512_DIR = "BSD_512/test"
3 VAL_512_DIR = "BSD_512/val"
4
5 DIR_TEST = "testdir"
6 DIR_RESULT = "resultdir"

```

Sono definite le directory di input e output, rispettivamente per i set di training, validation e test.

Padding centrato

Listing 4.21: Funzione di padding

```

1 def pad_to_512(img):
2     h, w = img.shape[:2]
3     pad_bottom = max(0, 512 - h)
4     pad_right = max(0, 512 - w)

```

Viene presa l'immagine e tramite `img.shape[:2]` e si estraggono altezza(**h**) e larghezza(**w**). Poi si calcola per le 2 grandezze lo scarto di pixel che manca ad arrivare a 512

Listing 4.22: Creazione del padding sui lati dell'immagine

```

1     pad_b_1 = pad_bottom // 2
2     pad_b_2 = pad_bottom - pad_b_1
3     pad_r_1 = pad_right // 2
4     pad_r_2 = pad_right - pad_r_1
5     return cv2.copyMakeBorder(img, pad_b_1, pad_b_2, pad_r_1,
        ↪ pad_r_2, cv2.BORDER_REFLECT_101)

```

Poi gli scarti calcolati vengono suddivisi per mantenere il padding centrato.

Infine viene applicato un bordo riflesso(padding) tramite `cv2.copyMakeBorder` su tutti lati dell'immagine mantenendo la risoluzione originale, dunque senza snaturarla e aumentandone le dimensioni a quelle volute.

`cv2.BORDER_REFLECT_101` è una costante di OpenCV utilizzata per specificare se l'immagine vada specchiata o ripetuta sui bordi riflessi

Funzione di elaborazione batch delle immagini

Listing 4.23: Definizione funzione di elaborazione immagini

```

1 def process_images(input_dir, output_dir):
2     count = 0
3     for fname in os.listdir(input_dir):
4         if not fname.lower().endswith((".png", ".jpg", ".jpeg")):
5             continue

```

Ad ogni file viene controllata l'estensione, in caso non sia un'immagine il programma lo salta, non precludendo l'esecuzione agli altri file

```

        img_path = os.path.join(input_dir, fname)
        img = cv2.imread(img_path)
        if img is None:
            print(f"[!] Errore nel leggere {fname}")
            continue

        out_img = pad_to_512(img)

        out_path = os.path.join(output_dir, fname)
        cv2.imwrite(out_path, out_img)
        count += 1

    print(f"{count} immagini processate")

```

Viene fatto un unione di percorso e nome immagine per compatibilità di sistema e convertita in un oggetto manipolabile da OpenCV.

Controlla che l'immagine non sia corrotta o illeggibile, applica il padding e crea il file nella cartella di output. Alla fine stampa il numero di file processati



Figura 4.1: Esempio di padding sul dataset BSD500.

4.3 Patching

Il **patching** è una tecnica comunemente utilizzata nell'ambito dell'elaborazione di immagini e dell'addestramento di reti neurali convoluzionali, che consiste nel suddividere un'immagine di grandi dimensioni in più sotto-immagini quadrate o rettangolari, dette *patch*. Questo approccio presenta diversi vantaggi: riduce il consumo di memoria e il tempo di addestramento, permette di aumentare la quantità di dati disponibili (poiché da una singola immagine possono essere ricavati numerosi patch), e consente al modello di concentrarsi su porzioni locali dell'immagine, migliorando la capacità di apprendere dettagli fini e strutture locali.

Durante la fase di ricostruzione o inferenza, i patch possono essere riuniti per ricomporre l'immagine originale, spesso con tecniche di sovrapposizione e fusione per evitare artefatti lungo i bordi. In sintesi, il patching rappresenta una strategia essenziale per la gestione efficiente di dataset ad alta risoluzione e per l'ottimizzazione delle prestazioni nei modelli di visione artificiale.

Padding iniziale

Listing 4.24: Costanti per il patching

```
1 PATCH_SIZE = 512
2 # Layout per immagini orizzontali
3 ROWS_HORIZONTAL = 3
4 COLS_HORIZONTAL = 4
5 # Layout per immagini verticali
6 ROWS_VERTICAL = COLS_HORIZONTAL
7 COLS_VERTICAL = ROWS_HORIZONTAL
```

Costanti usate per definire la dimensione delle patch e il loro numero totale tenendo conto che l'immagine sia verticale o orizzontale.

Listing 4.25: Padding per il patching

```
1 def pad_image(img, target_width, target_height):
2     h, w = img.shape[:2]
3     pad_bottom = max(0, target_height - h)
4     pad_right = max(0, target_width - w)
5     padded = cv2.copyMakeBorder(img, 0, pad_bottom, 0, pad_right,
6     ↪ cv2.BORDER_REFLECT_101)
7     return padded
```

Fare padding su immagini grandi per renderle divisibili per la dimensione delle patch, delle colonne e delle righe.

In questo caso vengono aggiunti pixel solo nei lati **BOTTOM** e **RIGHT**.

Estrazione delle patch

Variabili e costanti

Listing 4.26: Funzione di estrazione patch dalle immagini

```

1 def extract_orientation_aware_patches(image, patch_size=
  ↪ PATCH_SIZE):
2     h, w = image.shape[:2]
3     if w >= h:
4         rows, cols = ROWS_HORIZONTAL, COLS_HORIZONTAL
5     else:
6         rows, cols = ROWS_VERTICAL, COLS_VERTICAL
7     total_height = patch_size * rows
8     total_width = patch_size * cols
9
10    img = pad_image(image, total_width, total_height)

```

Questa funzione parte prendendo altezza e larghezza dell'immagine passata, controlla se deve usare le costanti per immagini orizzontali o verticali, calcola le dimensioni per il padding e poi

Calcolo dei passi e creazione delle patch

Listing 4.27: Calcolo dei passi e creazione patch

```

1     row_step = (h - patch_size) // (rows - 1) if rows > 1 else 0
2     col_step = (w - patch_size) // (cols - 1) if cols > 1 else 0
3     patches = []
4     for i in range(rows):
5         for j in range(cols):
6             y = i * row_step
7             x = j * col_step
8             patch = img[y:y+patch_size, x:x+patch_size]
9             if patch.shape[0] == patch_size and patch.shape[1] ==
  ↪ patch_size:
10                 patches.append(patch)
11
12     return patches

```

Questo frammento di codice si occupa di suddividere un'immagine in piccole sotto-immagini, chiamate *patches*, che vengono estratte in modo regolare lungo righe e colonne.

Per determinare la distanza fra una patch e la successiva, vengono calcolati `row_step` per il passo verticale e `col_step` per il passo orizzontale.

Successivamente, due cicli annidati percorrono l'immagine riga per riga e colonna per colonna, calcolando di volta in volta le coordinate `x` e `y` di partenza di ogni patch. Ogni ritaglio ha dimensione fissa (`patch_size`) e viene verificato che corrisponda esattamente alle dimensioni attese prima di essere aggiunto alla lista `patches`.

Infine, la funzione restituisce l'insieme di tutte le patch ottenute, che potranno essere usate come input per modelli di deep learning o per altre operazioni di elaborazione delle immagini.

Elaborazione delle cartelle di immagini

```

1 def process_folder(input_dir, output_dir):
2     os.makedirs(output_dir, exist_ok=True)
3     supported_ext = (".png", ".jpg", ".jpeg", ".dng")
4     filenames = [f for f in os.listdir(input_dir) if f.lower().
        ↪ endswith(supported_ext)]

```

Questa funzione serve per caricare tutte le immagini e creare le patch su disco. Viene chiamata la funzione per la cartella di output che crea la directory in caso non esista, vengono specificate le estensioni accettate dal programma, e viene stilata una lista con tutti i file trovati

Ciclo di elaborazione delle immagini e verifica preliminare

Listing 4.28: Ciclo di elaborazione delle immagini e verifica preliminare

```

1 for fname in filenames:
2     print(f"Processing {fname}...")
3     img_path = os.path.join(input_dir, fname)
4     img = cv2.imread(img_path)
5     if img is None:
6         print(f"[!] Immagine non valida: {fname}")
7         continue
8     if img.shape[0] < PATCH_SIZE or img.shape[1] < PATCH_SIZE
        ↪ :
9         print(f"[!] Immagine troppo piccola: {fname} ({img.
            ↪ shape[1]}x{img.shape[0]})")
10        continue
11    patches = extract_orientation_aware_patches(img)

```

Viene eseguito un ciclo su tutti i file contenuti in una lista di immagini (`filenames`), processandoli uno alla volta.

Per ciascun file viene costruito il percorso completo e l'immagine viene caricata con `cv2.imread`. Se l'immagine non è valida o non ha le dimensioni richieste, il programma passa al file successivo, evitando così crash o elaborazioni inutili.

Solo le immagini valide e sufficientemente grandi vengono infine suddivise in patch tramite la funzione `extract_orientation_aware_patches`.

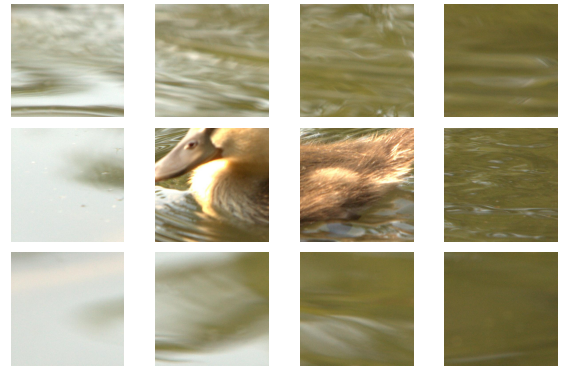
Salvataggio delle patch su disco

```

1     base_name = os.path.splitext(fname)[0]
2     for i, patch in enumerate(patches):
3         out_name = f"{base_name}_patch{i}.jpg"
4         cv2.imwrite(os.path.join(output_dir, out_name), patch
            ↪ )
5     print(f"Patch salvate in: {output_dir}")

```

Per ogni patch creata questa avrà il nome dell'immagine originale seguita dal numero di patch corrispondente per poi essere scritta su disco usando sempre OpenCV.



(a) Immagine completa presa dal dataset FiveK
(b) Patch estratti dall'immagine

Figura 4.2: Risultato del processo di patching sulle immagini grandi. L'immagine originale (a) è dunque suddivisa in 12 patch (b) di dimensione 512×512 pixel ciascuna.

4.4 Aggiunta del rumore

In questa sezione viene descritto il processo di **aggiunta artificiale di rumore** alle immagini, applicato unicamente ai dataset che non *noisy* (BSD512, Div2K, FiveK).

L'obiettivo è simulare condizioni realistiche in cui il modello deve operare, garantendo così una maggiore generalizzazione durante la fase di addestramento.

Il rumore più comunemente impiegato è quello **gaussiano additivo**, caratterizzato da media nulla e varianza controllabile, che permette di generare diverse intensità di corruzione. Questa scelta riflette uno dei modelli più utilizzati in letteratura per la simulazione del rumore fotografico, ed è adatta a valutare la capacità delle reti neurali di rimuovere disturbi casuali preservando i dettagli strutturali delle immagini.

Listing 4.29: Aggiunta di rumore da lista

```

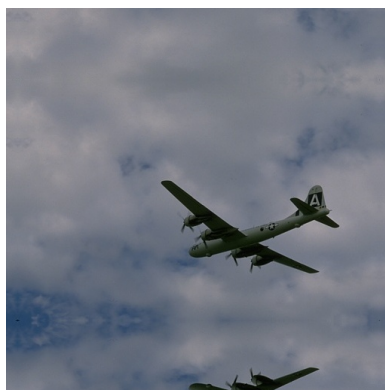
1 def add_noise(x):
2     noise_levels = [15 / 255.0, 25 / 255.0, 50 / 255.0]
3     noise_factor = tf.random.shuffle(noise_levels)[0]
4     noise = tf.random.normal(shape=tf.shape(x), mean=0.0, stddev=
      ↪ noise_factor)
5     x_noisy = x + noise
6     x_noisy = tf.clip_by_value(x_noisy, 0.0, 1.0)
7     return x_noisy

```

La funzione

La funzione implementata consente di:

- ricevere in input un'immagine priva di rumore;
- aggiungere rumore gaussiano di intensità variabile. L'intensità viene selezionata casualmente da una lista predefinita di valori normalizzati tra 0 e 1: (15, 25, 50) che divisi per 255 valgono (0,0588235294117647, 0,0980392156862745, 0,196078431372549);
- restituire l'immagine degradata e normalizzata tra 0 e 1, che può essere utilizzata come input per l'addestramento dei modelli.



(a) Immagine originale

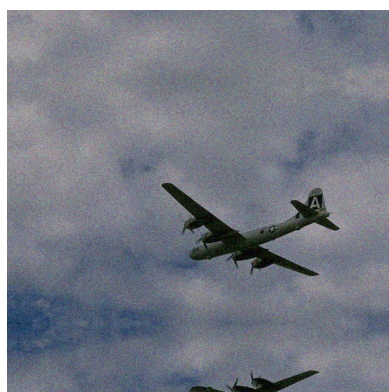
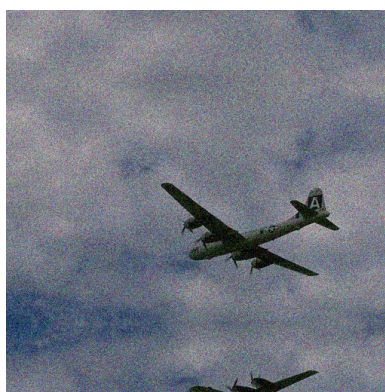
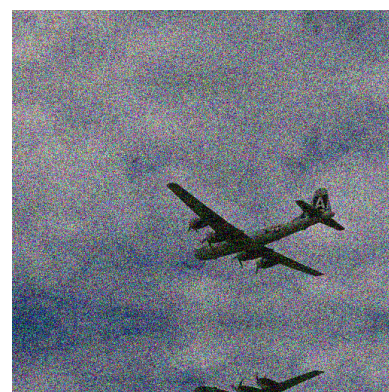
(b) $\sigma = 15$
PSNR = 24.64 dB(c) $\sigma = 25$
PSNR = 20.21 dB(d) $\sigma = 50$
PSNR = 14.34 dB

Figura 4.3: Esempio di immagine originale presa da **BSD** e corrispondenti versioni con tre diversi livelli di rumore

Listing 4.30: Aggiunta del rumore da funzione

```
1 def add_noise_2(x, noise_factor):  
2     noise = tf.random.normal(shape=tf.shape(x), mean=0.0, stddev=  
3         ↪ noise_factor)  
4     x_noisy = x + noise  
5     x_noisy = tf.clip_by_value(x_noisy, 0.0, 1.0)  
6     return x_noisy
```

Questa funzione è simile alla precedente, ma consente di specificare direttamente il livello di rumore desiderato tramite il parametro `noise_factor`.

Questo permette una maggiore flessibilità, ad esempio per testare il modello con diversi livelli di rumore in modo controllato.

4.5 Creazione dei dataset

Qui si andrà a descrivere il processo di creazione dei dataset di training, validation e test set, sia per le immagini non rumorose che per quelle con rumore aggiunto artificialmente. Tutti i dataset sono stati elaborati tramite script Python utilizzando la libreria TensorFlow per la gestione e il preprocessing delle immagini.

Creazione dataset non noisy

I dataset **BSD512**, **Div2K** e **FiveK** sono dataset di immagini non rumorose.

Per creare i dataset di training, validation e test set, si utilizza la libreria TensorFlow, che offre strumenti efficienti per la gestione e il preprocessing delle immagini.

Listing 4.31: Definizione della funzione di creazione dei dataset

```

1  def load_and_split_dataset(input_dir, train_pct=0.6, val_pct
    ↪ =0.2, test_pct=0.2, batch_size=32, shuffle=True):
2  assert abs(train_pct + val_pct + test_pct - 1.0) < 1e-5, "Le
    ↪ percentuali devono sommare a 1.0"
3
4  RANDOM_SEED = 42
5  # Prendi tutti i percorsi dei file nella cartella
6  file_paths = [os.path.join(input_dir, fname) for fname in os.
    ↪ listdir(input_dir)
7                  if fname.lower().endswith((''.jpg', '.jpeg', '.
    ↪ png', '.bmp'))]
8
9  random.seed(RANDOM_SEED)
10 random.shuffle(file_paths)

```

La funzione `load_and_split_dataset` inizia con la definizione delle percentuali di suddivisione del dataset in training, validation e test set. Viene poi impostato un seme casuale per garantire la riproducibilità della suddivisione. Successivamente, vengono raccolti tutti i percorsi dei file immagine presenti nella cartella di input, considerando solo i formati più comuni. Infine, la lista dei percorsi viene mescolata in modo casuale per evitare qualsiasi ordine predefinito che potrebbe influenzare la suddivisione.

Listing 4.32: Calcolo degli indici di divisione

```

1  # Calcola gli indici di divisione
2  total_files = len(file_paths)
3  print(f"total files: {total_files}")
4  train_size = int(train_pct * total_files)
5  val_size = int(val_pct * total_files)
6  # Il test size viene calcolato implicitamente prendendo il
    ↪ resto
7
8  # Dividi i percorsi dei file
9  train_files = file_paths[:train_size]
10 val_files = file_paths[train_size : train_size + val_size]
11 test_files = file_paths[train_size + val_size:]

```

Viene calcolato il numero totale di file e, in base alle percentuali specificate, vengono determinati gli indici per suddividere la lista dei percorsi in tre sottoinsiemi: training, validation e test set.

Listing 4.33: Definizione sottofunzione per caricare le immagini

```

1  def _load_and_preprocess_image(path):
2      print(path)
3      image = tf.io.read_file(path)
4      image = tf.image.decode_image(image, channels=3,
5          ↪ expand_animations=False)
6      image = tf.image.resize(image, [128, 128])
7      image = tf.image.convert_image_dtype(image, tf.float32) /
8          ↪ 255.0 # Normalizza in [0,1]
9      return image

```

Questa funzione interna si occupa di caricare e preprocessare ogni immagine. Riceve in input il percorso di un'immagine, la legge, la decodifica in un formato utilizzabile da TensorFlow, la ridimensiona a 128×128 pixel e normalizza i valori dei pixel nell'intervallo $[0, 1]$.

Listing 4.34: Creazione dei dataset TensorFlow

```

1  options = tf.data.Options()
2  options.experimental_optimization.apply_default_optimizations
3      ↪ = True
4  options.experimental_distribute.auto_shard_policy = tf.data.
5      ↪ experimental.AutoShardPolicy.OFF
6  # Crea i dataset TensorFlow
7  train_ds = tf.data.Dataset.from_tensor_slices(train_files)
8  val_ds = tf.data.Dataset.from_tensor_slices(val_files)
9  test_ds = tf.data.Dataset.from_tensor_slices(test_files)
10 train_ds = train_ds.map(_load_and_preprocess_image,
11     ↪ num_parallel_calls=tf.data.AUTOTUNE)
12 val_ds = val_ds.map(_load_and_preprocess_image,
13     ↪ num_parallel_calls=tf.data.AUTOTUNE)
14 test_ds = test_ds.map(_load_and_preprocess_image,
15     ↪ num_parallel_calls=tf.data.AUTOTUNE)
16
17 train_ds = train_ds.batch(batch_size).prefetch(tf.data.
18     ↪ AUTOTUNE).with_options(options)
19 val_ds = val_ds.batch(batch_size).prefetch(tf.data.AUTOTUNE).
20     ↪ with_options(options)
21 test_ds = test_ds.batch(batch_size).prefetch(tf.data.AUTOTUNE
22     ↪ ).with_options(options)
23
24 print(f"Dataset di training creato:\n{train_ds}\nNumero di
25     ↪ file: {len(train_files)}")
26 print(f"Dataset di validation creato:\n{val_ds}\nNumero di
27     ↪ file: {len(val_files)}")
28 print(f"Dataset di test creato:\n{test_ds}\nNumero di file: {
29     ↪ len(test_files)}")
30 return train_ds, val_ds, test_ds

```

Vengono impostate alcune opzioni per ottimizzare le prestazioni del dataset. Successivamente, vengono creati i dataset TensorFlow a partire dai percorsi dei file suddivisi in precedenza. Ogni dataset viene mappato alla funzione di caricamento e preprocessing delle immagini, con l'uso di chiamate parallele per migliorare l'efficienza. Infine, i dataset vengono raggruppati in batch, prefetchati per migliorare le prestazioni durante l'addestramento, e le opzioni di ottimizzazione vengono applicate. La funzione stampa informazioni sui dataset creati e restituisce i tre dataset.

Aggiunta del rumore nei dataset

Listing 4.35: Aggiunta del rumore nei dataset

```

1 def noisy_part(ds):
2     noisy_ds = ds.map((lambda x: (add_noise(x), x)),
3         ↪ num_parallel_calls=tf.data.AUTOTUNE)
4     return noisy_ds.prefetch(tf.data.AUTOTUNE)

```

Questa funzione è utilizzata per generare un dataset rumoroso a partire da un dataset originale. Riceve in input un dataset di immagini pulite e applica la funzione `add_noise` a ciascuna immagine, creando così una coppia di immagini: l'immagine rumorosa e l'immagine originale. Il dataset risultante viene prefetchato per migliorare le prestazioni durante l'addestramento.

Creazione dataset noisy

Listing 4.36: Creazione del dataset rumoroso

```

1 def load_and_split_paired_dataset(gt_dir, noisy_dir, train_pct
2     ↪ =0.6, val_pct=0.2, test_pct=0.2, batch_size=32, shuffle=
3     ↪ True):
4     # GT
5     gt_train, gt_val, gt_test = load_and_split_dataset(gt_dir,
6         ↪ train_pct, val_pct, test_pct, batch_size, shuffle)
7
8     # NOISY
9     noisy_train, noisy_val, noisy_test = load_and_split_dataset(
10        ↪ noisy_dir, train_pct, val_pct, test_pct, batch_size,
11        ↪ shuffle)
12
13     train_ds = tf.data.Dataset.zip((noisy_train, gt_train)).
14        ↪ prefetch(tf.data.AUTOTUNE)
15     val_ds = tf.data.Dataset.zip((noisy_val, gt_val)).prefetch(tf.
16        ↪ data.AUTOTUNE)
17     test_ds = tf.data.Dataset.zip((noisy_test, gt_test)).prefetch(
18        ↪ tf.data.AUTOTUNE)
19
20     return train_ds, val_ds, test_ds

```

Questa funzione consente di creare dataset di immagini rumorose e corrispondenti immagini pulite (ground truth) per l'addestramento, la validazione e il test di modelli di denoising. Riceve in input le directory contenenti le immagini pulite e rumorose, insieme alle percentuali di suddivisione e altre opzioni. Utilizza la funzione `load_and_split_dataset` per caricare e

suddividere entrambi i dataset. Successivamente, i dataset rumorosi e puliti vengono combinati in coppie utilizzando `tf.data.Dataset.zip`, e il risultato viene prefetchato per migliorare le prestazioni. Infine, la funzione restituisce i tre dataset pronti per l'addestramento. La caratteristica principale di questa funzione è la capacità di mantenere la corrispondenza tra immagini rumorose e pulite grazie al seed casuale che mantiene la stessa suddivisione in training, validation e test set per entrambi i dataset, essenziale per l'addestramento supervisionato di modelli di denoising.

Capitolo 5

Sperimentazione e analisi dei modelli

In questo capitolo vengono presentati i modelli implementati, i loro risultati e le analisi effettuate.

5.1 Implementazione modelli

RIDNet

In questa sezione viene riportata l'implementazione della rete **RIDNet** in Keras/TensorFlow presa dal caso di studio **Real Image Denoising with Feature Attention** (Anwar e Barnes 2019)

Blocchi di base

Listing 5.1: Implementazione del blocco di base BasicBlock

```
1 @register_keras_serializable("BasicBlock")
2 def BasicBlock(x, out_channels, ksize=3, stride=1, pad='same'):
3     x = Conv2D(out_channels, ksize, strides=stride, padding=pad)(
4         ↪ x)
5     x = ReLU()(x)
6     return x
```

Questo blocco esegue due operazioni fondamentali:

1. Una **convoluzione**, cioè un'operazione che “legge” l'immagine con un filtro (o kernel) e produce nuove mappe di caratteristiche (*feature maps*). Queste mappe contengono informazioni sui bordi, sui contorni e su pattern locali.
2. Una funzione di attivazione **ReLU**, che sostituisce i valori negativi con zero e mantiene i positivi invariati. Questo passaggio rende la rete capace di apprendere relazioni non lineari, fondamentali per rappresentare dettagli complessi dell'immagine.

In altre parole, il **BasicBlock** prende un'immagine o una mappa di caratteristiche e ne produce una nuova versione, mettendo in evidenza strutture locali utili per distinguere rumore da dettaglio.

Listing 5.2: Implementazione del blocco di base con sigmoide BasicBlockSig

```

1 @register_keras_serializable("BasicBlockSig")
2 def BasicBlockSig(x, out_channels, ksize=3, stride=1, pad='same')
3     ↪ :
4     x = Conv2D(out_channels, ksize, strides=stride, padding=pad)(
5         ↪ x)
6     x = tf.keras.activations.sigmoid(x)
7     return x

```

La differenza con il blocco precedente è che al posto della ReLU utilizza una **sigmoide**.

La sigmoide restituisce valori compresi tra 0 e 1, ed è molto utile quando vogliamo “pesare” le informazioni: un valore vicino a 1 indica che una certa caratteristica è molto importante, mentre un valore vicino a 0 indica che può essere trascurata.

Questo blocco viene usato nei moduli di attenzione.

Blocchi avanzati

Listing 5.3: Blocco Merge_Run_dual per l'estrazione multiscala

```

1 @register_keras_serializable("Merge_Run_dual")
2 def Merge_Run_dual(x, in_channels, out_channels):
3     b1 = Conv2D(out_channels, 3, padding='same')(x)
4     b1 = ReLU()(b1)
5     b1 = Conv2D(out_channels, 3, padding='same', dilation_rate=2)
6         ↪ (x)
7     b1 = ReLU()(b1)
8     b2 = Conv2D(out_channels, 3, padding='same', dilation_rate=3)
9         ↪ (x)
10    b2 = ReLU()(b2)
11    b2 = Conv2D(out_channels, 3, padding='same', dilation_rate=4)
12        ↪ (x)
13    b2 = ReLU()(b2)
14    c = Concatenate(axis=-1)([b1, b2])
15    out = Conv2D(out_channels, 3, padding='same')(c)
16    out = ReLU()(out)
17    return Add()([out, x])

```

L'input viene analizzato con convoluzioni che hanno diversi **tassi di dilatazione**.

Il tasso di dilatazione indica la distanza con cui i pesi del filtro convoluzionale vengono applicati all'input, cioè quanti pixel vengono "saltati" tra un campionamento e l'altro, permettendo al filtro di coprire un'area più ampia senza aumentare la dimensione del filtro. Questo consente di ampliare il campo visivo (receptive field) del filtro mantenendo sotto controllo il numero di parametri.

I risultati vengono **concatenati**, cioè messi uno accanto all'altro, e passati a un'altra convoluzione che li mescola ed infine, si aggiunge l'output all'input originale (**connessione residua**), in modo da non perdere informazioni.

Questo blocco serve a dare al modello la capacità di comprendere l'immagine a più “scale” contemporaneamente.

Listing 5.4: Blocco ResidualBlock con connessione residua

```

1 @register_keras_serializable("ResidualBlock")
2 def ResidualBlock(x, out_channels):
3     res = Conv2D(out_channels, 3, padding='same')(x)
4     res = ReLU()(res)
5     res = Conv2D(out_channels, 3, padding='same')(res)
6     return ReLU()(Add()([res, x]))

```

Questo è un classico **blocco residuo**, introdotto con le ResNet(Reti Neurali Residuali). L'idea è che invece di trasformare completamente l'input, la rete impara a calcolare solo la "correzione" necessaria (residuo). Questo rende l'apprendimento più stabile e veloce. Il risultato è la somma tra input originale e la trasformazione appresa.

Listing 5.5: Blocco EResidualBlock con convoluzioni a gruppi

```

1 @register_keras_serializable("EResidualBlock")
2 def EResidualBlock(x, out_channels, group=1):
3     res = Conv2D(out_channels, 3, padding='same', groups=group)(x
4         ↪ )
5     res = ReLU()(res)
6     res = Conv2D(out_channels, 3, padding='same', groups=group)(
7         ↪ res)
8     res = ReLU()(res)
9     res = Conv2D(out_channels, 1, padding='valid')(res)
10    return ReLU()(Add()([res, x]))

```

Variante più profonda del blocco residuo. Qui vengono usate le **convoluzioni a gruppi**, che dividono i canali in sottoinsiemi elaborati separatamente: in questo modo il modello può specializzarsi su diversi tipi di caratteristiche senza aumentare troppo i calcoli. Alla fine una convoluzione 1x1(Conv2D(out_channels, 1, [...])) ricompone i canali.

Layer di supporto

Listing 5.6: Layer MeanShift per normalizzazione dei valori dei pixel

```

1 @register_keras_serializable("MeanShift")
2 def MeanShift(rgb_range, rgb_mean, rgb_std, sign=-1):
3     @register_keras_serializable("shift")
4     def shift(x):
5         rgb_mean_tensor = tf.constant(rgb_mean, dtype=tf.float32)
6         rgb_std_tensor = tf.constant(rgb_std, dtype=tf.float32)
7         return (x + sign * rgb_range * rgb_mean_tensor) /
8             ↪ rgb_std_tensor
9     shift_layer = Lambda(shift)
10    shift_layer.shift_fn = shift
11    return shift_layer

```

Serve a **normalizzare** i valori dei pixel. Le immagini hanno tipicamente valori compresi tra 0 e 255, ma è più facile per la rete lavorare su dati con media zero e varianza unitaria. Questo strato sposta (shift(x)) e ridimensiona i valori dei pixel in base alla media e deviazione standard di ciascun canale RGB.

Listing 5.7: Layer di attenzione sui canali (CALayer)

```

1 @register_keras_serializable("CALayer")
2 def CALayer(x, channel, reduction=16):
3     y = GlobalAveragePooling2D()(x)
4     y = Reshape((1, 1, channel))(y)
5     y = BasicBlock(y, channel // reduction, ksize=1, pad='valid')
6     y = BasicBlockSig(y, channel, ksize=1, pad='valid')
7     return Multiply()([x, y])

```

È un **layer di attenzione sui canali**. Funziona così:

1. Fa una media globale per ogni canale (es. canale rosso, verde, blu), in modo da capire quanto ciascun canale contribuisce all'immagine(`GlobalAveragePooling2D`).
2. Passa questi valori attraverso piccole reti (convoluzioni 1x1 + sigmoide) per ottenere dei pesi tra 0 e 1.
3. Moltiplica questi pesi ai canali originali: accendendo i canali più utili e spegnendo quelli meno importanti(`Multiply()([x, y])`).

Blocco completo

Alla fine i layer `Merge_Run_Dual`(5.3), `ResidualBlock`(5.4), `EResidualBlock`(5.5) e `CALayer`(5.7) sono riuniti a formare l'unità base della rete

Listing 5.8: Blocco completo che unisce moduli multiscala, residui e attenzione

```

1 @register_keras_serializable("Block")
2 def Block(x, in_channels, out_channels):
3     x = Merge_Run_dual(x, in_channels, out_channels)
4     x = ResidualBlock(x, out_channels)
5     x = EResidualBlock(x, out_channels)
6     x = CALayer(x, out_channels)
7     return x

```

Architettura RIDNet

Qui di seguito viene mostrata la costruzione completa della rete **RIDNet**, che combina tutti i blocchi e layer precedentemente definiti.

Listing 5.9: Costruzione dell'architettura completa RIDNet

```

1 @register_keras_serializable("build_RIDNET")
2 def build_RIDNET(input_shape, n_feats, reduction, rgb_range=1.0):
3     rgb_mean = (0.4488, 0.4371, 0.4040)
4     rgb_std = (1.0, 1.0, 1.0)
5     inputs = Input(shape=input_shape)
6     x = MeanShift(rgb_range, rgb_mean, rgb_std, sign=-1)(inputs)

```

In questa prima parte viene definita la funzione `build_RIDNET`, che costruisce il modello. L'input è un'immagine con una forma specificata da `input_shape`. Prima di elaborarla, viene applicato un `MeanShift`, che normalizza i valori dei pixel in base alla media e alla deviazione standard dei canali RGB. Questo passaggio serve a semplificare l'apprendimento della rete.

Listing 5.10: Corpo della ridnet

```

1  x = BasicBlock(x, n_feats)
2  b1 = Block(x, n_feats, n_feats)
3  b2 = Block(b1, n_feats, n_feats)
4  b3 = Block(b2, n_feats, n_feats)
5  b4 = Block(b3, n_feats, n_feats)

```

Nella parte centrale viene applicato un primo `BasicBlock` 5.1, che funziona come livello iniziale di estrazione delle caratteristiche.

Successivamente, l'immagine passa attraverso una serie di `Block` 5.8, ognuno composto da più sottostrutture (convoluzioni multiscala, blocchi residui e meccanismi di attenzione sui canali). I blocchi sono concatenati (`b1`, `b2`, `b3`, `b4`), così che la rete possa catturare informazioni via via più complesse e dettagliate. La struttura dell'esperimento citato (Anwar e Barnes 2019) usa solo quattro blocchi

Listing 5.11: Ultimo blocco della rete

```

1  res = Conv2D(3, 3, padding='same')(b4)

```

Dopo l'ultimo blocco, si applica una convoluzione finale con 3 filtri, corrispondenti ai tre canali RGB.

Il risultato è un'immagine “ricostruita” che rappresenta una prima versione denoised dell'input.

Listing 5.12: Pezzo finale della rete

```

1  out = MeanShift(rgb_range, rgb_mean, rgb_std, sign=1)(res)
2  out = Add()([out, inputs])
3  model = Model(inputs=inputs, outputs=out)
4  return model

```

Infine si applica nuovamente un `MeanShift`, questa volta con `sign=1`, per riportare i valori dei pixel alla scala originale. Viene effettuata una somma residua (`Add`) tra l'output elaborato e l'immagine di input: questo permette alla rete di concentrarsi sulla rimozione del rumore, mantenendo però i dettagli originali. Infine il modello viene costruito utilizzando l'API funzionale di Keras, specificando gli input e gli output. In questo modo, la **RIDNet** sfrutta il concetto di connessioni residue e blocchi multiscala per ridurre il rumore dalle immagini mantenendo il più possibile la loro qualità visiva.

Listing 5.13: Costruzione della RIDNet

```

1  model = build_RIDNET(input_shape=(None, None, 3), n_feats=64,
    ↪ reduction=16)

```

Dunque la rete viene così inizializzata, seguendo in parte l'esperimento di partenza utilizzando lo shape delle immagini di input come `None` per poter analizzare qualsiasi tipo di risoluzione ma lasciando 3 per specificare immagini a colori, 64 features map (filtri convoluzionali interni) e la reduction a 16 che serve nei blocchi di attenzione per comprimere temporaneamente le informazioni e poi espanderle, riducendo il costo computazionale.

Listing 5.14: Compilazione del modello

```

1 model.compile(optimizer=tf.keras.optimizers.Adam(1e-04), loss="
  ↪ mse", metrics=["accuracy"])

```

Il modello viene compilato utilizzando l'ottimizzatore Adam con un tasso di apprendimento di $1e-4$, scelta che permette di aggiornare i pesi della rete in modo stabile ed efficace. La funzione di perdita adottata è la Mean Squared Error (MSE), cioè l'errore quadratico medio, molto comune nei problemi di denoising e ricostruzione di immagini. Inoltre, nella lista delle metriche di valutazione viene inclusa l'accuratezza del modello, utile per monitorare l'andamento dell'addestramento e consentire la generazione di grafici di performance nel tempo.

DnCNN

Qui di seguito l'implementazione iniziale della rete a partire dall'articolo originale in cui viene presentata. **Beyond a Gaussian Denoiser: Residual Learning of Deep CNN for Image Denoising** (K. Zhang, Zuo, Chen et al. 2017)

Definizione della funzione principale

Listing 5.15: Definizione della funzione build_DnCNN

```

1 def build_DnCNN(input_shape):
2     input = Input(input_shape, name = 'input')

```

Definizione della funzione Qui viene definita la funzione `build_DnCNN`, che riceve come parametro la dimensione dell'immagine in ingresso (`input_shape`). Viene quindi creato il *layer di input*, ossia il punto da cui i dati entreranno nella rete.

Primo strato convoluzionale

Listing 5.16: Primo strato convoluzionale

```

1 x = Conv2D(64, kernel_size = (3,3), padding = 'same', name =
  ↪ 'conv2d_l1')(input)
2 x = Activation('relu', name = 'act_l1')(x)

```

Primo strato Viene applicata una convoluzione con 64 filtri 3×3 con padding `same`, seguita da una funzione di attivazione ReLU. Questo consente alla rete di estrarre le prime caratteristiche di basso livello dall'immagine di input.

Blocchi ripetuti

Listing 5.17: Ciclo con blocchi ripetuti

```

1  for i in range(17):
2      x = Conv2D(64, kernel_size = (3,3), padding = 'same',
           ↪ name = 'conv2d_' + str(i))(x)
3      x = BatchNormalization(axis = -1, name = 'BN_' + str(i))(
           ↪ x)
4      x = Activation('relu', name = 'act_' + str(i))(x)

```

Blocchi intermedi La rete esegue un ciclo di 17 blocchi identici, ognuno composto da una convoluzione con 64 filtri, una normalizzazione batch (`BatchNormalization`) per stabilizzare e velocizzare l'allenamento e una funzione di attivazione ReLU per introdurre non-linearità

Questi blocchi permettono di catturare informazioni più profonde e complesse, rendendo la rete capace di modellare il rumore presente nell'immagine.

Strato finale e sottrazione residua

Listing 5.18: Strato finale e output

```

1  x = Conv2D(3, kernel_size = (3,3), padding = 'same', name = '
           ↪ conv2d_l3')(x)
2  x = Subtract(name = 'subtract')([input, x])
3  return Model(input, x)

```

Alla fine, un ultimo strato convoluzionale riduce i canali di uscita a 3 (uno per ogni canale RGB). Successivamente, viene eseguita una sottrazione tra l'immagine di input e il risultato della convoluzione.

Questa operazione implementa il concetto di **residual learning**: la rete impara a stimare il rumore presente nell'immagine, che viene poi sottratto per ottenere un'immagine pulita.

Autoencoder

Definizione e primo livello convoluzionale e pooling

Listing 5.19: Definizione e primo livello convoluzionale

```

1  def build_autoencoder(input_shape):
2      input = Input(input_shape, name = 'Input')
3      x = Conv2D(64, kernel_size = (3,3), kernel_initializer = '
           ↪ he_normal',
4              activation = 'relu', padding = 'same', name = '
           ↪ Conv2d_1')(input)
5      x = MaxPool2D(name = 'Maxpool_1')(x)

```

La funzione `build_autoencoder` riceve come parametro la dimensione delle immagini di input. Il layer di input è il punto da cui i dati vengono forniti alla rete.

Si applica una convoluzione con 64 filtri 3×3 , con inizializzazione dei pesi `he_normal` e funzione di attivazione ReLU.

Segue un'operazione di **max pooling**, che riduce la dimensione spaziale (compressione), mantenendo le informazioni più rilevanti.

Secondo livello di convoluzione e pooling

Listing 5.20: Secondo livello convolutivo + pooling

```

1  x = Conv2D(64, kernel_size = (3,3), kernel_initializer = '
    ↪ he_normal',
2      activation = 'relu', padding = 'same', name = '
    ↪ Conv2d_2')(x)
3  x = MaxPool2D(name = 'Maxpool_2')(x)

```

Viene applicata una nuova convoluzione con le stesse caratteristiche della precedente, seguita da un ulteriore **max pooling**. Questa sequenza riduce ulteriormente le dimensioni spaziali e costituisce la parte di **encoder**, ossia compressione delle informazioni.

Inizio del decoder con upsampling

Listing 5.21: Terzo livello convolutivo + upsampling

```

1  x = Conv2D(64, kernel_size = (3,3), kernel_initializer = '
    ↪ he_normal',
2      activation = 'relu', padding = 'same', name = '
    ↪ Conv2d_3')(x)
3  x = UpSampling2D(name = 'Upsample_1')(x)
4  x = Conv2D(64, kernel_size = (3,3), kernel_initializer = '
    ↪ he_normal',
5      activation = 'relu', padding = 'same', name = '
    ↪ Conv2d_4')(x)
6  x = UpSampling2D(name = 'Upsample_2')(x)

```

Dopo la compressione, inizia la fase di ricostruzione (**decoder**).

Viene applicata una convoluzione seguita da un'operazione di **upsampling**, che aumenta la dimensione spaziale riportandola gradualmente alla risoluzione originale con una convoluzione a 64 filtri seguita da un upsampling, che ricostruisce ulteriormente l'immagine.

Strato finale

Listing 5.22: Strato finale dell'autoencoder

```

1  x = Conv2D(3, kernel_size = (3,3), kernel_initializer = '
    ↪ he_normal',
2      activation = 'relu', padding = 'same', name = '
    ↪ Conv2d_5')(x)
3
4  return Model(input, x)

```

Output della rete Infine, uno strato convoluzionale con 3 filtri (uno per canale RGB) ricostruisce l'immagine di output. La funzione `Model` collega l'input con l'output, restituendo il modello completo.

5.2 Allenamento

In questa sezione vengono descritti i processi di addestramento delle tre architetture analizzate: **RIDNet**, **DnCNN** e **Autoencoder**. Per ciascun modello sono stati definiti i parametri principali di training, come il numero di epoche, la dimensione del batch, la funzione di perdita e l'ottimizzatore. L'allenamento è stato eseguito utilizzando immagini corrotte da diversi livelli di rumore gaussiano, con l'obiettivo di apprendere una mappatura robusta verso la corrispondente immagine pulita. Vengono inoltre discusse le differenze nell'approccio all'addestramento: RIDNet con i suoi blocchi residui e meccanismi di attenzione, DnCNN con la strategia del residuo, e l'Autoencoder come architettura compressiva generica.

Allenamento RIDNet

RIDNet è stato addestrato sfruttando i suoi blocchi residui e i meccanismi di attenzione spaziale e di canale, che consentono di focalizzare l'elaborazione sulle regioni più significative dell'immagine.

Listing 5.23: Creazione della Ridnet

```
1 ridnet = build_RIDNET(  
2     input_shape = (None, None, 3),  
3     n_feats     = 64,  
4     reduction   = 16  
5 )
```

Utilizzando la funzione di creazione della Ridnet [5.23], il modello viene inizializzato con immagini di input di qualsiasi dimensione a 3 canali (RGB), 64 feature map interne e una riduzione di 16 nei blocchi di attenzione. Questo setup permette alla rete di gestire immagini di varie risoluzioni, mantenendo una buona capacità di estrazione delle caratteristiche.

Listing 5.24: Compilazione della Ridnet

```
1 ridnet.compile(  
2     optimizer = tf.keras.optimizers.Adam(1e-04),  
3     loss      = "mse",  
4     metrics   = ['accuracy'])
```

Il modello viene dunque compilato con l'ottimizzatore Adam, noto per la sua efficienza nell'aggiornamento dei pesi, e la funzione di perdita MSE (Mean Squared Error), adatta ai problemi di regressione come il denoising.

Adam con un tasso di apprendimento di $1e^{-4}$ consente un'apprendimento stabile e graduale. Infine le metriche sono impostate per monitorare anche l'accuratezza oltre che alla loss durante l'addestramento.

Listing 5.25: Creazione delle callback

```

1 cp_callback = tf.keras.callbacks.ModelCheckpoint(filepath="
    ↳ best_ridnet.keras", save_weights_only=False, verbose=1,
    ↳ save_best_only=True) # To save the model if the metric is
    ↳ improved
2
3 cp_callback_2 = tf.keras.callbacks.ModelCheckpoint(filepath="
    ↳ ridnet_128_8.keras", save_weights_only=False, verbose=1,
    ↳ save_best_only=False)
4
5 cp_weights_callback = tf.keras.callbacks.ModelCheckpoint(filepath
    ↳ ="best_ridnet_8_128_saed.weights.h5", save_weights_only=
    ↳ True, verbose=1, save_best_only=False)
6
7 # Salva la history in un CSV man mano che procede
8 csv_logger_cb = CSVLogger("ridnet_training_log.csv", append=True)
9
10 tensorboard_callback = tf.keras.callbacks.TensorBoard('.',
    ↳ histogram_freq=1)
11
12
13 def scheduler(lr):
14     return lr * 0.9
15
16 lrScheduler = tf.keras.callbacks.LearningRateScheduler(scheduler)
17
18 callbacks = [
19     cp_callback,
20     cp_callback_2
21     cp_weights_callback,
22
23     csv_logger_cb,
24
25     tensorboard_callback,
26     lrScheduler,
27 ]

```

Qui si vanno a creare le **callback** utilizzate durante l'addestramento:

- **ModelCheckpoint**: per salvare il modello ad ogni iterazione.
 - **cp_callback**: salva il modello solo se la metrica di validazione migliora. Utilizzato per monitorare il miglioramento effettivo.
 - **cp_callback_2**: salva il modello ad ogni epoca, indipendentemente dal miglioramento. Utilizzato per avere il modello completo di tutte le epoche e anche dei non miglioramenti.
- **CSVLogger**: per registrare la storia dell'addestramento in un file CSV.
- **TensorBoard**: per visualizzare metriche e grafici durante l'addestramento.

- `LearningRateScheduler`: per modificare il tasso di apprendimento in base a una funzione definita.

La funzione scheduler riduce il tasso di apprendimento del 90% ogni epoca, favorendo una convergenza più stabile.

Listing 5.26: Compilazione della Ridnet

```

1 history_ridnet = model.fit(
2     train_ds_noisy,
3     #initial_epoch = 8,
4     epochs=8,
5     validation_data=val_ds_noisy,
6     callbacks=callbacks)

```

L'addestramento viene eseguito per 8 epoche, con valutazione continua delle prestazioni su un set di validazione.

Le callback definite in precedenza vengono utilizzate per monitorare e salvare i progressi del modello.

Listing 5.27: Caricamento e nuovo train della ridnet

```

1 custom_objects = {
2     'BasicBlock': BasicBlock,
3     'BasicBlockSig': BasicBlockSig,
4     'Merge_Run_dual': Merge_Run_dual,
5     'ResidualBlock': ResidualBlock,
6     'EResidualBlock': EResidualBlock,
7     'MeanShift': MeanShift,
8     'CALayer': CALayer,
9     'Block': Block,
10    'build_RIDNET': build_RIDNET,
11    'shift': MeanShift(1.0, (0.4488, 0.4371, 0.4040), (1.0, 1.0,
12    ↪ 1.0)).shift_fn} # Access the inner function
13
14 ridnet = load_model("best_ridnet.keras")
15
16 #[...] callbacks definition [...]
17 df = pd.read_csv('ridnet_training_log_15.csv')
18 best_value = df["val_loss"].iloc[-1]
19
20 cp_callback.best = best_value
21 cp_callback_2.best = best_value
22 callbacks = [...] # same as before
23
24 history_2 = model.fit(train_ds_noisy,
25     initial_epoch = 8, # next: 16
26     epochs = 16, # next: 23
27     validation_data = val_ds_noisy,
28     callbacks = callbacks)

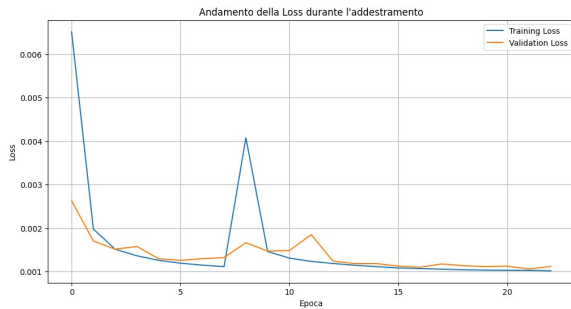
```

Il modello viene caricato, e vengono salvati in memoria anche tutti i blocchi personalizzati e le funzioni di supporto utilizzate per costruire la rete, in modo che il caricamento del modello

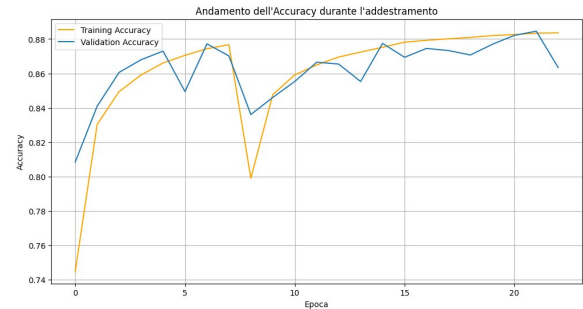
avvenga senza errori.

Viene caricato anche il csv con la history dell'allenamento precedente per poter prenderel'ultimo valore tra tutte le entry di validazione della loss, e venire ricaricato come valore da cui ripartire per il salvataggio del modello.

Infine viene eseguito un nuovo ciclo di addestramento per altre 8 epoche, partendo dall'epoca 8 fino alla 16, sempre con valutazione continua su un set di validazione e con le callback attive. Il successivo e ultimo train parte dall'epoca 16 e arriva a 23 epoche totali senza aver cambiato nulla tra callback e il resto delle opzioni.



(a) Loss di training e validation della Ridnet



(b) Accuracy di train e validation della RID-Net

Figura 5.1: Andamento delle curve di addestramento della RIDNet.

Allenamento DnCNN

DnCNN è stato addestrato per apprendere il rumore residuo da sottrarre all'immagine rumorosa. La presenza di *batch normalization* e ReLU ha permesso una stabilizzazione del training.

Listing 5.28: Creazione, compilazione e training di DnCNN

```
1 dncnn = build_DnCNN(input_shape=(None, None, 3))
```

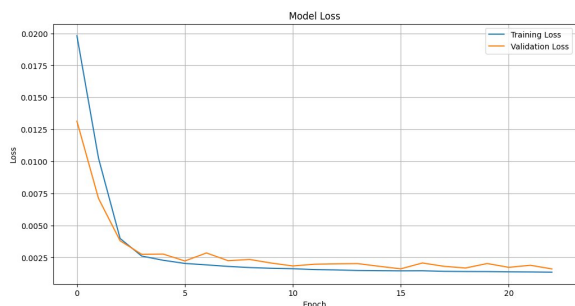
Creazione del modello DnCNN con immagini di input di qualsiasi dimensione a 3 canali (RGB) usando la funzione di creazione [5.15].

```
1 dncnn.compile(optimizer=tf.keras.optimizers.Adam(1e-04), loss="
   ↪ mse", metrics=['accuracy'])
2
3 history_dncnn = dncnn.fit(train_ds_noisy,
4                           epochs=8,
5                           validation_data=val_ds_noisy,
6                           callbacks=callbacks)
7
8 history_dncnn = dncnn.fit(train_ds_noisy,
9                           epochs=13, # next: 14
10                          initial_epoch=8, # next: 23
11                          validation_data=val_ds_noisy,
12                          callbacks=callbacks)
```

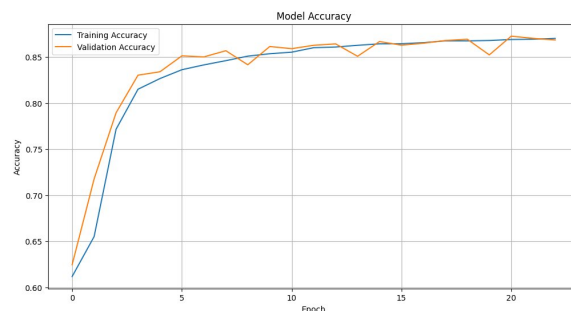
La compilazione avviene nello stesso metodo della Ridnet, con Adam, MSE e accuratezza.

Le callback sono le stesse definite in precedenza per la ridnet[5.25].

L'addestramento viene eseguito esattamente come per la ridnet, per 8 epoche, poi come per la ridnet si usano i checkpoint per arrivare a 23 epoche totali.



(a) Loss di training e validation della DnCNN



(b) Accuracy di training e validation della DnCNN

Figura 5.2: Andamento delle curve di addestramento di DnCNN.

Allenamento Autoencoder

L'Autoencoder è stato allenato come architettura compressiva, sfruttando la fase di codifica e decodifica per ridurre il rumore durante la ricostruzione dell'immagine.

Listing 5.29: Creazione, compilazione e training di un Autoencoder

```

1 model = build_autoencoder(input_shape=(None, None, 3))
2
3 model.compile(optimizer=tf.keras.optimizers.Adam(1e-04), loss="
  ↪ mse", metrics=metrics )

```

Il modello viene creato utilizzando la funzione di costruzione dell'Autoencoder [5.19], con immagini di input di qualsiasi dimensione a 3 canali (RGB).

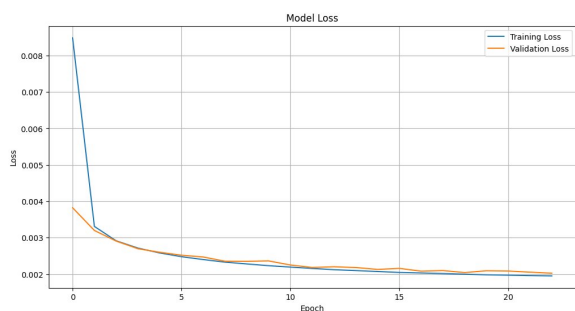
La compilazione avviene nello stesso modo delle reti precedenti, con Adam, MSE e accuratezza.

```

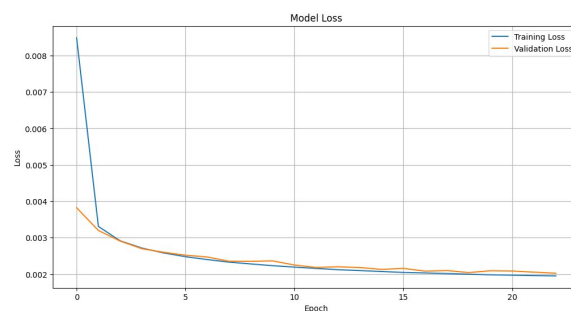
1 history_2 = model.fit(train_ds_noisy,
2                       epochs=23,
3                       validation_data=val_ds_noisy,
4                       callbacks=callbacks)

```

L'addestramento dell'Autoencoder è stato eseguito per 23 epoche direttamente poichè più veloce da addestrare, con valutazione continua su un set di validazione e con le callback attive come per i modelli precedenti.



(a) Loss di training e validation dell'Autoencoder



(b) Accuracy di training e validation dell'Autoencoder

Figura 5.3: Andamento delle curve di addestramento dell'Autoencoder.

BM3D

BM3D (Block-Matching and 3D Filtering) è un algoritmo di denoising basato su tecniche di filtraggio collaborativo in domini trasformati.

Non richiede un processo di addestramento come le reti neurali, ma si basa su principi matematici e statistici per ridurre il rumore nelle immagini. BM3D funziona raggruppando blocchi simili nell'immagine e applicando un filtraggio 3D per migliorare la qualità visiva.

È noto per la sua efficacia nel preservare i dettagli dell'immagine durante il processo di denoising. In questo studio, BM3D è stato utilizzato come baseline di confronto per valutare le prestazioni delle reti neurali addestrate.

Di seguito vengono mostrate le due implementazioni di BM3D, una semplice che richiede l'input di un dataset rumoroso con coppie (noisy, gt) per stimare il livello di rumore, e una più complessa che richiede l'input del dataset rumoroso e il livello di rumore noto.

Listing 5.30: Definizione di BM3D

```

1 def bm3d_psnr_plot_paired(test_ds_noisy_paired, plot_num=5):
2     print("Estimating noise level...")
3     noise_levels = []
4     for batch in test_ds_noisy_paired.take(1):
5         x_noisy_batch, x_original_batch = batch
6         # Calculate the difference between noisy and original
7         # → images
8         noise = x_noisy_batch - x_original_batch
9         for i in range(min(16, x_noisy_batch.shape[0])):
10             patch = noise[i, 0:32, 0:32, :] # Take a small patch
11             noise_levels.append(np.std(patch.numpy()))
12
13     estimated_sigma = np.mean(noise_levels) * 255.0 # Convert back
14     # → to 0-255 range
15     print(f"Estimated noise level (sigma_psd) for BM3D: {
16         # → estimated_sigma:.4f}")

```

In questa prima parte della funzione `bm3d_psnr_plot_paired`, viene stimato il livello di rumore (`sigma_psd`) analizzando le differenze tra le immagini rumorose e quelle originali in un batch del dataset. Essendo che passare tutta l'immagine potrebbe essere computazionalmente costoso, si prendono in considerazione solo piccole patch di dimensione 32×32 da un massimo di 16 immagini per ottenere una stima più robusta. La media delle deviazioni standard calcolate dalle patch viene moltiplicata per 255 per riportare il valore alla scala originale dei pixel (0-255).

Listing 5.31: Stima e denoising delle immagini con BM3D

```

1  for batch in test_ds_noisy_paired.take(1):
2      x_noisy_batch, x_original_batch = batch
3      break
4
5  noisy_np = x_noisy_batch.numpy()
6  original_np = x_original_batch.numpy()
7
8  # Denoising con BM3D usando il sigma stimato
9  decoded_imgs = []
10 for i in tqdm(range(len(noisy_np)), desc="Applying BM3D"):
11     # BM3D attende immagini in uint8 con valori tra 0 e 255
12     noisy_img_uint8 = (noisy_np[i] * 255).astype(np.uint8)
13     denoised_img = bm3d.bm3d(noisy_img_uint8, sigma_psd=
14         ↪ estimated_sigma, stage_arg=bm3d.BM3DStages.ALL_STAGES)
15     decoded_imgs.append(denoised_img.astype(np.float32) / 255.0)
16
17 decoded_imgs = np.array(decoded_imgs)
18
19 return decoded_imgs

```

Una volta stimato il livello di rumore, la funzione procede a denoising delle immagini rumorose nel batch. Per ogni immagine:

- L'immagine rumorosa viene convertita in formato `uint8` con valori tra 0 e 255, come richiesto dalla libreria BM3D.
- Viene applicato l'algoritmo BM3D utilizzando il livello di rumore stimato.
- L'immagine denoised viene convertita nuovamente in formato `float32` con valori normalizzati tra 0 e 1.

Le immagini denoised vengono raccolte in una lista e convertite in un array NumPy, che viene restituito come output della funzione.

Listing 5.32: Definizione di BM3D con rumore definito

```

1  def bm3d_psnr_plot_gt(test_ds_gt, noise_levels = [15 / 255.0, 25
2      ↪ / 255.0, 50 / 255.0], plot_num=5):
3      for level in noise_levels:
4          print(f"\nProcessing with noise level: {level:.4f}")
5          # Add noise to the ground truth images
6          noisy_ds = test_ds_gt.map(lambda sample: (add_noise_2(sample,
7              ↪ level), sample))
8          for batch in noisy_ds.take(1):
9              x_noisy_batch, x_original_batch = batch
10             break
11         noisy_np = x_noisy_batch.numpy()
12         original_np = x_original_batch.numpy()

```

A differenza della funzione precedente, in questa parte della funzione `bm3d_psnr_plot_gt`, viene aggiunto rumore gaussiano alle immagini di ground truth per simulare immagini rumorose a diversi livelli di rumore specificati nella lista `noise_levels`.

Listing 5.33: Denoising con BM3D

```

1  decoded_imgs = []
2  # Convert sigma from [0,1] range to [0,255] range for BM3D
3  sigma_psd_bm3d = level * 255.0
4  for i in tqdm(range(len(noisy_np)), desc=f"Applying BM3D (
    ↳ sigma={sigma_psd_bm3d:.2f})):
5      # BM3D attende immagini in uint8 con valori tra 0 e 255
6      noisy_img_uint8 = (noisy_np[i] * 255).astype(np.uint8)
7      denoised_img = bm3d.bm3d(noisy_img_uint8, sigma_psd=
    ↳ sigma_psd_bm3d, stage_arg=bm3d.BM3DStages.ALL_STAGES)
8      # Converti di nuovo a float32 in [0, 1]
9      decoded_imgs.append(denoised_img.astype(np.float32) /
    ↳ 255.0)
10 decoded_imgs = np.array(decoded_imgs)
11 return decoded_imgs

```

Il processo di denoising è simile a quello della funzione precedente, ma qui il livello di rumore (`sigma_psd_bm3d`) viene calcolato direttamente dal livello di rumore specificato nella lista, moltiplicato per 255 per adattarsi alla scala richiesta da BM3D. Le immagini denoised vengono raccolte e restituite come array NumPy.

5.3 Valutazioni

Per valutare l'efficacia dei modelli è stato utilizzato il **PSNR**, una metrica standard per quantificare la qualità delle immagini ricostruite rispetto a quelle di riferimento.

Le valutazioni sono state effettuate su tre diversi livelli di rumore, al fine di testare la capacità dei modelli di generalizzare a condizioni più o meno difficili.

I risultati sono confrontati tra loro e con **BM3D**, considerato uno dei baseline più solidi nel campo del denoising.

L'analisi consente di evidenziare punti di forza e limiti di ciascuna architettura in scenari controllati.

Listing 5.34: Calcolo di PSNR

```

1 psnrs = [psnr(original_np[i], decoded_imgs[i], data_range=1.0)
    ↳ for i in range(len(decoded_imgs))]

```

Il calcolo del PSNR viene effettuato confrontando ogni immagine originale con la corrispondente immagine denoised, utilizzando la funzione `psnr` della libreria `skimage.metrics`.

Il parametro `data_range=1.0` indica che i valori dei pixel sono normalizzati tra 0 e 1.

Il PSNR è definito come il rapporto tra il massimo valore possibile di un segnale e il valore del rumore che disturba la qualità della sua rappresentazione.

Normalmente misurato in una scala logaritmica in decibel(dB).

Qui di seguito sono riportati i risultati ottenuti in termini di PSNR per ciascun modello e dataset, suddivisi per dataset, livelli di rumore e modelli, i cui iperparametri sono stati scelti in accordo a quelli proposti dagli autori negli articoli di riferimento.

Dataset Rumore		Modelli											
		Ridnet			DnCNN			Autoencoder					
		Max	Min	Avg	Max	Min	Avg	Max	Min	Avg	Max	Min	Avg
Renoir	$\sigma = 10.49$	40.09	25.18	30.85	35.36	21.40	28.18	37.20	21.32	29.49	46.31	18.33	32.37
SIDD	$\sigma = 7.22$	37.84	22.38	30.13	34.93	19.68	27.31	34.43	21.64	28.11	45.36	19.99	36.05
BSD500	$\sigma = 15$	31.46	27.73	29.57	28.98	26.39	27.72	27.74	17.51	22.48	31.37	25.63	28.31
	$\sigma = 25$	29.10	24.69	26.85	26.96	23.55	25.18	26.83	17.38	22.16	28.74	22.29	25.35
	$\sigma = 50$	25.93	20.66	23.35	23.54	19.33	21.46	24.69	16.94	21.21	25.89	18.22	21.79
FiveK	$\sigma = 15$	35.02	20.57	30.05	31.23	17.99	26.88	33.07	20.50	27.43	44.41	26.38	34.09
	$\sigma = 25$	32.39	19.58	27.76	28.79	16.42	24.61	31.59	19.27	26.03	42.29	23.49	31.51
	$\sigma = 50$	28.54	17.15	24.38	24.27	11.69	20.39	28.03	16.38	23.79	37.65	20.73	27.35
Div2K	$\sigma = 15$	35.17	23.36	29.76	31.39	15.15	27.41	34.89	18.18	23.32	42.39	25.88	29.82
	$\sigma = 25$	31.94	21.17	27.07	29.08	13.45	24.98	33.02	18.04	22.82	37.13	22.58	26.79
	$\sigma = 50$	28.43	18.87	23.58	24.96	9.52	21.27	28.80	17.26	21.73	28.82	18.74	22.77

Tabella 5.1: Confronto PSNR (Max, Min, Avg) per dataset e modello. I dataset non noisy sono valutati su tre livelli di rumore: $\sigma = 15$, $\sigma = 25$, $\sigma = 50$.

I risultati qualitativi sono riportati in Appendice A.

La tabella riporta i risultati ottenuti in termini di PSNR (Peak Signal-to-Noise Ratio) per quattro modelli di denoising — Ridnet, DnCNN, Autoencoder e BM3D — su diversi dataset e livelli di rumore. I valori sono presentati come massimo (Max), minimo (Min) e valore medio (Avg) per ciascun modello nei vari contesti.

Osservazioni generali

Ridnet mostra generalmente i valori medi e massimi di PSNR più elevati su quasi tutti i dataset e livelli di rumore, indicando una capacità superiore di preservare la qualità dell'immagine denoised.

BM3D, pur essendo un metodo classico non basato su deep learning, presenta spesso il valore massimo di PSNR più alto in alcuni dataset come Renoir e SIDD, ma con una media e un minimo generalmente inferiori rispetto a Ridnet. Questo può indicare che BM3D ottiene buone prestazioni in condizioni favorevoli ma meno costanti su immagini diverse.

DnCNN ottiene risultati competitivi, con valori medi spesso superiori a quelli di Autoencoder, ma inferiore rispetto a Ridnet in termini di prestazioni medie.

Autoencoder tende ad avere i valori di PSNR più bassi, soprattutto per i livelli più alti di rumore, suggerendo una minore efficacia nel preservare dettagli ed eliminare rumore.

Variazione con il livello di rumore

Come atteso, il PSNR medio diminuisce all'aumentare del livello di rumore (α da 15 a 50), riflettendo una maggiore difficoltà nel recupero dell'immagine originale. Questo trend è presente in tutti i modelli e dataset.

Ridnet mantiene comunque un vantaggio consistente anche ai livelli di rumore più elevati, dimostrando robustezza.

Differenze tra dataset

Dataset con rumore più basso come Renoir e SIDD evidenziano PSNR complessivamente più elevati, indicando che il rumore meno aggressivo permette un recupero di qualità superiore.

Dataset di riferimento più "difficili" o con rumore maggiore come BSD500, FiveK e Div2K mostrano un calo più marcato di PSNR, evidenziando la complessità delle immagini e la sfida del denoising in questi contesti.

Dalla tabella si evince che Ridnet è il modello più performante e stabile tra quelli valutati, seguito da DnCNN. BM3D, pur mantenendo punte elevate in alcuni casi, è meno consistente, mentre gli Autoencoder appaiono meno efficaci in termini di qualità media del recupero.

5.4 Fine Tuning

Oltre all'addestramento iniziale, è stato condotto un processo di **fine tuning** utilizzando il **Kvasir dataset**, composto da immagini endoscopiche reali. Lo scopo di questa fase è valutare la capacità dei modelli di adattarsi a domini specifici, caratterizzati da rumore e texture differenti rispetto ai dataset sintetici. Il fine tuning permette infatti di affinare i pesi delle reti in modo da specializzarle al contesto medico, migliorando la loro efficacia nella rimozione del rumore in immagini endoscopiche.

Listing 5.35: Definizione nuovo learning scheduler

```

1 def lr_schedule(epoch):
2     """Linear warmup + cosine decay"""
3     if epoch < warmup_epochs:
4         return initial_lr * (epoch / warmup_epochs)
5     else:
6         decay_epochs = total_epochs - warmup_epochs
7         cosine_decay = 0.5 * (1 + np.cos(np.pi * (epoch -
8             ↪ warmup_epochs) / decay_epochs))
9         return initial_lr * cosine_decay

```

Per il fine tuning, è stato implementato un nuovo **learning rate scheduler** che combina un riscaldamento lineare iniziale con un decadimento coseno successivo [5.35]. Questa strategia consente di iniziare l'addestramento con un tasso di apprendimento basso, per poi aumentarlo gradualmente, riducendolo infine in modo più morbido. Questo approccio aiuta a stabilizzare l'addestramento, evitando oscillazioni e favorendo una convergenza più efficace verso un minimo locale ottimale.

Listing 5.36: Ottimizzatore e compilazione per fine tuning

```

1 initial_lr = 1e-5
2 warmup_epochs = 5
3 optimizer = tf.keras.optimizers.Adam(learning_rate=initial_lr) #
    ↳ Usa initial_lr dallo scheduler
4 model.compile(optimizer=optimizer, loss="mse", metrics=['accuracy
    ↳ '])

```

Il tasso di apprendimento iniziale è stato impostato a $1e^{-5}$, con un riscaldamento lineare per le prime 5 epoche.

I modelli sono nuovamente compilati con l'ottimizzatore Adam con il nuovo learning rate e la funzione di perdita MSE.

Listing 5.37: Callback per ridurre il LR

```

1 reduce_lr = tf.keras.callbacks.ReduceLROnPlateau(
2     monitor='val_loss',
3     factor=0.2,
4     patience=5,
5     verbose = 1,
6     min_lr=1e-7
7 )

```

In aggiunta, è stata introdotta una callback `ReduceLROnPlateau` per ridurre ulteriormente il tasso di apprendimento se la metrica di validazione non migliora per 5 epoche consecutive.

Questo aiuta a evitare stagnazioni durante il fine tuning, permettendo al modello di adattarsi meglio ai nuovi dati.

Listing 5.38: Fine tuning dei modelli

```

1 callbacks = [
2     cp_callback,
3     cp_callback_2,
4     tensorboard_callback,
5     lrScheduler,
6     csv_logger_cb,
7     reduce_lr
8 ]
9
10 dncnn_finetune = dncnn.fit(
11     train_ds_noisy,
12     validation_data=val_ds_noisy,
13     epochs=23,
14     callbacks=callbacks
15 )

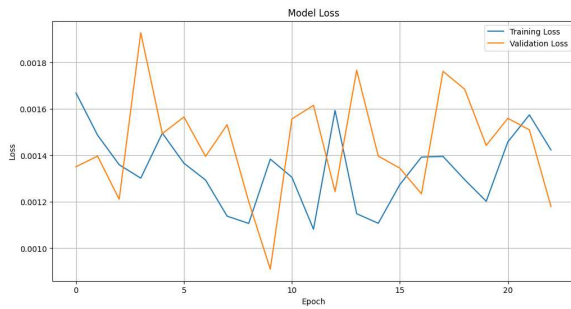
```

Nelle callback per il fine tuning sono incluse quelle definite in precedenza per salvare il modello a ogni epoca e il csv con i dati di allenamento, con l'aggiunta di `reduce_lr` per la riduzione del learning rate.

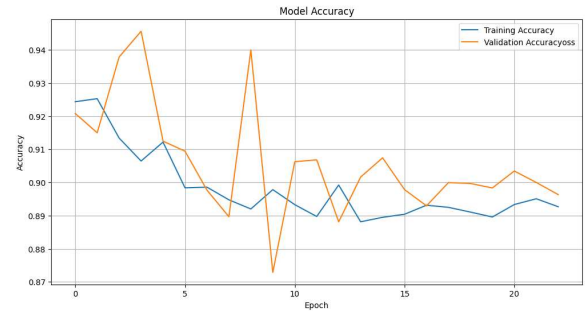
Il fine tuning viene eseguito a partire da capo per 23 epoche in modo da coprire l'intero intervallo di apprendimento, con valutazione continua.

Grafici di fine tuning

Qui di seguito sono riportati i grafici delle curve di loss e accuracy per ciascun modello durante il fine tuning sul dataset Kvasir.

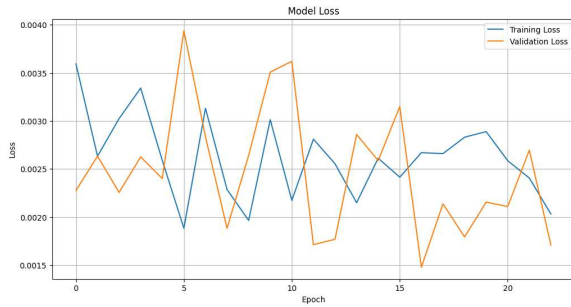


(a) Loss di training e validation dell'Autoencoder

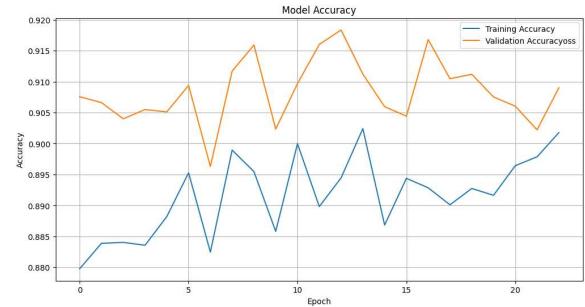


(b) Accuracy di training e validation dell'Autoencoder

Figura 5.4: Andamento delle curve di fine tuning della Ridnet.

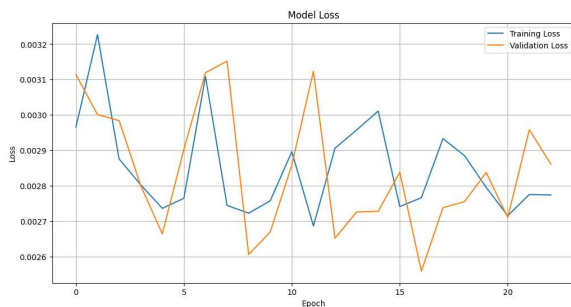


(a) Loss di training e validation dell'Autoencoder

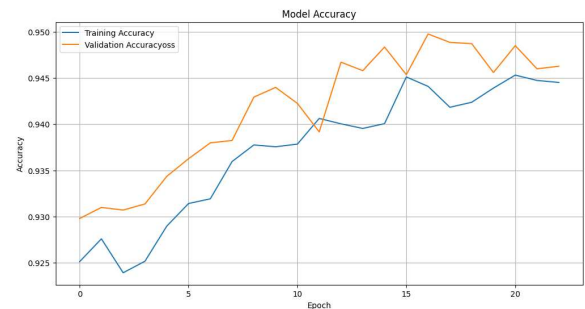


(b) Accuracy di training e validation dell'Autoencoder

Figura 5.5: Andamento delle curve di fine tuning della DnCNN.



(a) Loss di training e validation dell'Autoencoder



(b) Accuracy di training e validation dell'Autoencoder

Figura 5.6: Andamento delle curve di finetuning dell'Autoencoder.

Valutazione risultati di denoising pre e post fine-tuning

Di seguito sono riportati i risultati ottenuti in termini di PSNR per ciascun modello prima e dopo il fine tuning sul dataset Kvasir, suddivisi per livelli di rumore e modelli.

Dataset	Rumore	Modelli								
		Ridnet			DnCNN			Autoencoder		
		Max	Min	Avg	Max	Min	Avg	Max	Min	Avg
Kvasir pre fine tuning	$\sigma = 15$	33.66	29.81	31.67	31.48	27.39	29.50	26.03	29.84	22.74
	$\sigma = 25$	31.72	27.17	29.29	29.28	25.42	27.28	29.10	22.54	25.57
	$\sigma = 50$	26.06	28.57	23.92	25.28	22.19	23.51	27.41	21.77	24.39
Kvasir post fine tuning	$\sigma = 15$	32.18	34.30	30.26	32.49	28.84	30.48	30.45	22.86	26.29
	$\sigma = 25$	32.38	27.57	29.86	30.33	26.62	28.22	29.78	22.62	25.91
	$\sigma = 50$	28.92	24.32	26.56	25.91	22.76	24.16	27.95	21.96	24.76

Tabella 5.2: Confronto PSNR (Max, Min, Avg) pre e post fine tuning sul dataset Kvasir. Il dataset essendo non noisy è valutato sui tre livelli di rumore: $\sigma = 15$, $\sigma = 25$, $\sigma = 50$.

La tabella mostra i valori di PSNR ottenuti da tre modelli di denoising — Ridnet, DnCNN e Autoencoder — sul dataset Kvasir, prima e dopo il fine tuning, con diversi livelli di rumore $\sigma=15,25,50$. Per ciascun modello e livello di rumore sono riportati i valori massimo, minimo e medio di PSNR.

Dopo il fine tuning sul dataset Kvasir, tutti i modelli mostrano un miglioramento significativo delle prestazioni in termini di PSNR rispetto alla fase preliminare (pre - fine tuning).

Ridnet continua a presentare i valori medi e massimi di PSNR più alti, confermandosi il modello più performante su questo dataset anche dopo l'adattamento.

DnCNN mostra anch'esso un miglioramento netto post - fine tuning, con un incremento medio di PSNR significativo per tutti i livelli di rumore.

L'Autoencoder beneficia in misura minore del fine tuning, ma migliora comunque in modo consistente i valori medi e massimi di PSNR.

Come prevedibile, il PSNR medio diminuisce all'aumentare del livello di rumore σ per tutti i modelli, ma l'efficacia del fine tuning attenua questo calo.

In particolare, a $\sigma = 50$, il miglioramento post fine tuning è più marcato, segnalando che i modelli riescono a gestire meglio rumori elevati dopo l'adattamento specifico al dataset Kvasir.

Ridnet mantiene un margine di prestazioni superiore rispetto a DnCNN e Autoencoder in quasi tutte le condizioni, con valori massimi anche molto alti (es. oltre 34 per $\sigma = 15$ dopo fine tuning).

Risulta evidente che il fine tuning su dataset specifici come Kvasir è cruciale per migliorare la capacità di generalizzazione e l'efficacia pratica dei modelli di denoising, specialmente in applicazioni reali con rumori complessi e caratteristiche particolari.

Confronto con BM3D

Rumore		BM3D		
		Max	Min	Avg
Kvasir	$\sigma = 15$	33.22	27.85	30.52
	$\sigma = 25$	30.38	24.83	27.43
	$\sigma = 50$	25.56	21.12	23.60

Tabella 5.3: Statistiche di BM3D come confronto per il fine tuning dei modelli.

Confrontando i risultati PSNR del modello BM3D sul dataset Kvasir con quelli precedenti dei modelli Ridnet, DnCNN e Autoencoder, emergono diverse considerazioni:

Il valore medio di PSNR di BM3D a ciascun livello di rumore ($\sigma=15, 25, 50$) sul dataset Kvasir si colloca generalmente tra il modello Autoencoder e DnCNN, risultando inferiore a Ridnet ma superiore all'Autoencoder. Per esempio, a $\sigma = 15$ BM3D ottiene un valore medio di PSNR pari a circa 30.52, che è leggermente inferiore al Ridnet post fine tuning (31.67) e vicino al DnCNN (29.50), ma superiore all'Autoencoder (22.74).

BM3D mantiene un valore massimo di PSNR competitivo con Ridnet, soprattutto a bassi livelli di rumore, ma i valori minimi indicano una maggiore variabilità rispetto a Ridnet e DnCNN, suggerendo meno stabilità sulle diverse immagini.

Rispetto ai modelli deep learning, BM3D mostra prestazioni inferiori nel valore medio di PSNR, confermando che, pur essendo ancora un riferimento solido e efficace, i moderni modelli basati su reti neurali profonde come Ridnet sono generalmente più efficaci nel denoising su dataset complessi come Kvasir.

Nell'appendice Appendice B sono riportati i risultati qualitativi ottenuti sui diversi dataset con i tre modelli di denoising addestrati: **Ridnet**, **DnCNN** e **Autoencoder**, e **BM3D**.

5.5 Ottimizzazione degli iperparametri

Infine, è stato svolto uno **studio di ottimizzazione degli iperparametri** mediante la libreria **Optuna**, uno strumento di ottimizzazione automatica basato su ricerca bayesiana ed evolutiva. Lo scopo di questo studio è identificare le configurazioni ottimali di parametri quali il learning rate, il numero di filtri convoluzionali e la profondità della rete, al fine di massimizzare le prestazioni in termini di accuratezza e PSNR. Questa analisi sistematica consente di comprendere meglio la sensibilità dei modelli ai diversi settaggi e di definire strategie di addestramento più robuste ed efficienti.

Studio iperparametri finetuning per Ridnet

Il primo caso di studio riguarda il modello Ridnet, con l'obiettivo di ottimizzare gli iperparametri per il fine tuning sul dataset Kvasir. Gli iperparametri considerati includono:

- **Numero di filtri convoluzionali (n_feats)**: valori possibili 32, 64, 128. Che determina la capacità di estrazione delle caratteristiche del modello.

- **Fattore di riduzione** (reduction): valori possibili 8, 16, 32. Che influenza la compressione delle informazioni all'interno della rete.

Listing 5.39: Definizione della funzione obiettivo

```

1 def objective(trial):
2     # Spazio di ricerca
3     n_feats = trial.suggest_categorical("n_feats", [32, 64, 128])
4     reduction = trial.suggest_categorical("reduction", [8, 16,
5         ↪ 32])
6     model = build_RIDNET(input_shape=(None, None, 3), n_feats=
7         ↪ n_feats, reduction = reduction)
8     model.compile(optimizer=keras.optimizers.Adam(),
9         loss="mse", metrics=["accuracy"] )

```

La funzione obiettivo `objective` definisce lo spazio di ricerca degli iperparametri utilizzando le funzioni di Optuna.

Per ogni trial, viene costruito un nuovo modello Ridnet a cui è stata modificata la funzione di build per poter accettare i nuovi iperparametri.

Il modello viene quindi compilato allo stesso modo di prima.

Listing 5.40: Corpo funzione obiettivo

```

1     cp_callback = tf.keras.callbacks.ModelCheckpoint(filepath=f"
2         ↪ ridnet_trial_{trial.number}.keras", save_weights_only=
3         ↪ False, verbose=1, save_best_only=True)
4
5     cp_callback_weights = tf.keras.callbacks.ModelCheckpoint(
6         ↪ filepath=f"ridnet_trial_{trial.number}_weight.keras.h5"
7         ↪ , save_weights_only=True, verbose=1, save_best_only=
8         ↪ True)
9
10    # Addestramento rapido con early stopping (per velocizzare la
11    ↪ ricerca)
12    history = model.fit(
13        train_ds_noisy,
14        validation_data=val_ds_noisy,
15        epochs=10,
16        callbacks=[
17            keras.callbacks.EarlyStopping(patience=3,
18                ↪ restore_best_weights=True),
19            cp_callback, cp_callback_weights
20        ]
21    )
22
23    # Ritorna la loss di validazione media sull'ultima epoca
24    val_loss = history.history["val_loss"][-1]
25    return val_loss

```

Per ogni trial, viene definito un callback per salvare il modello e i pesi migliori durante l'addestramento.

Il modello viene addestrato per un numero limitato di epoche (10) con early stopping per velocizzare la ricerca.

La funzione obiettivo ritorna la loss di validazione media dell'ultima epoca, che Optuna cercherà di minimizzare.

Listing 5.41: Lancio dello studio Optuna

```

1 # Lancio dello studio Optuna
2 study = optuna.create_study(direction="minimize", study_name='
    ↪ fine_tuning_ridnet-study')
3 study.optimize(objective, n_trials=30)
4
5 print("Migliori parametri:", study.best_params)
6 print("Migliore loss:", study.best_value)

```

Lo studio viene creato specificando che l'obiettivo è minimizzare la loss di validazione.

Vengono eseguiti 30 trial per esplorare lo spazio degli iperparametri.

Avendo definito due serie da 3 parametri ciascuno, il numero totale di combinazioni possibili è 9 e per poterle esplorare tutte almeno 3 volte ciascuna, sono stati eseguiti 30 trial.

Al termine, vengono stampati i migliori parametri trovati e il valore della loss corrispondente.

Lo studio è stato dunque completato con iperparametri migliori per il fine tuning di Ridnet:

- trial = 11
- n_feats = 32
- reduction = 8

Con un valore di loss di validazione pari a 0.010027101263403893.

Studio iperparametri addestramento per DnCNN

Per la **DnCNN**, lo studio di ottimizzazione degli iperparametri si è concentrato su:

- **Numero di filtri convoluzionali (n_feats)**: valori possibili 32, 64. Che determina la capacità di estrazione delle caratteristiche del modello.
- **Funzione di attivazione (activation)**: valori possibili "relu", "leaky_relu". Che influenza la non linearità introdotta nel modello.
- **Parametro alpha per LeakyReLU (alpha)**: valori possibili 0.1, 0.2, 0.3. Che determina la pendenza della funzione di attivazione LeakyReLU.

Per un totale di 12 combinazioni possibili. Lo studio è stato condotto sull'addestramento iniziale sui 5 dataset iniziali, piuttosto che sul fine tuning, per valutare la robustezza della DnCNN in un contesto più generale.

Listing 5.42: Definizione della funzione obiettivo per DnCNN

```

1 def objective(trial):
2     # Spazio di ricerca
3     n_feats = trial.suggest_categorical("n_feats", [32, 64])
4     activation = trial.suggest_categorical("activation", ["relu",
5         ↪ "leaky_relu"])
6
7     # costruisci l'oggetto attivazione
8     if activation == "relu":
9         act_layer = tf.keras.layers.ReLU()
10    else: # leaky_relu
11        alpha = trial.suggest_float("alpha", 0.1, 0.3, step=0.1)
12        act_layer = tf.keras.layers.LeakyReLU(alpha=alpha)
13
14    model = build_DnCNN(input_shape=(None, None, 3), activia=
15        ↪ act_layer, n_feats=n_feats)

```

La funzione obiettivo per DnCNN è simile a quella di Ridnet, ma con l'aggiunta della scelta della funzione di attivazione e del parametro alpha per LeakyReLU.

Il modello viene costruito utilizzando questi iperparametri e una versione modificata della funzione di build per accettare la nuova attivazione. Per il resto, la compilazione e l'addestramento sono analoghi a quelli di Ridnet con l'unica differenza di un Early Stop con pazienza 3.

Dello studio sono state completate 18 trial su 30 totali, con i seguenti migliori iperparametri:

- trial = 13
- n_feats = 32
- activation = 'relu'
- alpha = None

Con un valore di loss di validazione pari a 0.00201316736638546.

Il trial 18 aveva gli stessi iperparametri ma un valore di loss più alto(4.324199199676514), probabilmente a causa della natura stocastica dell'addestramento delle reti neurali.

Studio iperparametri addestramento per Autoencoder

Per la **Autoencoder**, lo studio di ottimizzazione degli iperparametri si è concentrato su:

- **Numero di filtri convoluzionali (n_feats)**: valori possibili 32, 64, 128. Che determina la capacità di estrazione delle caratteristiche del modello.
- **Funzione di attivazione (activation)**: valori possibili "relu", "leaky_relu". Che influenza la non linearità introdotta nel modello.
- **Parametro alpha per LeakyReLU (alpha)**: valori possibili 0.1, 0.2, 0.3. Che determina la pendenza della funzione di attivazione LeakyReLU.

Per un totale di 18 combinazioni possibili. Lo studio è stato condotto sull'addestramento iniziale sui 5 dataset iniziali, piuttosto che sul fine tuning, per valutare la robustezza dell'Autoencoder in un contesto più generale.

Listing 5.43: Definizione della funzione obiettivo per Autoencoder

```

1 def objective(trial):
2     # Spazio di ricerca
3     n_feats = trial.suggest_categorical("n_feats", [32, 64, 128])
4     activation = trial.suggest_categorical("activation", ["relu",
5         ↪ "leaky_relu"])
6
7     # costruisci l'oggetto attivazione
8     if activation == "relu":
9         act_layer = tf.keras.layers.ReLU()
10    else: # leaky_relu
11        alpha = trial.suggest_float("alpha", 0.1, 0.3, step=0.1)
12        act_layer = tf.keras.layers.LeakyReLU(alpha=alpha)
13
14    model = build_autoencoder(input_shape=(None, None, 3),
15        ↪ n_feats=n_feats,
16        activation = act_layer)

```

La funzione obiettivo per Autoencoder è uguale a quella di DnCNN, con l'aggiunta di un filtro convoluzionale in più.

Il modello viene costruito utilizzando questi iperparametri e una versione modificata della funzione di build per accettare la nuova attivazione. Per il resto, la compilazione e l'addestramento sono analoghi a quelli di DnCNN. Dello studio sono state completate 34 trial su 40 totali, con i seguenti migliori iperparametri:

- trial = 29
- n_feats = 64
- activation = 'relu'
- alpha = None

Con un valore di loss di validazione pari a 0.0016350658843293786.

Il trial 34 aveva gli stessi iperparametri ma un valore di loss più alto(0.0026542791165411472), sempre a causa della natura stocastica dell'addestramento delle reti neurali.

5.6 Fine tuning con iperparametri ottimizzati

Visto che i risultati migliori sono stati ottenuti con il modello Ridnet, si è deciso di eseguire un ulteriore fine tuning solo per questom modello sul dataset Kvasir utilizzando gli iperparametri trovati dallo studio di ottimizzazione(5.5).

Setup addestramento con iperparametri ottimizzati

Gli iperparametri ottimizzati sono stati integrati nel codice di addestramento e fine tuning precedentemente sviluppato.

In particolare, essendo i pesi preaddestrati disponibili solo per la versione standard di Ridnet, è stato necessario eseguire nuovamente l'addestramento iniziale sui 5 dataset sintetici con i nuovi iperparametri prima di procedere al fine tuning sul dataset Kvasir.

Il setup di addestramento rimane invariato rispetto a quello descritto in precedenza, però è stato applicato un sistema di Distillazione tramite tecniche Teacher-Student per migliorare ulteriormente le prestazioni del modello Ridnet.

In questo approccio, un modello "insegnante" (teacher) più grande e complesso viene utilizzato per guidare l'addestramento di un modello "studente" (student) più piccolo e leggero.

Il modello insegnante fornisce "soft targets" (probabilità di output) che il modello studente cerca di imitare, oltre ai target originali.

Questo metodo aiuta il modello studente a generalizzare meglio, catturando informazioni più ricche dal modello insegnante.

In questo caso, il modello insegnante è la prima versione di Ridnet addestrata con gli iperparametri standard (feats = 64, reduction = 16), mentre il modello studente è la nuova versione di Ridnet con gli iperparametri ottimizzati (feats = 32, reduction = 8).

Listing 5.44: Definizione della classe Distiller

```

1 class Distiller(keras.Model):
2     def __init__(self, student, teacher, alpha=0.5):
3         super().__init__()
4         self.teacher = teacher
5         self.student = student
6         self.alpha = alpha
7         self.student_loss_fn = keras.losses.MeanSquaredError()
8         self.distillation_loss_fn = keras.losses.MeanSquaredError
           ↪ ()

```

La classe `Distiller` estende la classe base di Keras `Model` e implementa il metodo di addestramento personalizzato per la distillazione.

Nel costruttore, vengono passati il modello studente e insegnante, insieme a un parametro `alpha` che bilancia l'importanza tra la loss standard e la loss di distillazione.

Vengono definite due funzioni di perdita: una per la loss standard tra il modello studente e i target originali, e un'altra per la loss di distillazione tra il modello studente e le predizioni del modello insegnante.

Listing 5.45: Compile di Distiller

```

1     def compile(self, optimizer, loss, metrics):
2         super().compile(optimizer=optimizer, loss=loss, metrics=
           ↪ metrics)

```

Viene sovrascritto il metodo `compile` per accettare l'ottimizzatore, la funzione di perdita e le metriche da utilizzare durante l'addestramento.

Listing 5.46: Definizione della funzione per i cicli di train

```

1     def train_step(self, data):
2         x, y_true = data # x = immagine rumorosa, y_true =
           ↪ immagine pulita
3
4         with tf.GradientTape() as tape:
5             # Predizioni student e teacher
6             y_student = self.student(x, training=True)
7             y_teacher = self.teacher(x, training=False)

```

La funzione `train_step` implementa il ciclo di addestramento personalizzato.

Riceve un batch di dati composto da immagini rumorose (input) e immagini pulite (target).

Utilizza un `GradientTape` per registrare le operazioni di calcolo delle gradienti.

All'interno del tape, vengono calcolate le predizioni del modello studente e del modello insegnante.

Listing 5.47: Calcolo perdite modelli e bilanciamento

```

1      # Loss standard (student vs ground-truth)
2      student_loss = self.student_loss_fn(y_true, y_student
      ↪ )
3
4      # Distillation loss (student vs teacher)
5      distill_loss = self.distillation_loss_fn(y_teacher,
      ↪ y_student)
6
7      # Combina con alpha
8      loss = self.alpha * student_loss + (1 - self.alpha) *
      ↪ distill_loss

```

Dopo aver ottenuto le predizioni, vengono calcolate due perdite: la perdita standard tra le predizioni del modello studente e i target originali, e la perdita di distillazione tra le predizioni del modello studente e quelle del modello insegnante.

Queste due perdite vengono combinate utilizzando il parametro `alpha` per bilanciare la loro importanza.

Listing 5.48: Calcolo gradiente Distiller

```

1      # Backprop
2      grads = tape.gradient(loss, self.student.
      ↪ trainable_variables)
3      self.optimizer.apply_gradients(zip(grads, self.student.
      ↪ trainable_variables))

```

Viene calcolato il gradiente della perdita combinata rispetto ai pesi del modello studente, e questi gradienti vengono applicati per aggiornare i pesi del modello studente.

Listing 5.49: Aggiornamento delle metriche dello student

```

1      # Update metrics
2      for metric in self.metrics:
3          metric.update_state(y_true, y_student)
4      return {m.name: m.result() for m in self.metrics} | {"
      ↪ loss": loss}

```

Vengono aggiornate le metriche dello student in base alle predizioni attuali.

Infine, viene restituito un dizionario con i valori aggiornati delle metriche e la perdita totale.

Listing 5.50: Definizione della funzione di validazione validation/test

```

1  def test_step(self, data):
2      x, y_true = data
3      y_student = self.student(x, training=False)
4      loss = self.student_loss_fn(y_true, y_student)
5      # Update metrics
6      for metric in self.metrics:
7          metric.update_state(y_true, y_student)
8      return {m.name: m.result() for m in self.metrics} | {"
          ↪ loss": loss}

```

La funzione `test_step` implementa il ciclo di validazione/test personalizzato.

Riceve un batch di dati composto da immagini rumorose (input) e immagini pulite (target).

Calcola le predizioni del modello studente in modalità di valutazione (`training=False`) e la perdita standard tra le predizioni del modello studente e i target originali.

Per poi aggiornare le metriche dello student in base alle predizioni attuali.

Infine, viene restituito un dizionario con i valori aggiornati delle metriche e la perdita totale.

Listing 5.51: Definizione della callback per Distiller

```

1  save_student_callback = SaveStudentCallback()
2  # [...] stesse callback di prima [...]

```

Viene definito un callback personalizzato che include tutta la classe `Distiller` in modo che possa essere usata per salvare il modello studente al termine dell'addestramento.

Vengono mantenute le stesse callback del training originale.

Listing 5.52: Definizione della callback per Distiller

```

1  distiller = Distiller(student=student, teacher=teacher, alpha
          ↪ =0.7)
2  distiller.compile(
3      optimizer=keras.optimizers.Adam(),
4      loss = "mse",
5      metrics=["accuracy", keras.metrics.CategoricalAccuracy()]
6  )
7  distiller.fit(train_ds_noisy,
8              epochs = 23,
9              validation_data=val_ds_noisy,
10             callbacks=callbacks
11 )

```

La classe `Distiller` viene istanziata con il modello studente, il modello insegnante e un valore di `alpha` di 0.7, dando più peso alla loss standard rispetto alla loss di distillazione.

Il distiller viene compilato con l'ottimizzatore Adam, la loss MSE e le metriche di accuratezza e `CategoricalAccuracy`.

Infine, il distiller viene addestrato sul dataset di training rumoroso per 23 epoche, con i dati di validazione e le callback definite in precedenza.

Risultati addestramento iniziale con iperparametri ottimizzati

Qui di seguito sono riportati i risultati ottenuti dall'addestramento iniziale della Ridnet Student con gli iperparametri ottimizzati sugli stessi dataset usati in precedenza.

Dataset	Rumore	Max	Min	Avg
Renoir	$\sigma = 10.49$	48.60	25.55	35.85
SIDD	$\sigma = 7.22$	41.29	31.47	37.77
BSD500	$\sigma = 15$	32.09	27.71	30.14
	$\sigma = 25$	29.50	24.80	27.37
	$\sigma = 50$	26.94	21.73	24.34
FiveK	$\sigma = 15$	35.44	20.54	30.16
	$\sigma = 25$	33.63	18.61	27.89
	$\sigma = 50$	31.74	17.20	25.44
Div2K	$\sigma = 15$	34.37	23.86	29.92
	$\sigma = 25$	32.03	21.70	27.19
	$\sigma = 50$	29.46	19.23	24.14

Tabella 5.4: Nuovi risultati del training iniziale della Ridnet con iperparametri ottimizzati.

La tabella riporta i valori di PSNR ottenuti dal modello Ridnet Student con gli iperparametri ottimizzati sui 5 dataset sintetici utilizzati per l'addestramento iniziale. Per ciascun dataset e livello di rumore sono riportati i valori massimo, minimo e medio di PSNR.

I risultati mostrano che il modello Ridnet Student con iperparametri ottimizzati mantiene prestazioni competitive rispetto alla versione standard di Ridnet, con valori di PSNR medi leggermente inferiori ma ancora elevati.

In particolare, il valore medio di PSNR rimane sopra 30 per i livelli di rumore più bassi ($\sigma=10.49, 7.22, 15$) e sopra 25 anche per il livello di rumore più alto ($\sigma=50$).

Questi risultati indicano che l'ottimizzazione degli iperparametri ha permesso di ottenere un modello più leggero e veloce, senza compromettere significativamente la qualità del denoising. I risultati visivi dell'addestramento iniziale con i nuovi iperparametri sono riportati nell'appendice sezione C.1.

Risultati fine tuning con iperparametri ottimizzati

Una volta completato l'addestramento iniziale, si è proceduto con il fine tuning sul dataset Kvasir, con le stesse impostazioni del fine tuning precedente ma con i nuovi iperparametri. Qui di seguito sono riportati i risultati ottenuti prima e dopo il fine tuning sul dataset Kvasir con i nuovi iperparametri ottimizzati.

Dataset	Rumore	Max	Min	Avg
Kvasir	$\sigma = 15$	33.62	29.66	31.77
pre	$\sigma = 25$	31.34	27.41	29.26
fine tune	$\sigma = 50$	28.79	23.91	26.57
Kvasir	$\sigma = 15$	35.38	30.61	33.06
post	$\sigma = 25$	33.32	28.06	30.71
fine tune	$\sigma = 50$	30.01	24.50	27.46

Tabella 5.5: Nuovi risultati del fine tuning della Ridnet con iperparametri ottimizzati.

Come si può osservare, il modello Ridnet Student con iperparametri ottimizzati mostra un miglioramento significativo delle prestazioni in termini di PSNR dopo il fine tuning sul dataset Kvasir.

I valori medi di PSNR aumentano in modo consistente per tutti i livelli di rumore ($\sigma=15, 25, 50$) dopo il fine tuning, confermando l'efficacia dell'adattamento specifico al dataset.

In particolare, il valore medio di PSNR raggiunge 33.06 a $\sigma=15$, 30.71 a $\sigma=25$ e 27.46 a $\sigma=50$, che sono miglioramenti notevoli rispetto ai valori pre fine tuning.

Questi risultati indicano che l'ottimizzazione degli iperparametri, combinata con il fine tuning, ha permesso di ottenere un modello più performante e adattato alle caratteristiche specifiche del dataset Kvasir.

I risultati visivi del fine tuning con i nuovi iperparametri sono riportati nell'appendice sezione C.2.

5.7 Conclusioni

In questa tesi si è esplorato il problema del denoising delle immagini. Sono stati studiati in particolare tre denoiser basati su reti neurali presenti in letteratura: Ridnet, DnCNN e Autoencoder.

Sono stati confrontati con il metodo classico BM3D, uno dei baseline più solidi nel campo del denoising.

Ciascuna di queste architetture è stata allenata con attenzione all'ottimizzazione degli iperparametri realizzata con Optuna.

Ridnet mostra generalmente i valori medi e massimi di PSNR più elevati su quasi tutti i dataset e livelli di rumore, indicando una capacità superiore di preservare la qualità dell'immagine denoised.

Appendice A

Confronto visivo tra modelli

In questa appendice sono riportati i risultati qualitativi ottenuti sui diversi dataset con i tre modelli di denoising addestrati: **Ridnet**, **DnCNN** e **Autoencoder**, e **BM3D**. Per ogni dataset e livello di rumore si confrontano le uscite dei modelli.

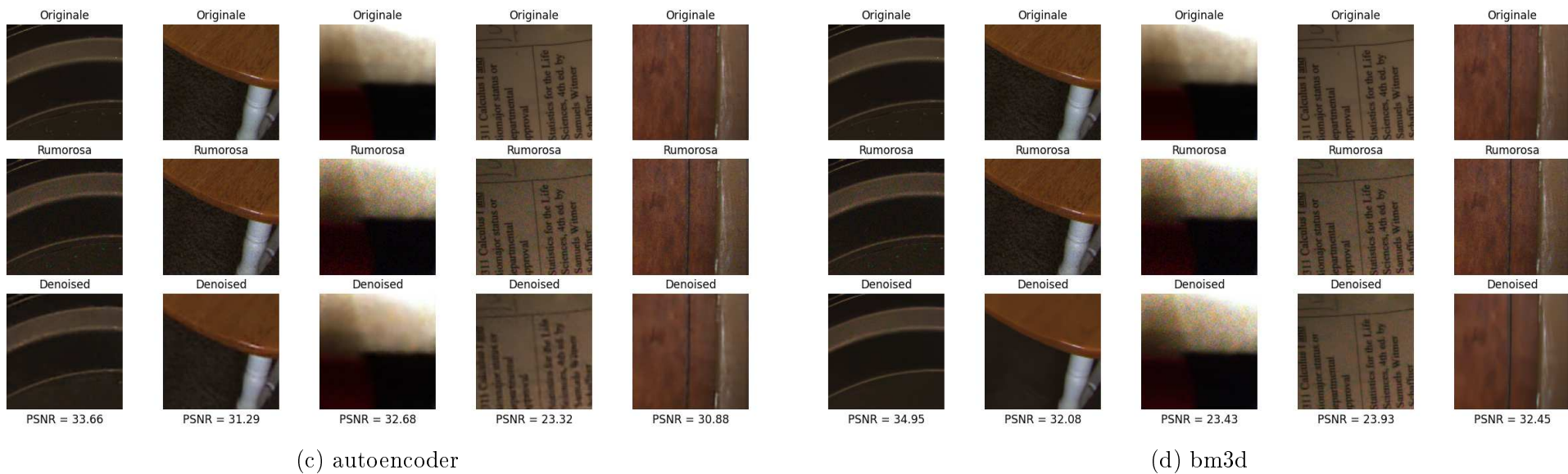
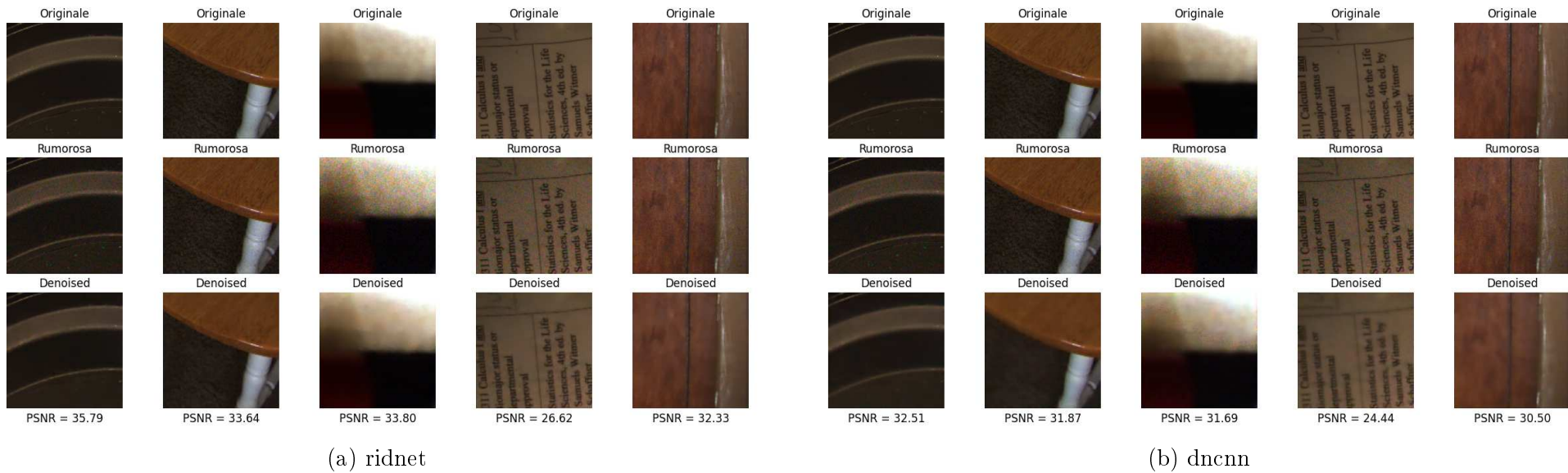
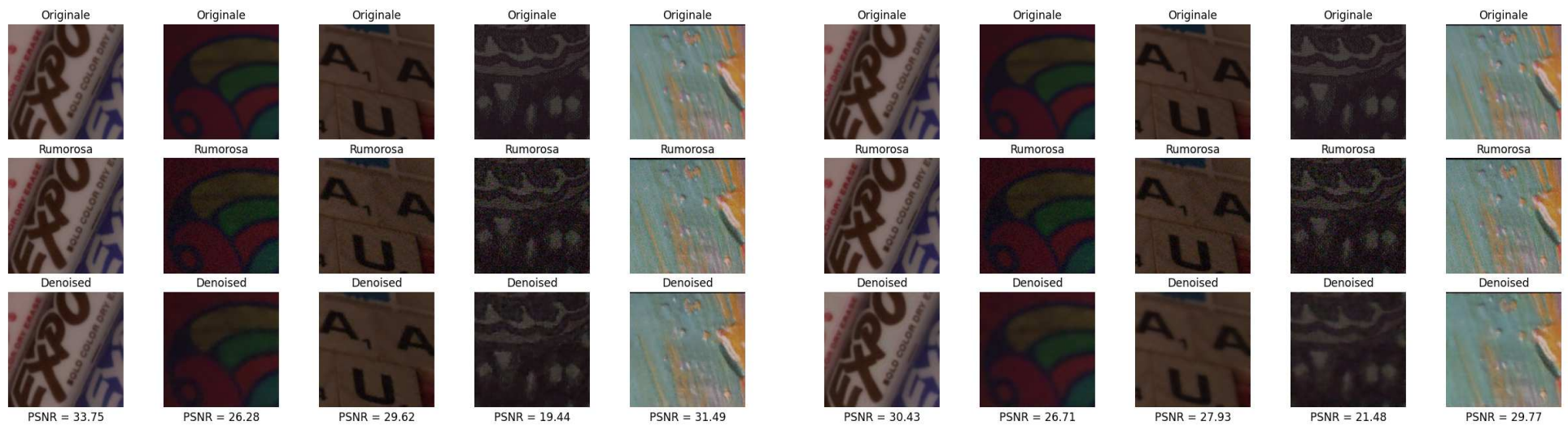
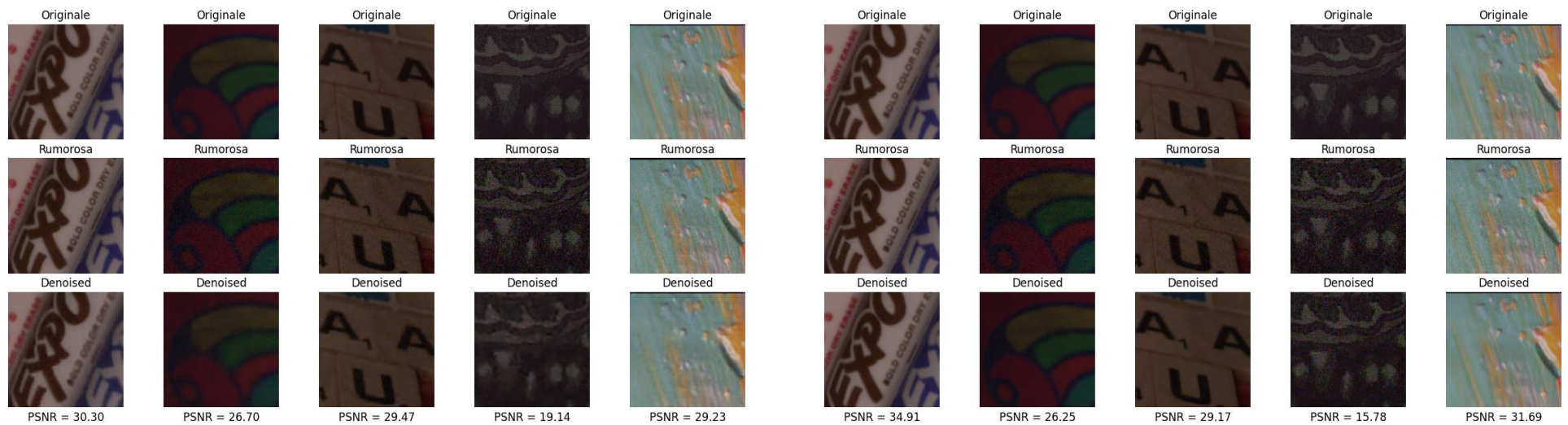


Figura A.1: Dataset renoir



(a) ridnet

(b) dncnn



(c) autoencoder

(d) bm3d

Figura A.2: Dataset sidd

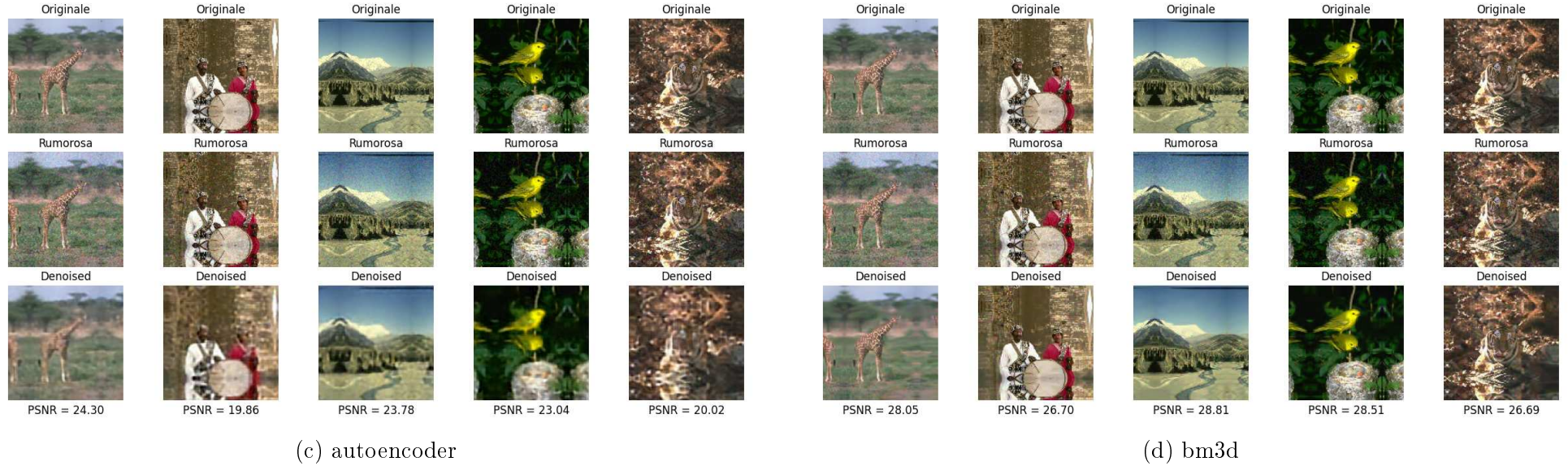
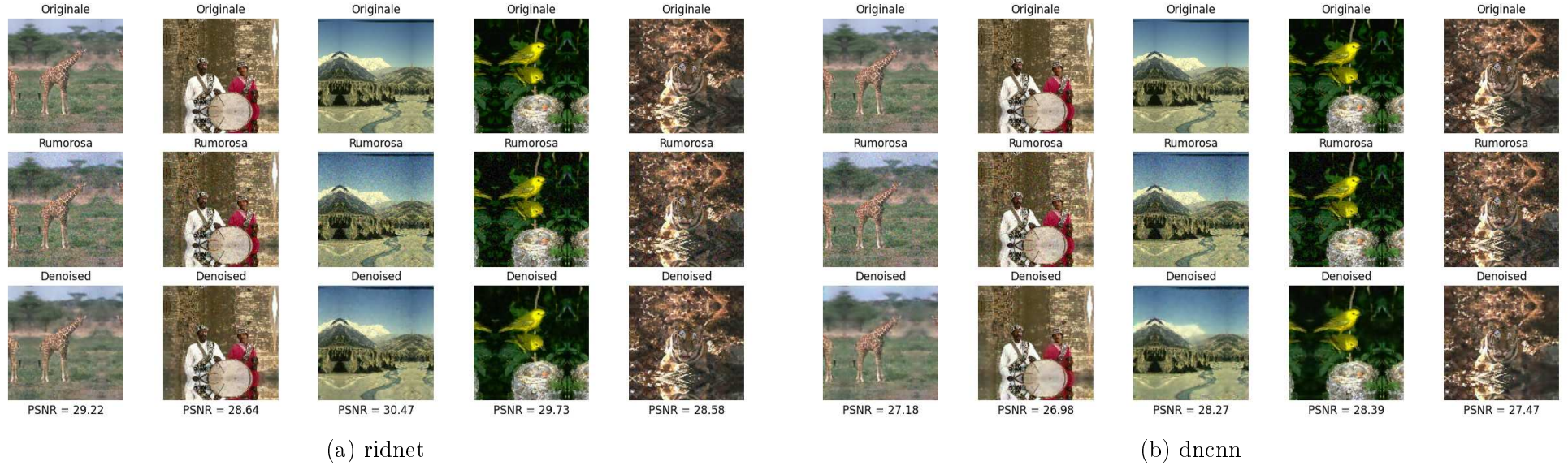
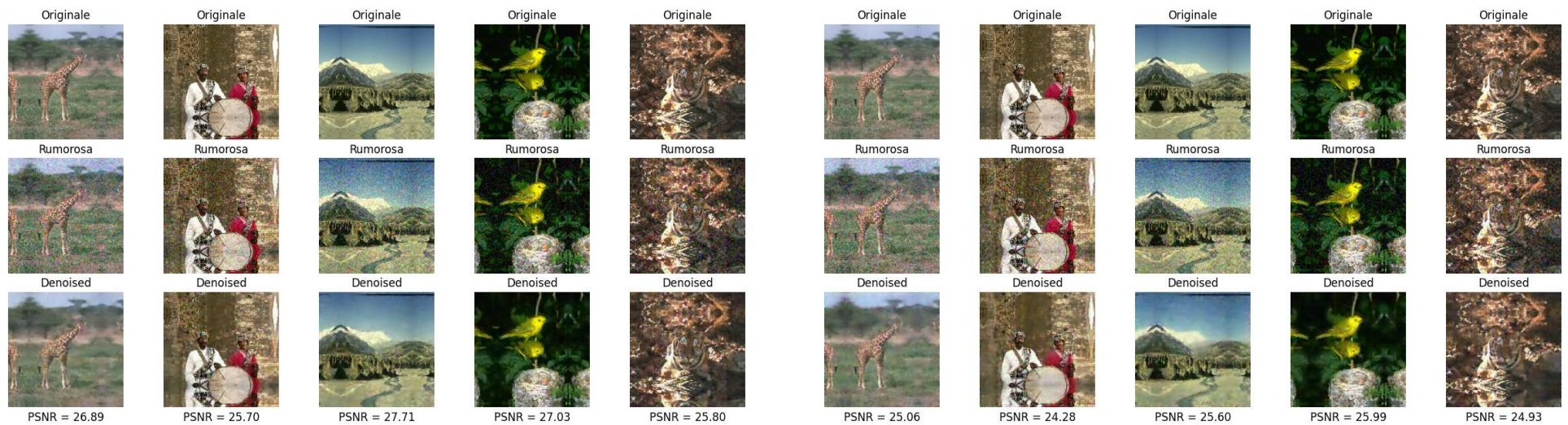
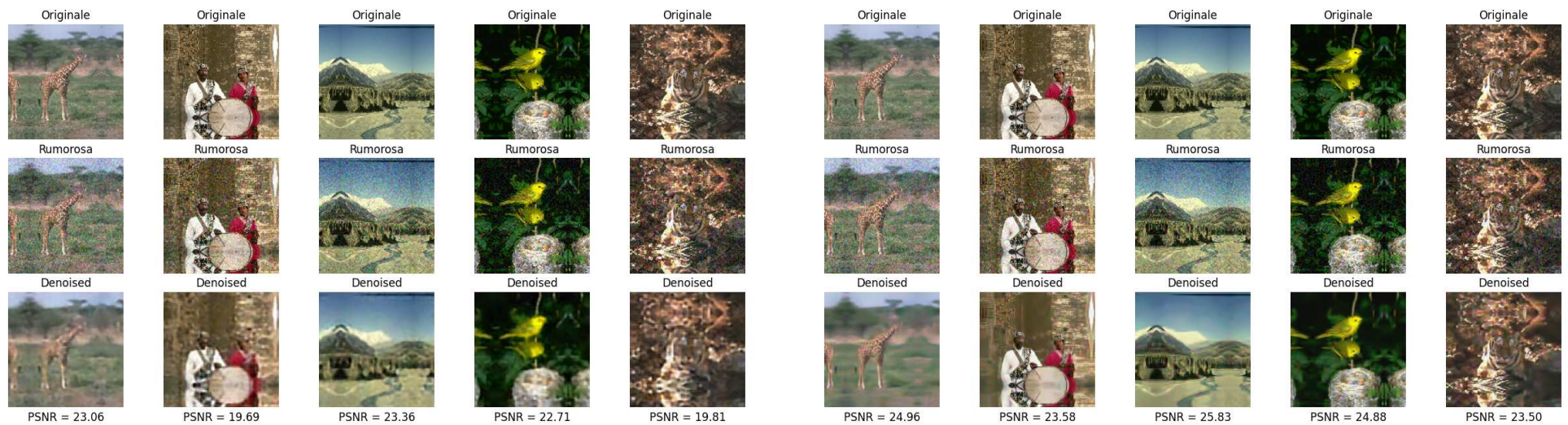


Figura A.3: Dataset bsd- Livello di rumore $\sigma = 15$.



(a) ridnet

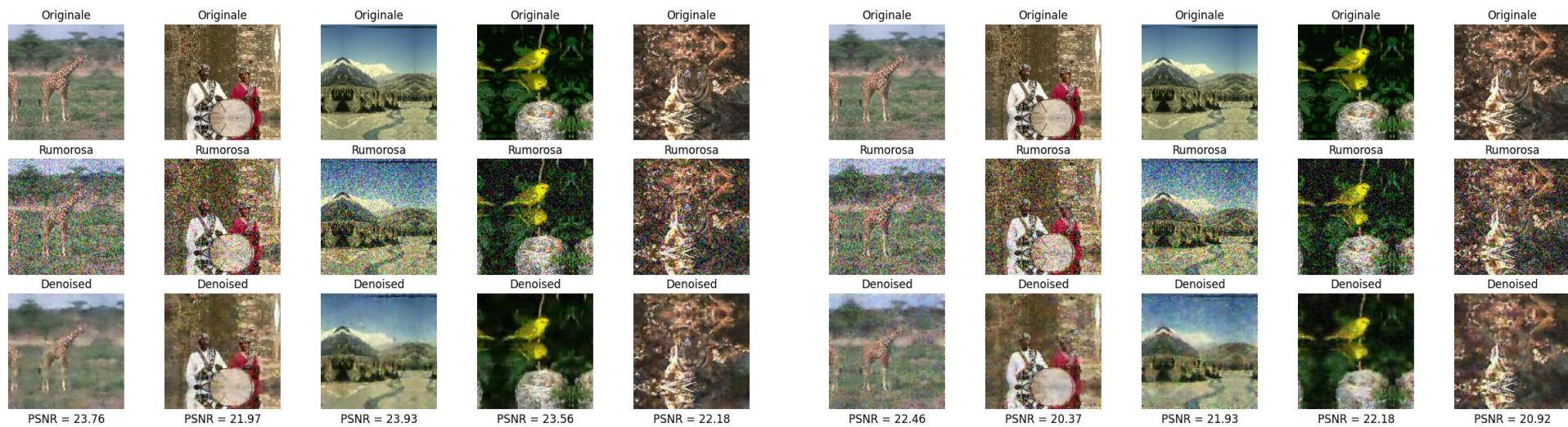
(b) dncnn



(c) autoencoder

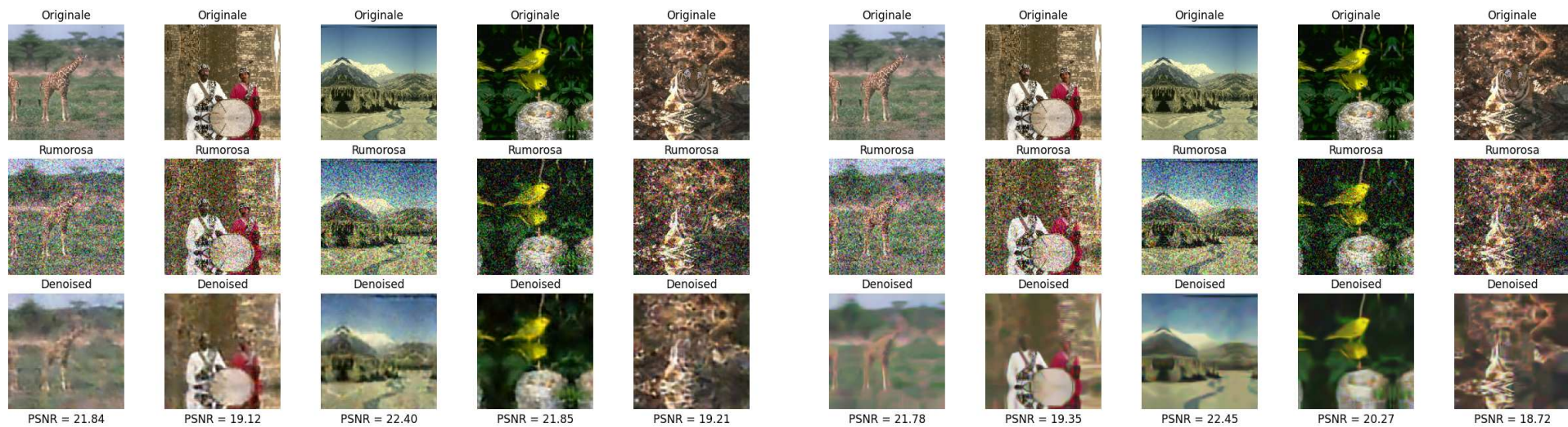
(d) bm3d

Figura A.4: Dataset bsd- Livello di rumore $\sigma = 25$.



(a) ridnet

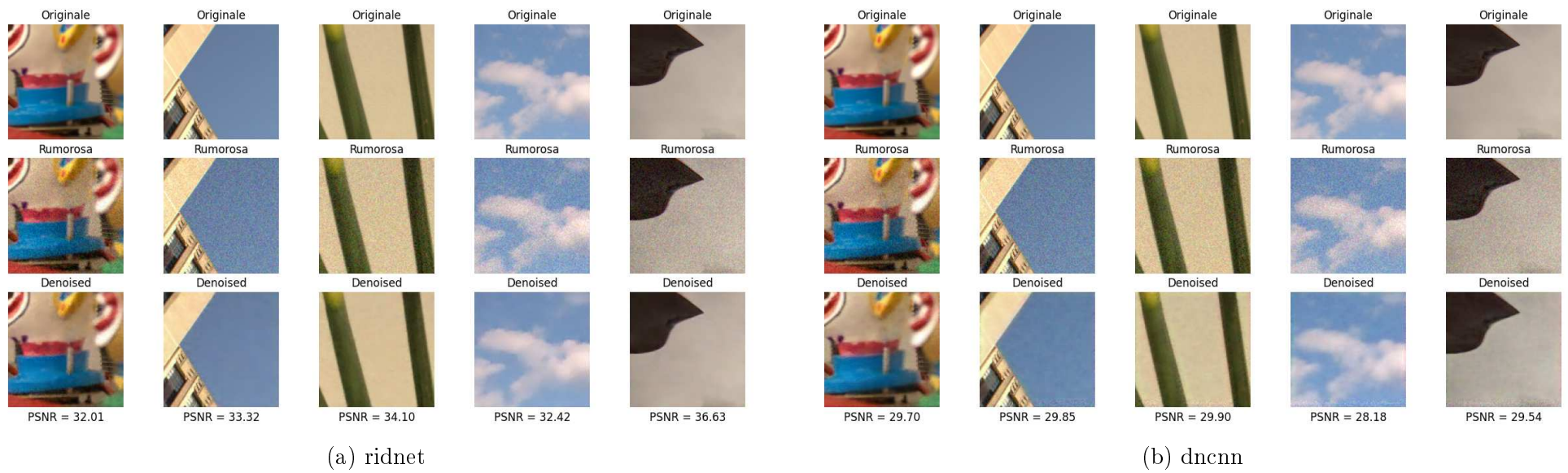
(b) dncnn



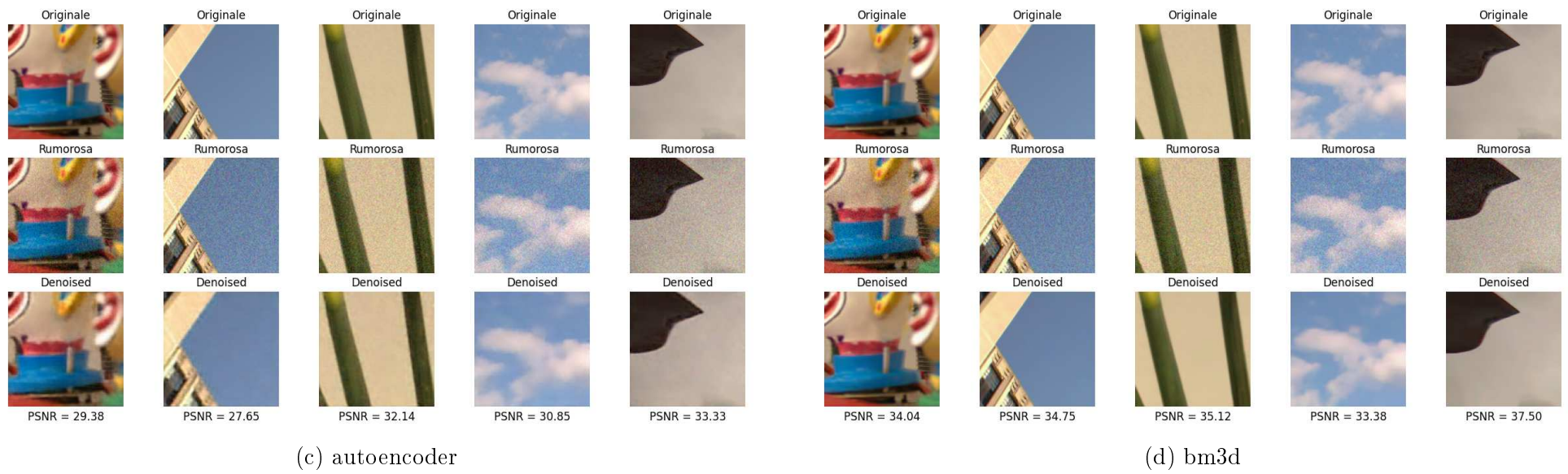
(c) autoencoder

(d) bm3d

Figura A.5: Dataset bsd- Livello di rumore $\sigma = 50$.



(b) dncnn



(d) bm3d

Figura A.6: Dataset fivek– Livello di rumore $\sigma = 15$.

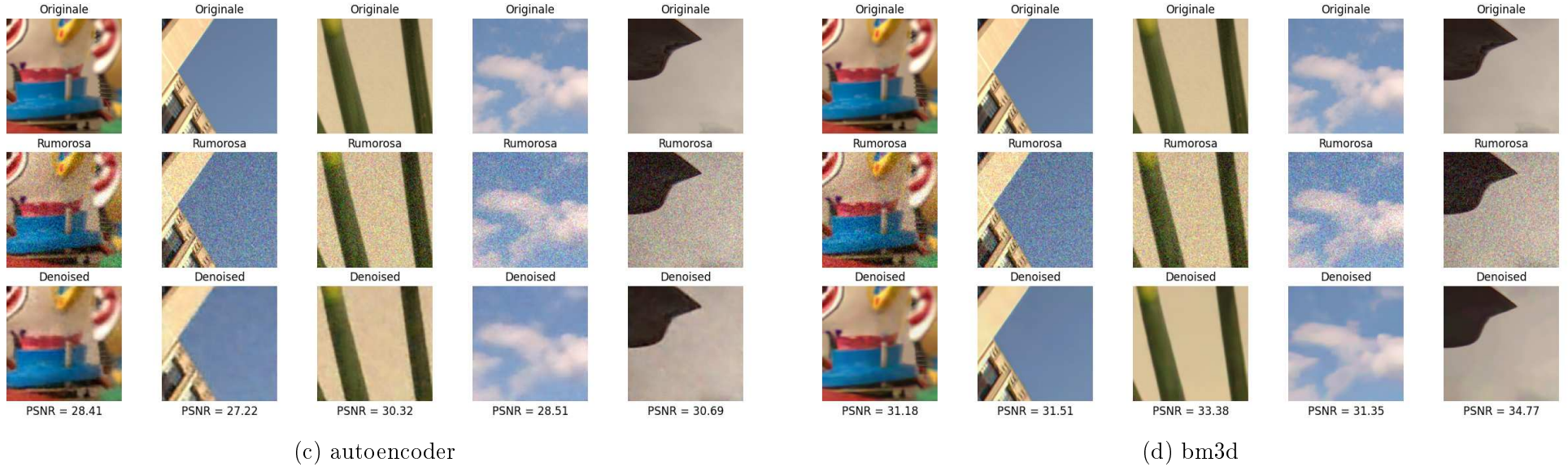
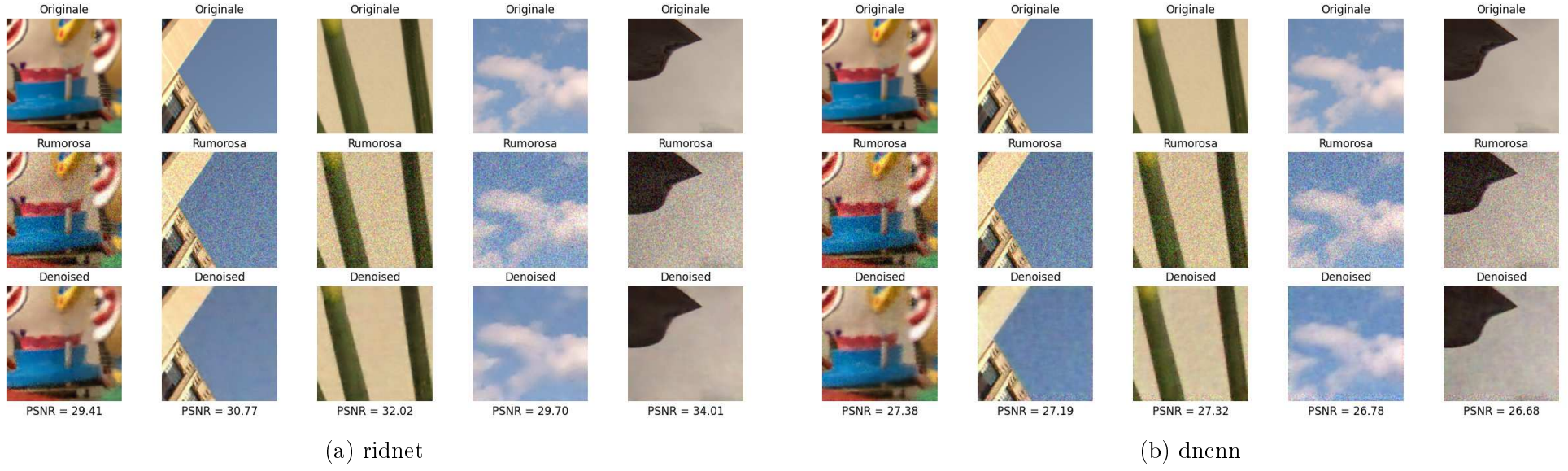


Figura A.7: Dataset fivek– Livello di rumore $\sigma = 25$.

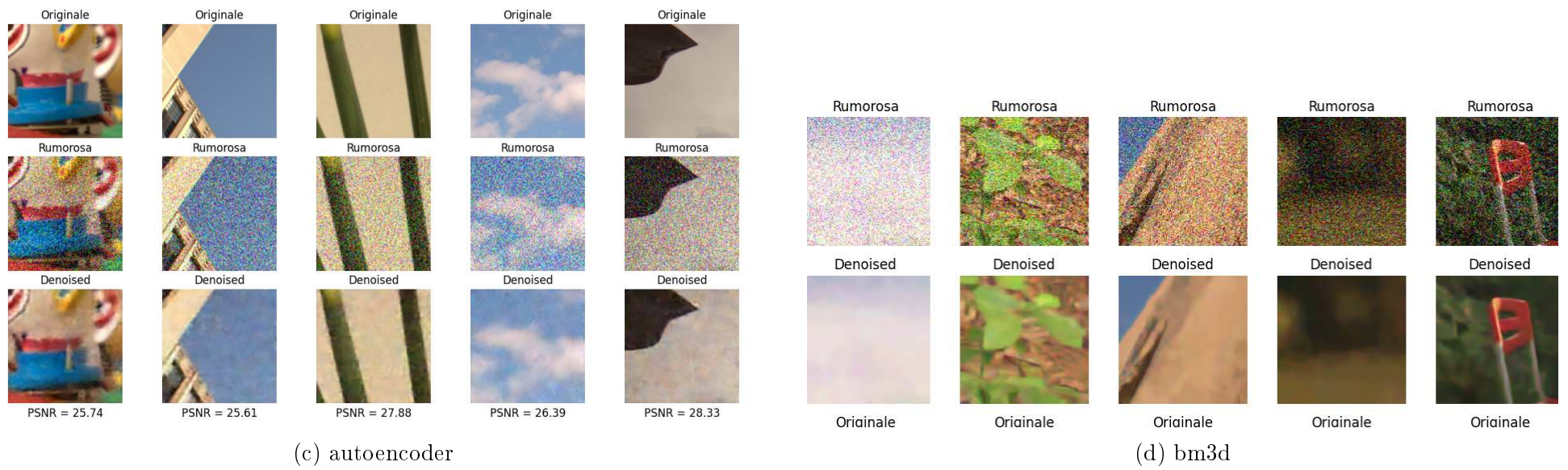
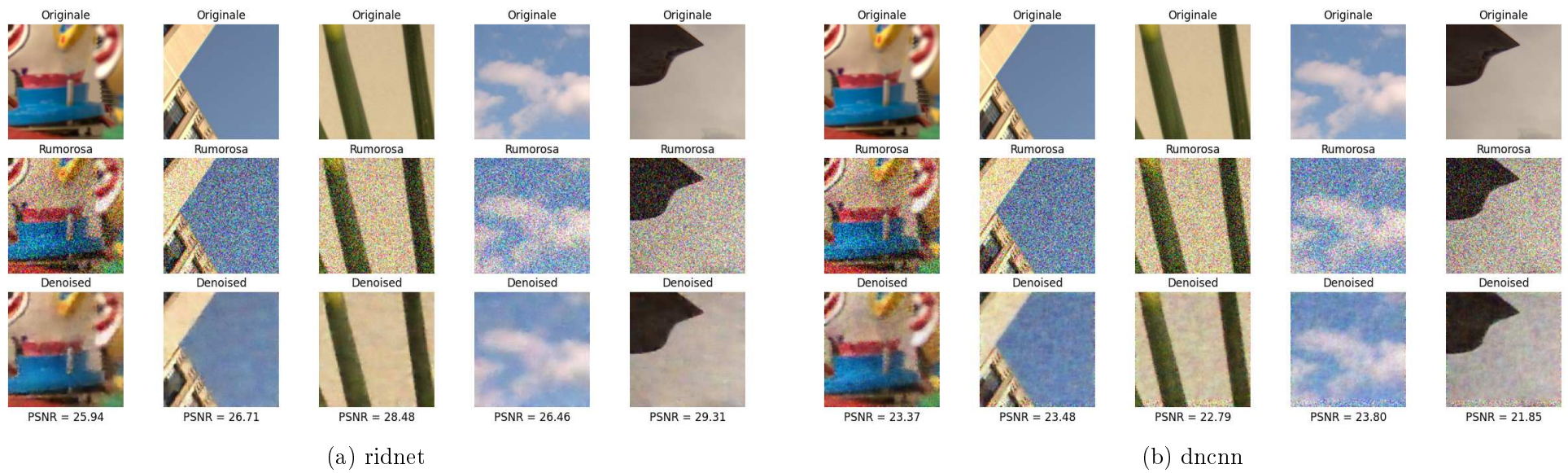
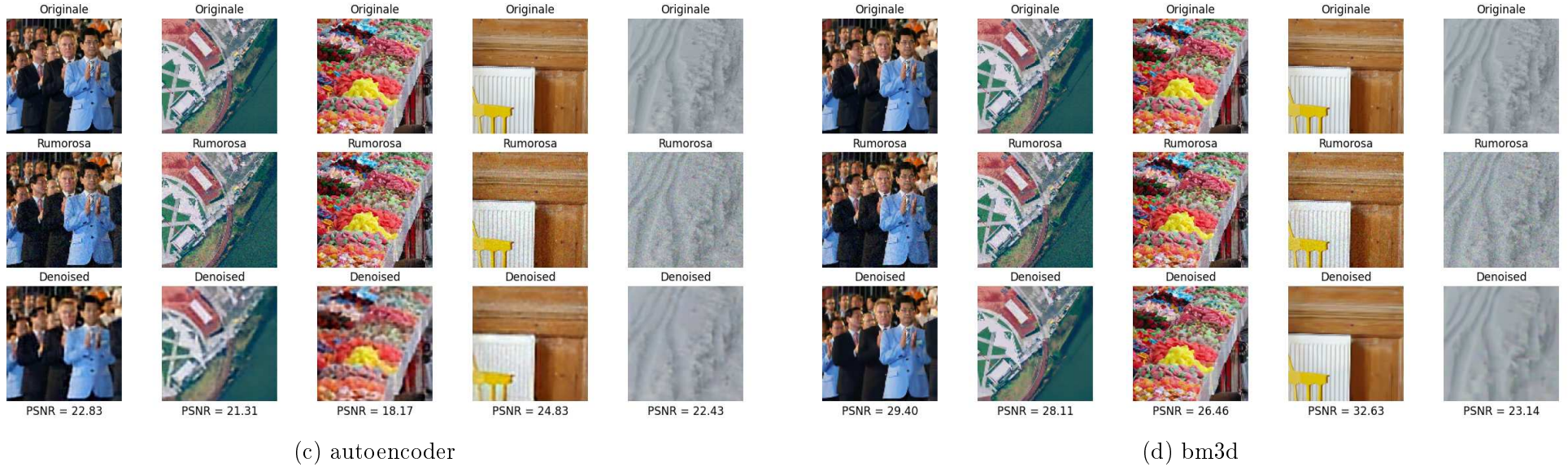
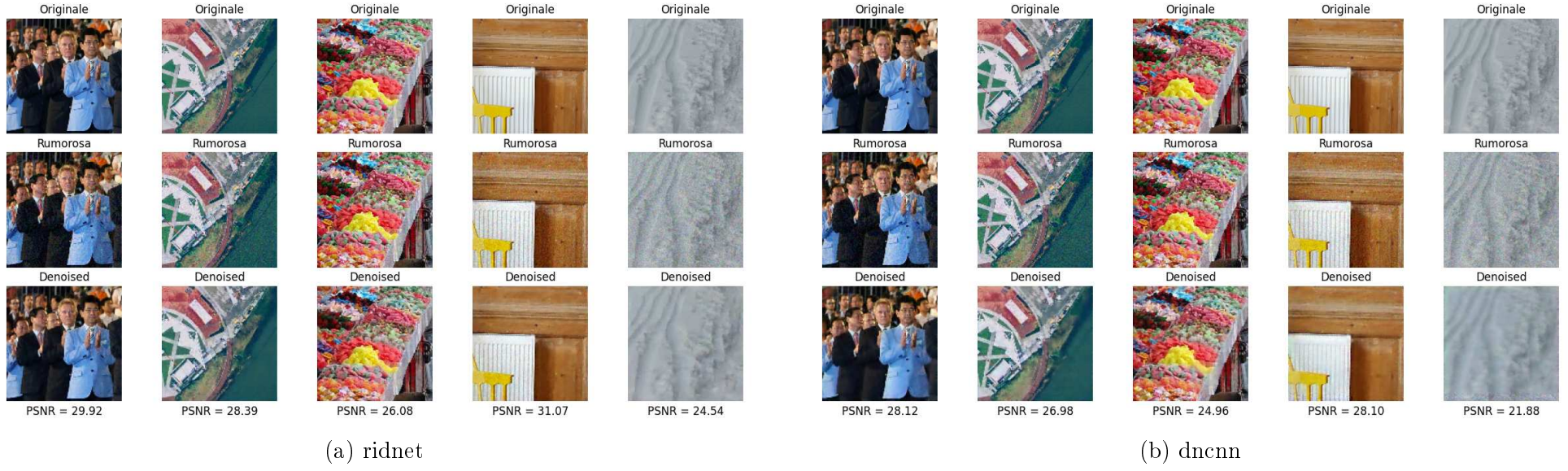


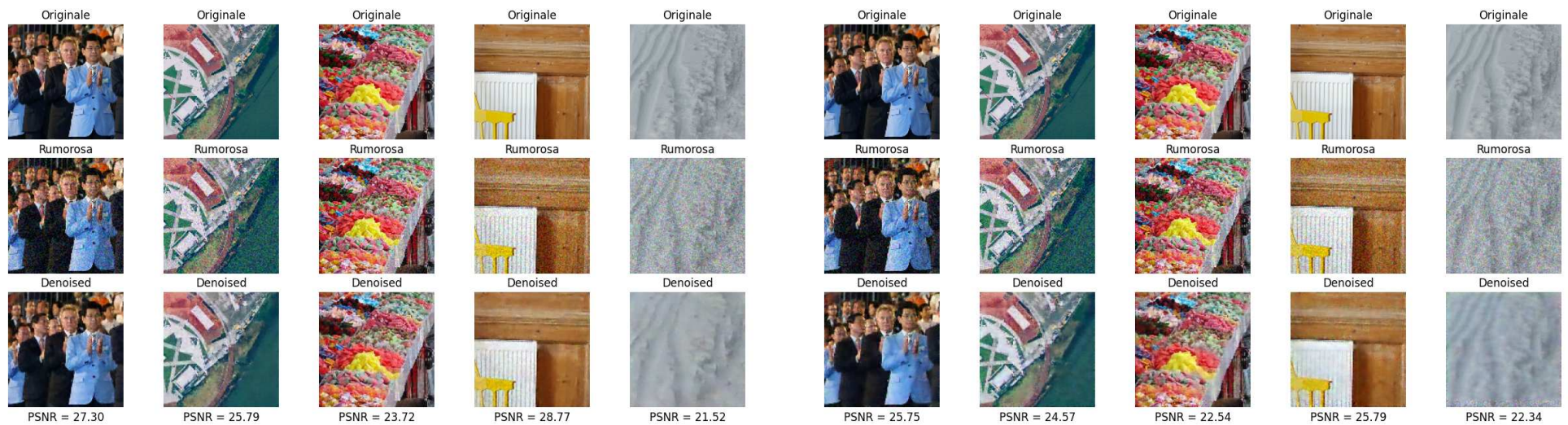
Figura A.8: Dataset fivek– Livello di rumore $\sigma = 50$.



(b) dncnn

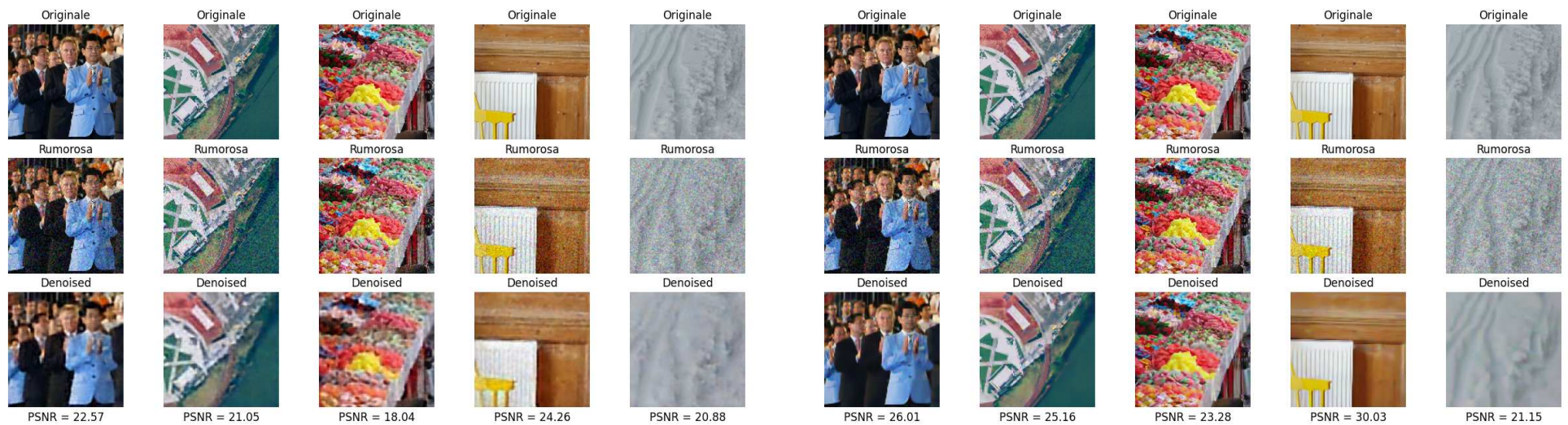
(d) bm3d

Figura A.9: Dataset div2k– Livello di rumore $\sigma = 15$.



(a) ridnet

(b) dncnn



(c) autoencoder

(d) bm3d

Figura A.10: Dataset div2k– Livello di rumore $\sigma = 25$.

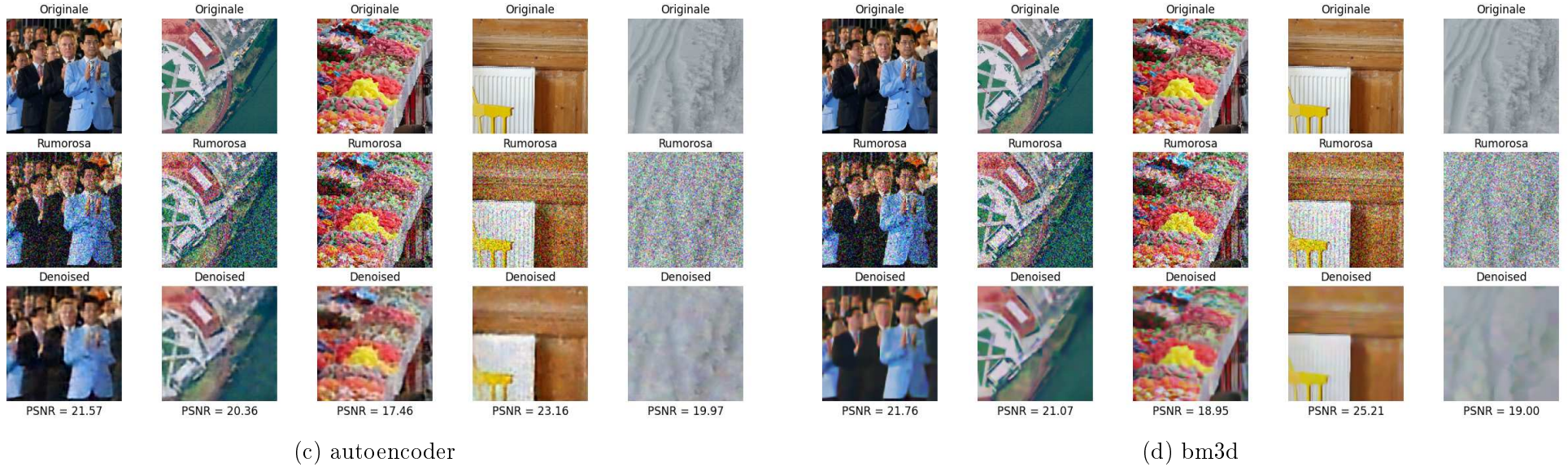
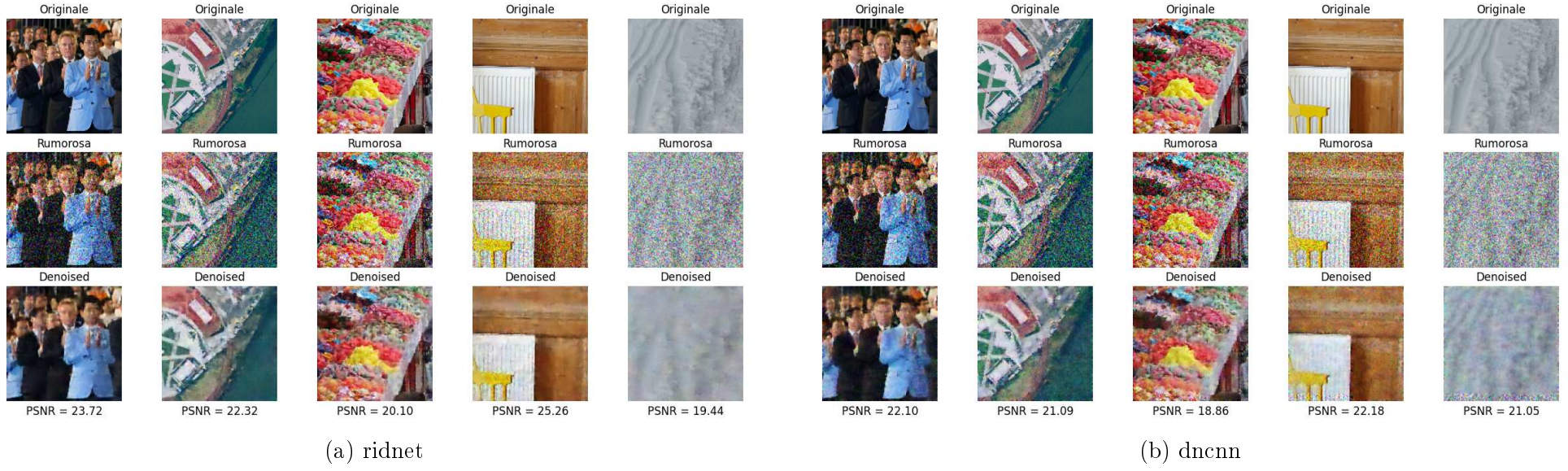


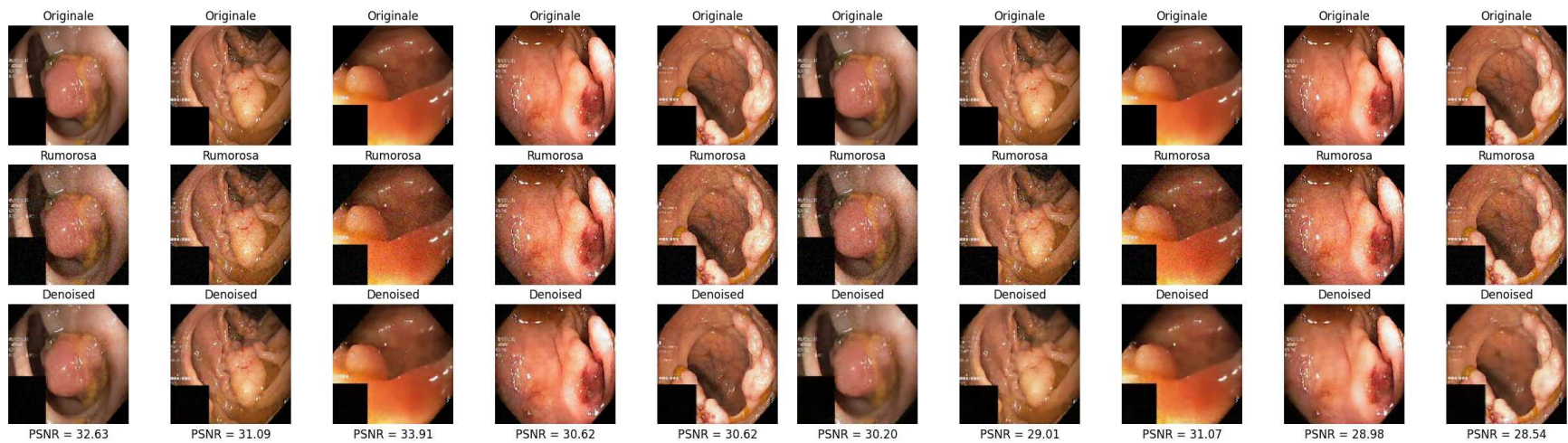
Figura A.11: Dataset div2k– Livello di rumore $\sigma = 50$.

Appendice B

Risultati Fine Tuning

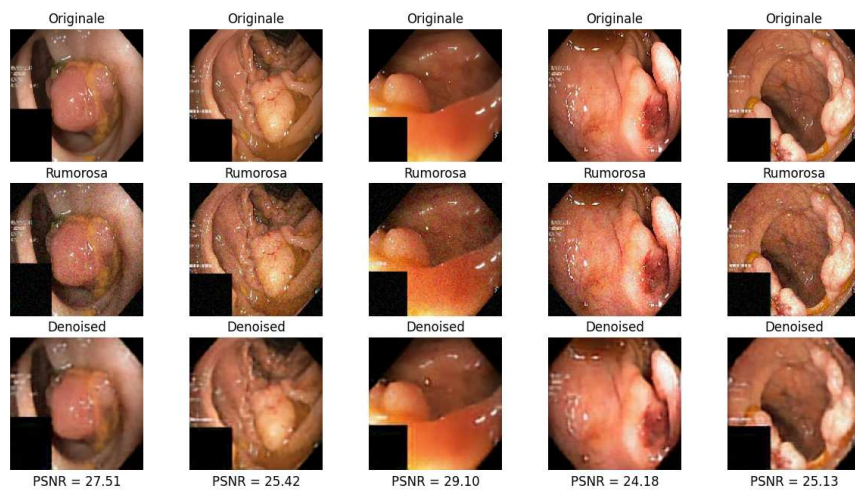
Qui di seguito sono riportati i risultati visivi pre e post fine tuning sul dataset Kvasir, per i tre modelli addestrati: **Ridnet**, **DnCNN** e **Autoencoder**, e **BM3D**.

B.1 Pre Fine Tuning



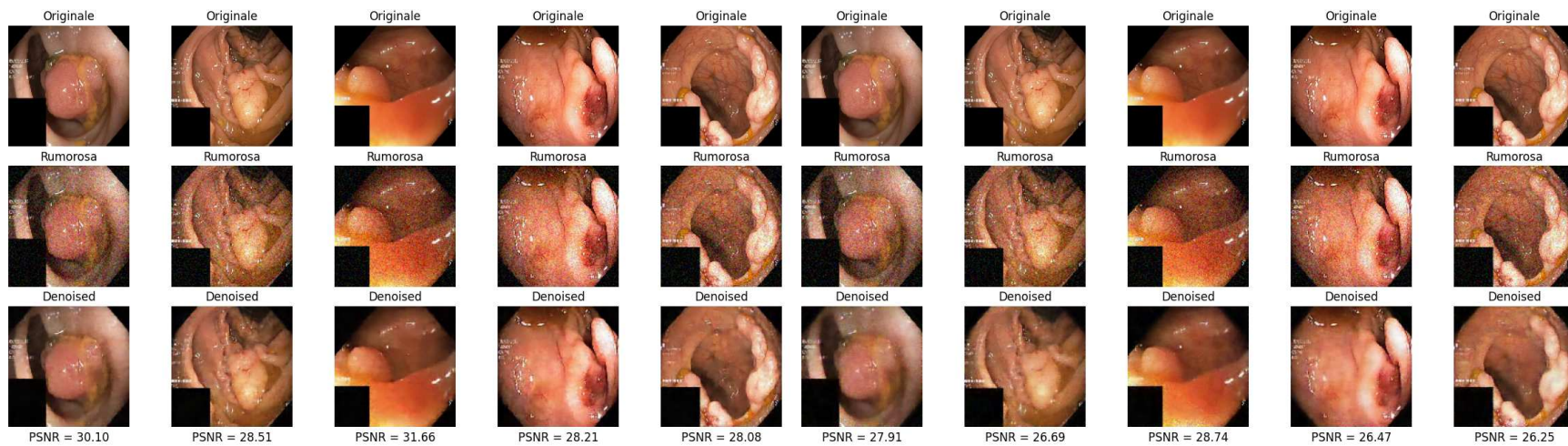
(a) ridnet

(b) dncnn



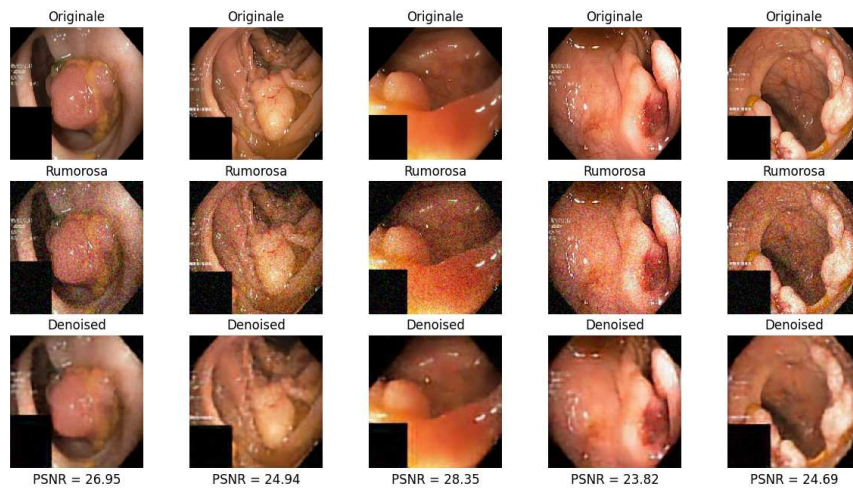
(c) autoencoder

Figura B.1: Kvasir – Livello di rumore $\sigma = 15$.



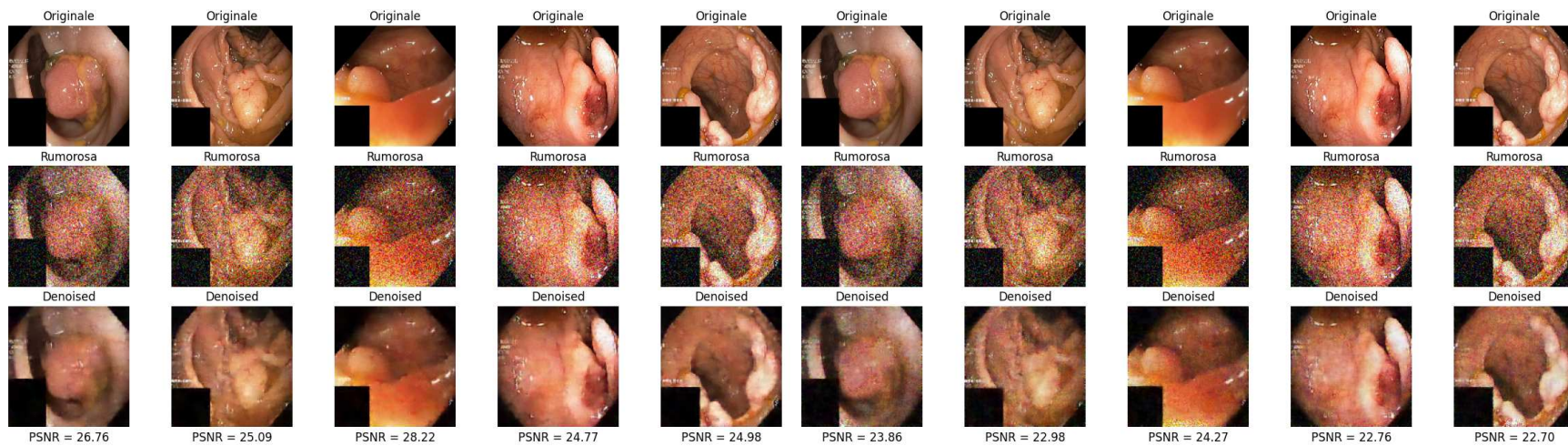
(a) ridnet

(b) dncnn



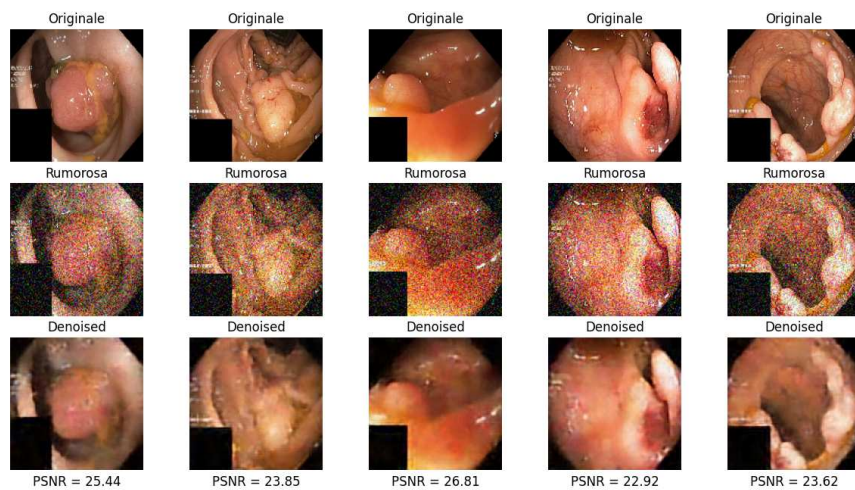
(c) autoencoder

Figura B.2: Kvasir – Livello di rumore $\sigma = 25$.



(a) ridnet

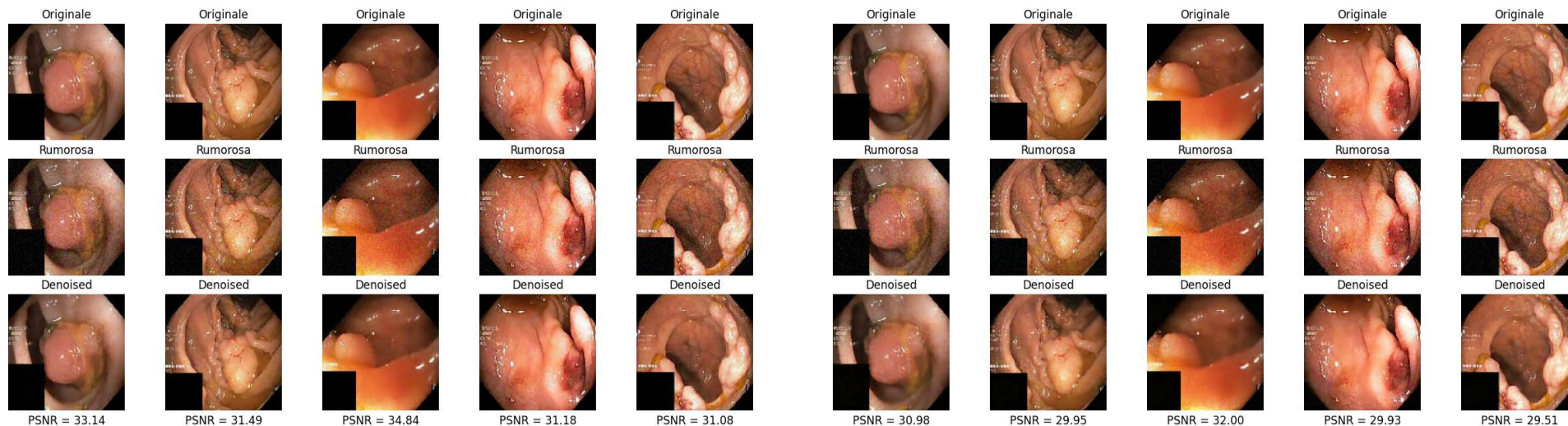
(b) dncnn



(c) autoencoder

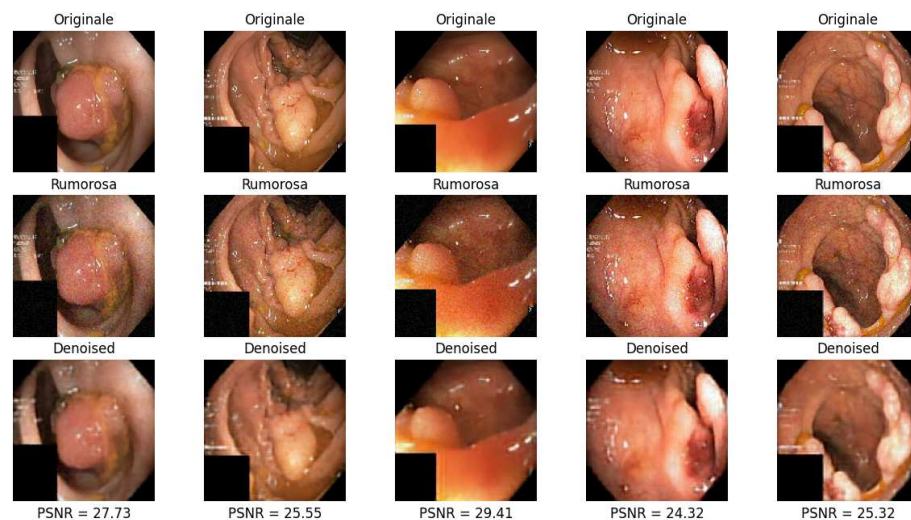
Figura B.3: Kvasir – Livello di rumore $\sigma = 50$.

B.2 Post Fine Tuning



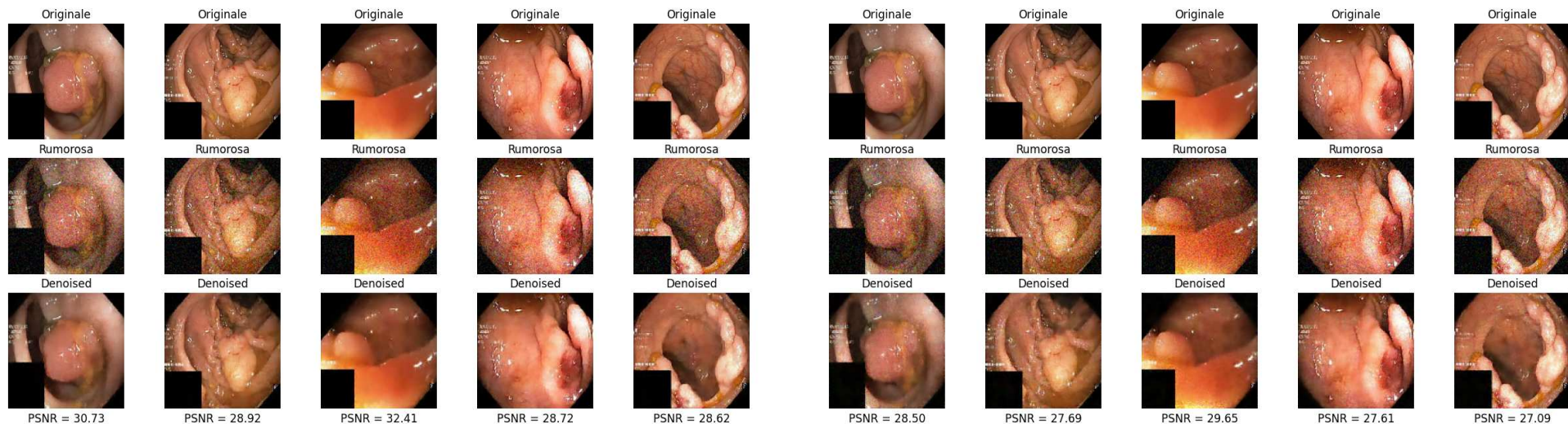
(a) ridnet

(b) dncnn



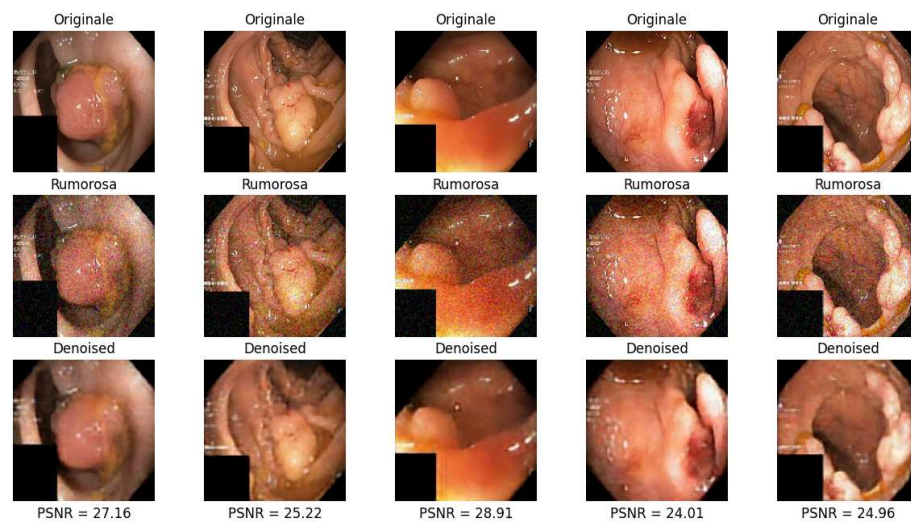
(c) autoencoder

Figura B.4: Kvasir – Livello di rumore $\sigma = 15$.



(a) *ridnet*

(b) *dncnn*



(c) autoencoder

Figura B.5: Kvasir – Livello di rumore $\sigma = 25$.

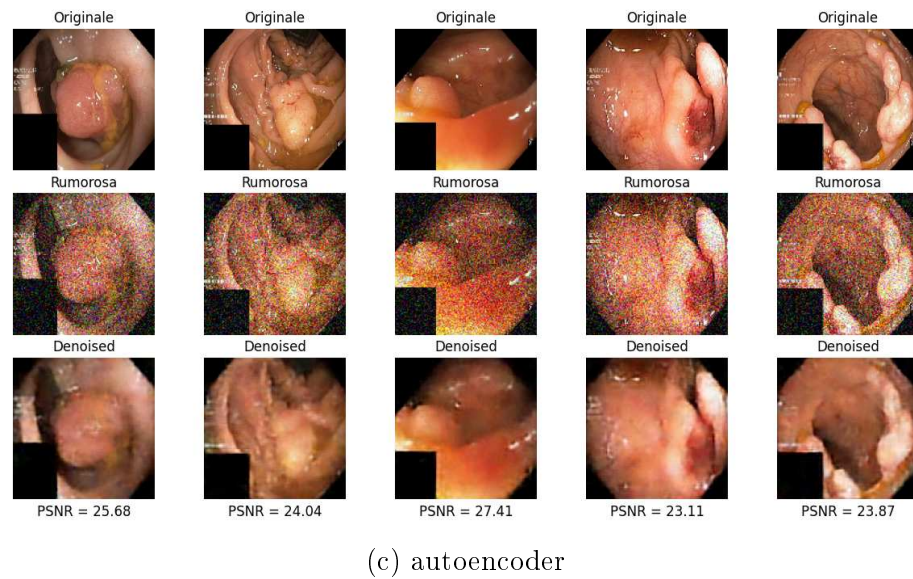
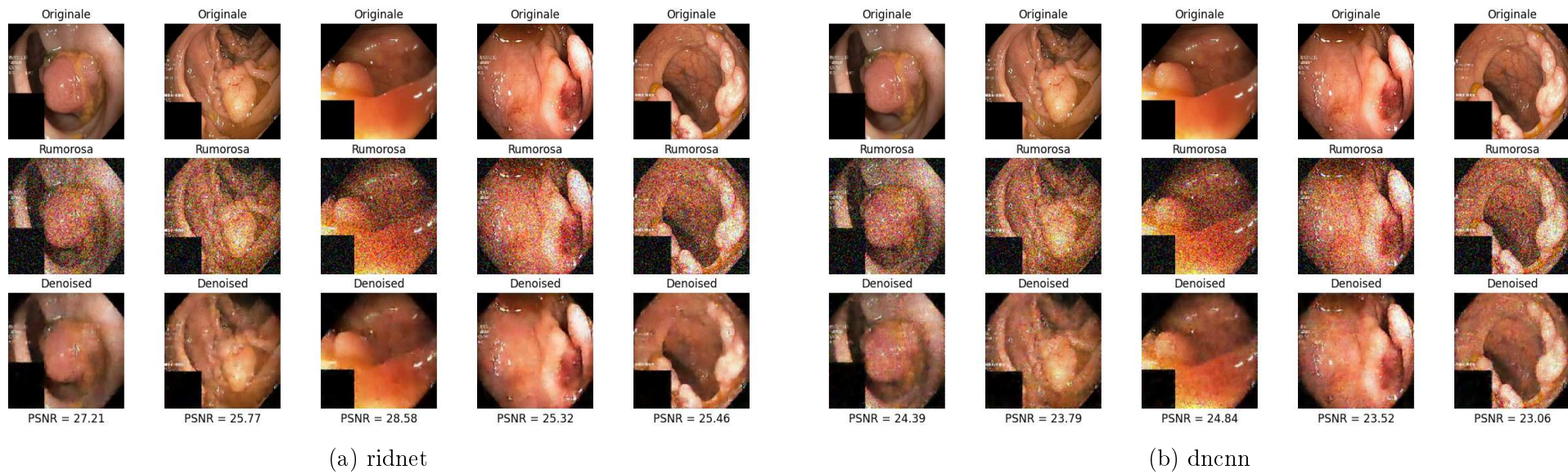
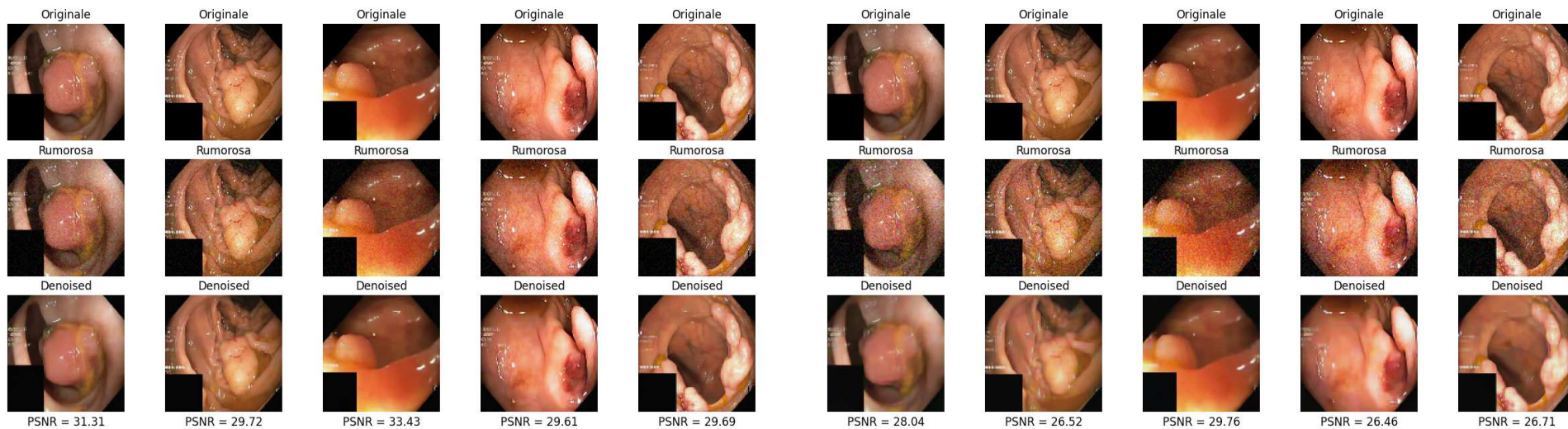


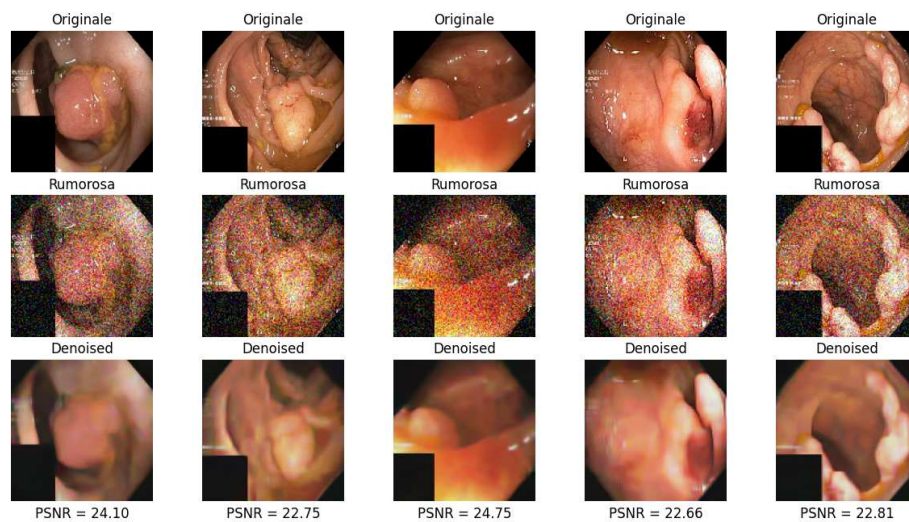
Figura B.6: Kvasir – Livello di rumore $\sigma = 50$.

BM3D



(a) BM3D - $\sigma = 15$

(b) BM3D - $\sigma = 25$



(c) BM3D - $\sigma = 50$

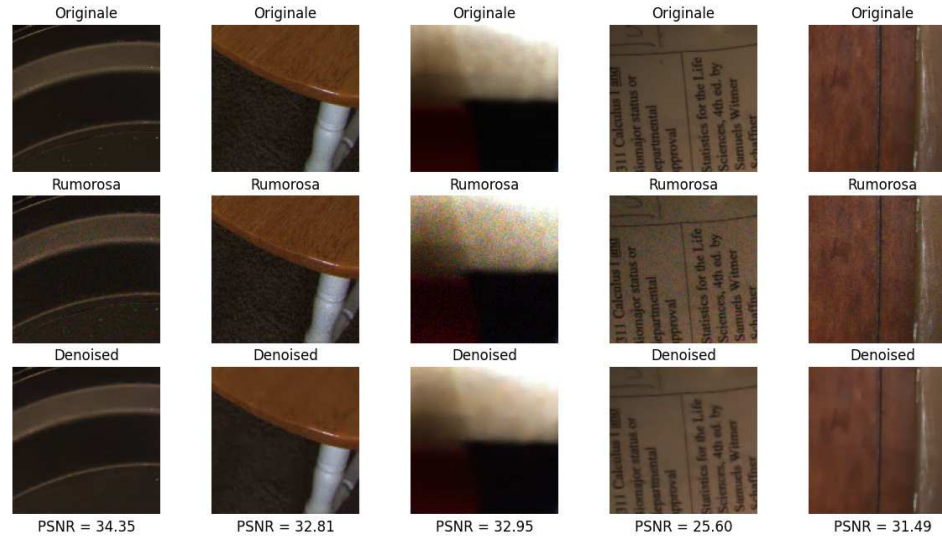
Figura B.7: Kvasir – Confronto BM3D ai diversi livelli di rumore.

Appendice C

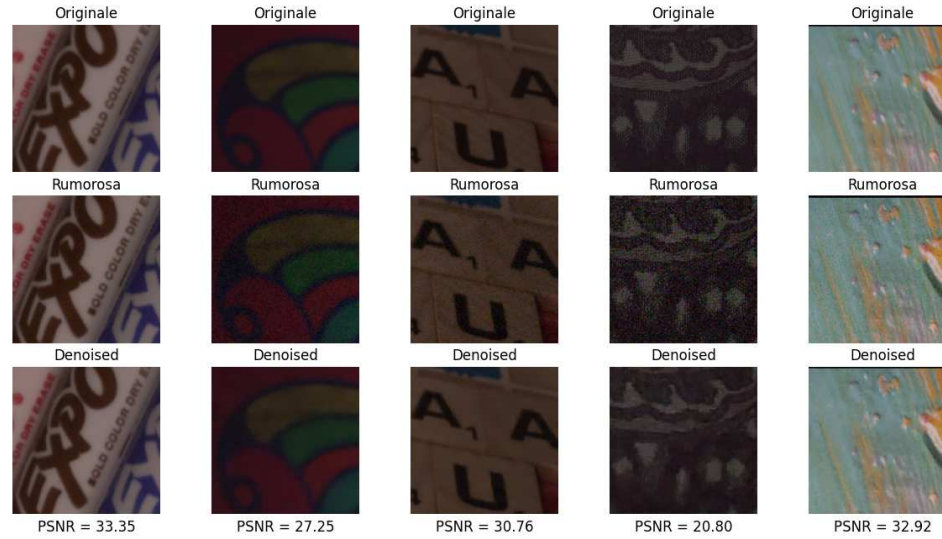
Risultati con Iperparametri ottimizzati

In questa appendice sono riportati i risultati ottenuti con la Ridnet addestrata usando gli iperparametri ottimizzati tramite Optuna, come descritto nella Sezione ??.

C.1 Training iniziale con iperparametri ottimizzati



(a) Dataset Renoir



(b) Dataset SIDD

Figura C.1: Risultati della Ridnet con iperparametri ottimizzati

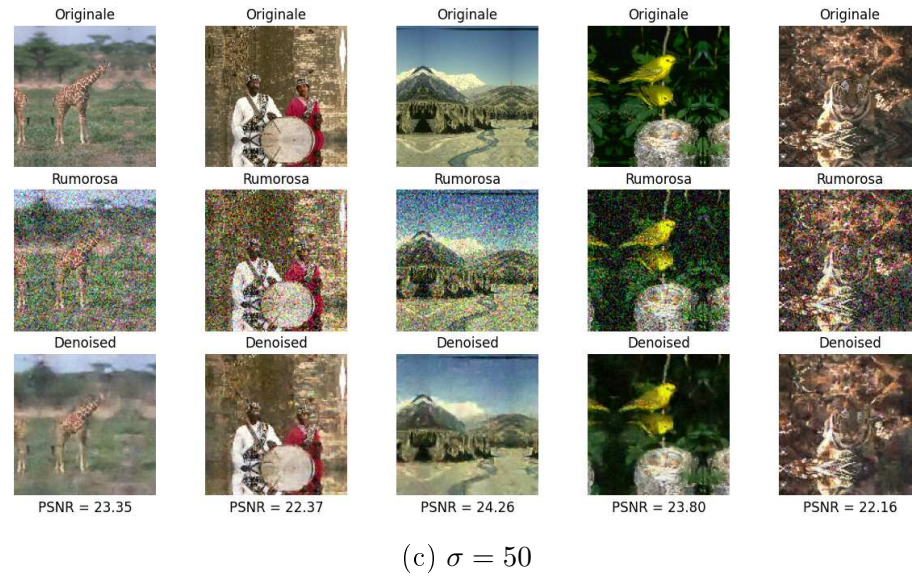
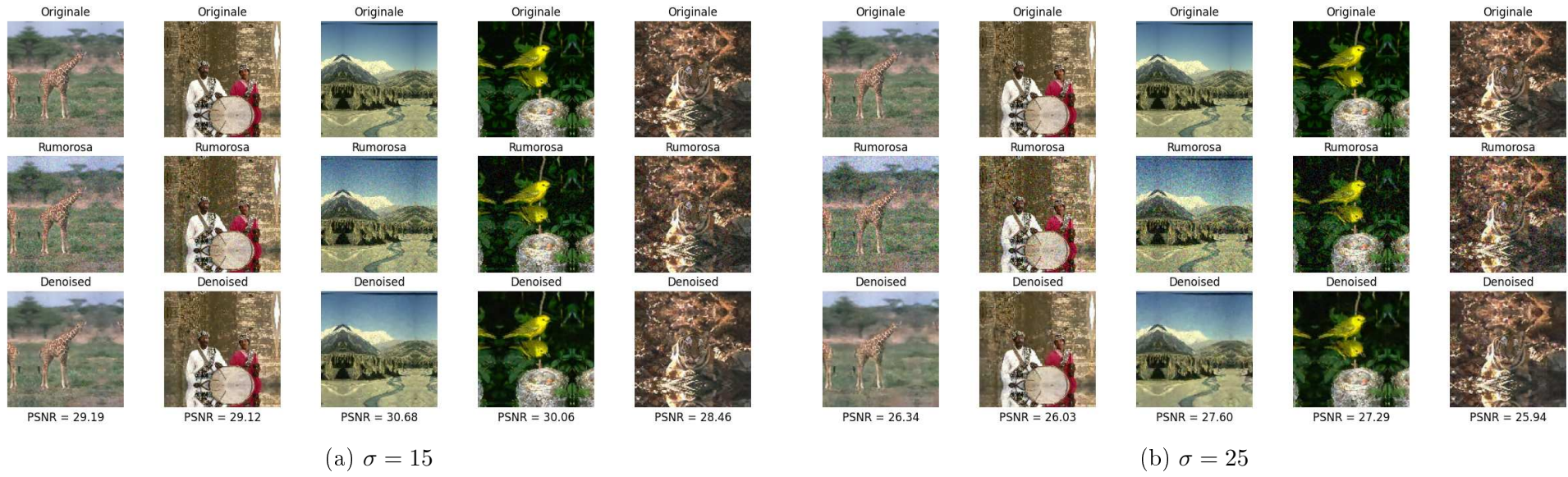


Figura C.2: Dataset BSD500 – Confronto Ridnet Student ai diversi livelli di rumore.

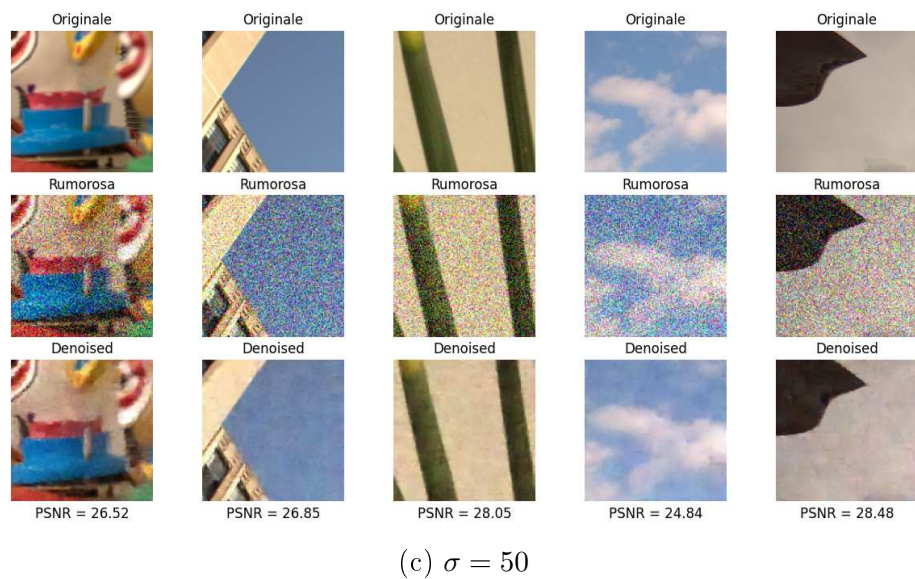
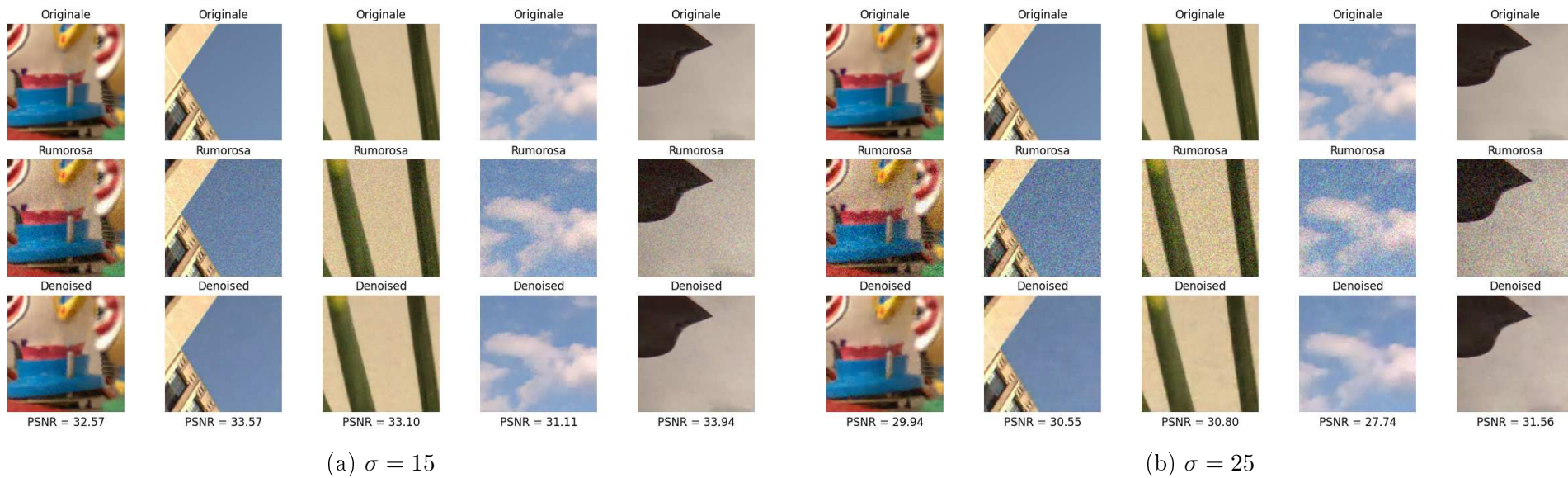


Figura C.3: Dataset MIT Adobe FiveK – Confronto Ridnet Student ai diversi livelli di rumore.

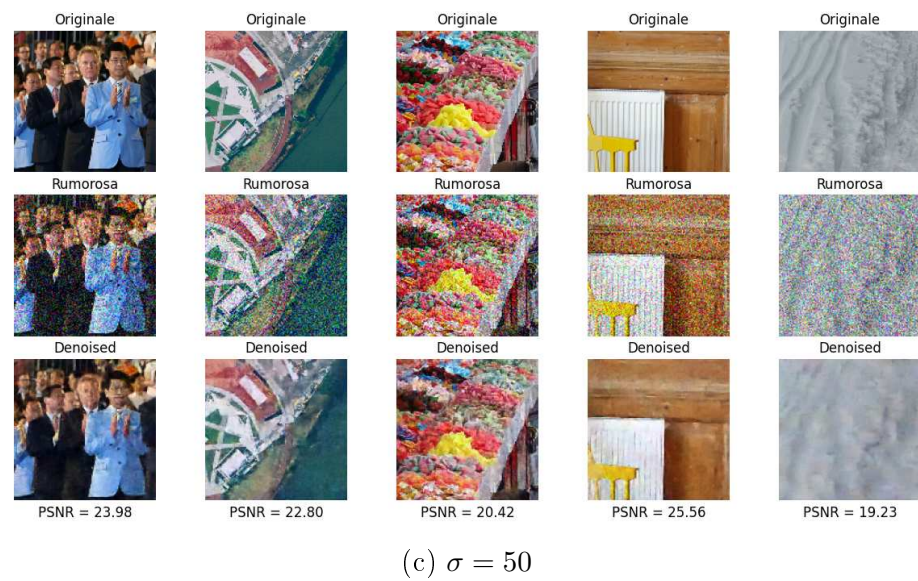
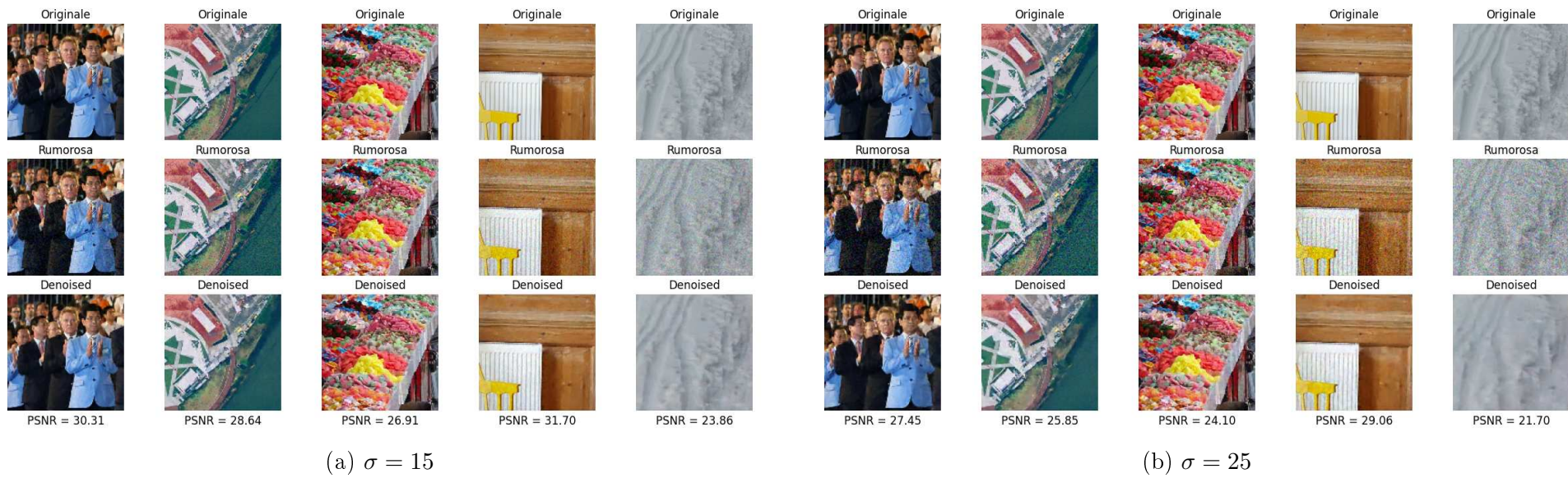


Figura C.4: Dataset Div2K – Confronto Ridnet Student ai diversi livelli di rumore.

C.2 Risultati Fine Tuning con iperparametri ottimizzati

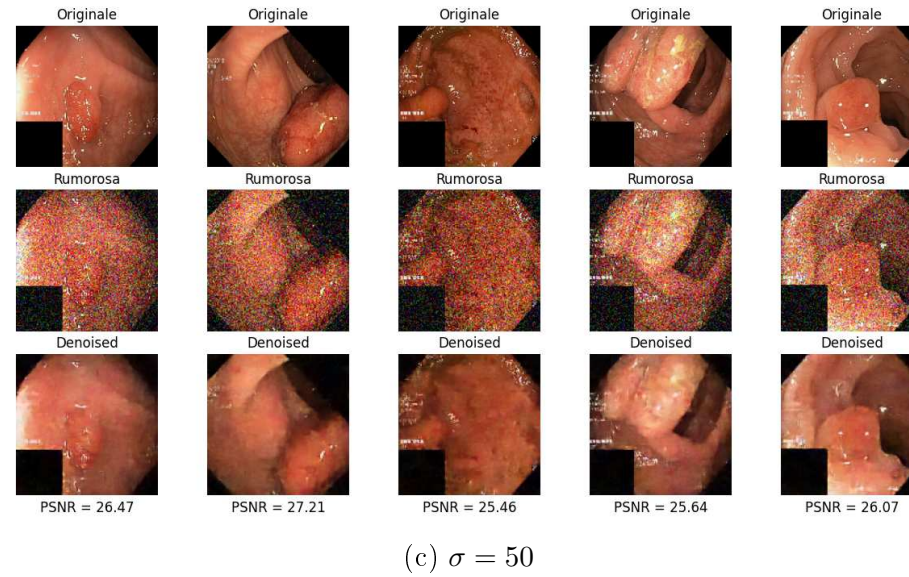
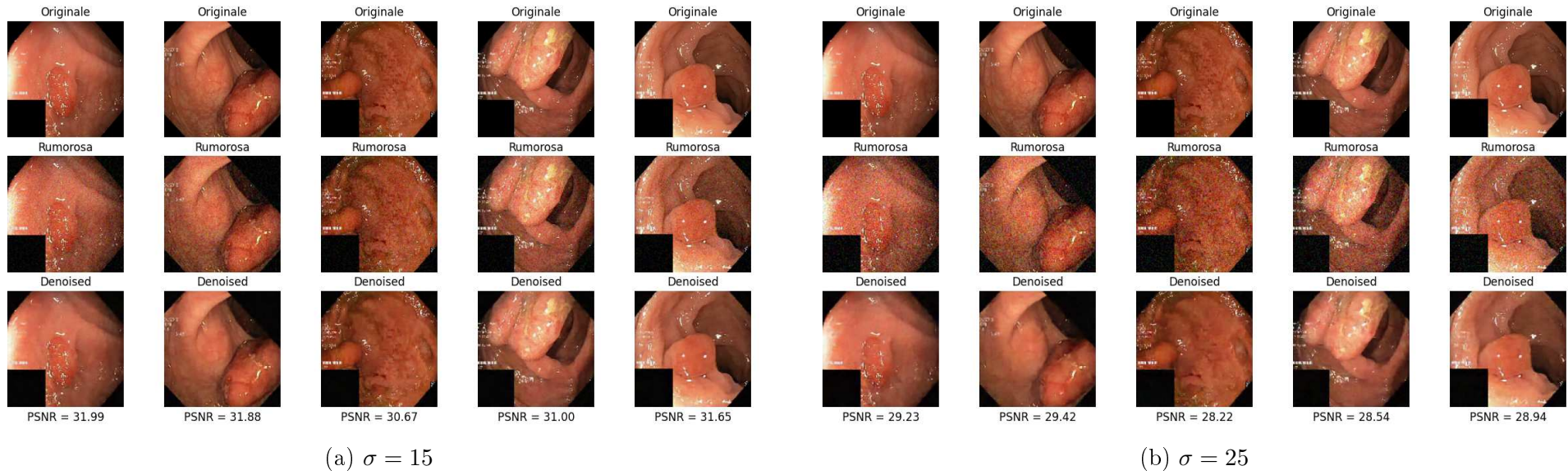
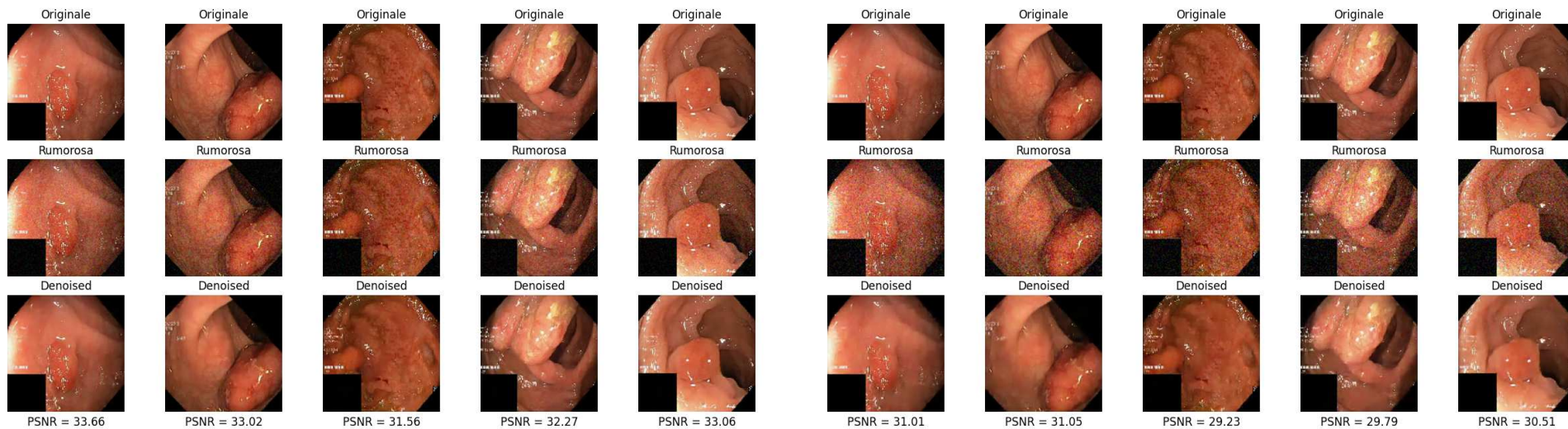
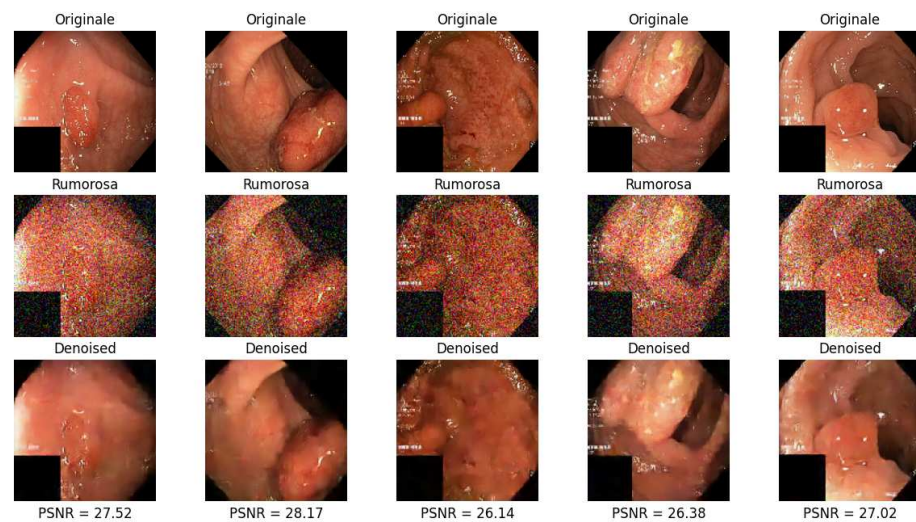


Figura C.5: Dataset Kvasir pre Fine tuning – Confronto Ridnet Student ai diversi livelli di rumore.



(a) $\sigma = 15$

(b) $\sigma = 25$



(c) $\sigma = 50$

Figura C.6: Dataset Kvasir post Fine tuning – Confronto Ridnet Student ai diversi livelli di rumore.

Appendice D

Ringraziamenti

Ringrazio la Prof. Lazzaro per la supervisione e aver sopportato la mia follia.

Grazie agli amici conosciuti in università: Serafino, Laura e Sofia per avermi aiutato ad andare avanti in questa pazza università aiutandomi con consigli, appunti e pazienza.

Un grazie speciale ai miei genitori, che mi hanno supportato e sopportato durante questo periodo folle.

Grazie ai quei tre amici che mi ritrovo dalle superiori che non ho capito come facciano ancora a sopportarmi: Monte, Pode e Pane.

Grazie alla vecchia guardia di S.P.R.I.Te, Andrea LR, FraDente, Delu, Perla, Cesca ed EleRizz per avermi aiutato in un modo o nell'altro sia con esami che solo qualche chiaccherata.

Anche alle nuove leve di S.P.R.I.Te per i bei momenti passati insieme.

Infine, un grazie a tutte le persone conosciute in questo periodo, che mi hanno fatto sorridere e divertire, e capire tante cose di me, rendendo questa esperienza indimenticabile.

Grazie a tutti voi.

Bibliografia

- [1] James R. Janesick. *Photon Transfer: $DN \rightarrow \lambda$* . Bellingham, WA, USA: SPIE Press, 2007. ISBN: 978-0-8194-6722-5. URL: <https://picture.iczhiku.com/resource/eetop/shITqHhziPt0LvCx.pdf>.
- [2] Alessandro Foi et al. “Practical Poissonian–Gaussian Noise Modeling and Fitting for Single-Image Raw-Data”. In: *IEEE Transactions on Image Processing* 17.10 (2008), pp. 1737–1754. DOI: 10.1109/TIP.2008.2001399. URL: https://webpages.tuni.fi/foi/papers/Foi-PoissonianGaussianClippedRaw-2007-IEEE_TIP.pdf.
- [3] Markku Mäkitalo e Alessandro Foi. “Optimal inversion of the generalized Anscombe transformation for Poisson–Gaussian noise”. In: *IEEE Transactions on Image Processing* 22.1 (2013), pp. 91–103. DOI: 10.1109/TIP.2012.2202675. URL: <https://webpages.tuni.fi/foi/papers/OptGenAnscombeInverse-doublecolumn-preprint.pdf>.
- [4] Hilario Gómez-Moreno et al. “A “Salt and Pepper” Noise Reduction Scheme for Digital Images Based on Support Vector Machines Classification and Regression”. In: *The Scientific World Journal* 2014 (2014), Article ID 826405. DOI: 10.1155/2014/826405. URL: <https://www.hindawi.com/journals/tswj/2014/826405/>.
- [5] J. W. Goodman. “Some fundamental properties of speckle”. In: *Journal of the Optical Society of America* (1976). seminal treatment of speckle statistics and properties; vedi anche review papers su despeckling per applicazioni medicali e SAR. URL: <https://materias.df.uba.ar/15a2021c1/files/2021/05/goodman1976.pdf>.
- [6] A. Rajamani e V. Krishnaveni. “Performance Analysis Survey of Various SAR Image Despeckling Techniques”. In: *International Journal of Computer Applications* 90.7 (2014), pp. 5–17. DOI: 10.5120/15584-4254.
- [7] G. K. Wallace. “The JPEG still-picture compression standard”. In: *Communications of the ACM* 34.4 (1991), pp. 30–44. URL: https://www.lirmm.fr/~wpuech/enseignement/master_informatique/Compression_Insertion/articles/15_Wallace_JPEG_1992.pdf.
- [8] R. Eschbach. “Conditional Post-Processing of JPEG Compressed Images”. In: *Proceedings (paper/report)* (1997). vedi anche letteratura su riduzione artefatti JPEG e blocking artifacts. URL: <https://www.imaging.org/common/uploaded%20files/pdfs/Papers/1997/IST-0-4/122.pdf>.
- [9] Rafael C. Gonzalez e Richard E. Woods. *Digital Image Processing*. 4th. Pearson, 2018.
- [10] Ery Arias-Castro e David L. Donoho. “Does median filtering truly preserve edges better than linear filtering?” In: *Annals of Statistics* (2009). analisi comparativa tra filtro mediano e filtro lineare.
- [11] Carlo Tomasi e Roberto Manduchi. “Bilateral Filtering for Gray and Color Images”. In: *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*. 1998, pp. 839–846.

- [12] Sylvain Paris et al. “Bilateral Filtering: Theory and Applications”. In: *Foundations and Trends® in Computer Graphics and Vision* 4.1 (2008), pp. 1–73.
- [13] G. J. Yang T. S. Huang e G. Y. Tang. “A Fast Two-Dimensional Median Filtering Algorithm”. In: *IEEE Transactions on Acoustics, Speech, and Signal Processing* 27.1 (1979), pp. 13–18.
- [14] David L. Donoho. “De-noising by soft thresholding”. In: *IEEE Transactions on Information Theory* 41.3 (1995), pp. 613–627.
- [15] Antonin Chambolle et al. “A Parameter Selection Method for Wavelet Shrinkage Denoising”. In: *BIT Numerical Mathematics* 43 (2003), pp. 611–626.
- [16] Anonymous Unsupervised Wavelet Denoising. “A wavelet denoising approach based on unsupervised learning model”. In: *EURASIP Journal on Advances in Signal Processing* 2020 (2020).
- [17] Stuart Geman e Donald Geman. “Stochastic Relaxation, Gibbs Distributions, and the Bayesian Restoration of Images”. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* PAMI-6.6 (1984), pp. 721–741.
- [18] Antoni Buades, Bartomeu Coll e Jean-Michel Morel. “A non-local algorithm for image denoising”. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. Vol. 2. IEEE. 2005, pp. 60–65.
- [19] Antoni Buades, Bartomeu Coll e Jean-Michel Morel. “A Non-Local Algorithm for Image Denoising”. In: *Proceedings of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR)*. 2005, pp. 60–65. URL: <https://www.iro.umontreal.ca/~mignotte/IFT6150/Articles/Buades-NonLocal.pdf>.
- [20] Kostadin Dabov et al. “Image denoising by sparse 3D transform-domain collaborative filtering”. In: *IEEE Transactions on Image Processing* 16.8 (2007), pp. 2080–2095. DOI: 10.1109/TIP.2007.901238.
- [21] Hua Wang et al. “A Review of Image Denoising Methods”. In: *Communications in Information and Systems* 20.4 (2020). Review covering classical and modern denoising methods, pp. 461–480. URL: https://www.intlpress.com/site/pub/files/_fulltext/journals/cis/2020/0020/0004/CIS-2020-0020-0004-a004.pdf.
- [22] Linwei Fan et al. “Brief review of image denoising techniques”. eng. In: *Visual computing for industry, biomedicine and art* 2.1 (2019), pp. 7–12. ISSN: 2524-4442.
- [23] Marc Lebrun e Arthur Leclaire. “An Implementation and Detailed Analysis of the K-SVD Image Denoising Algorithm”. In: *Image Processing On Line* 2 (2012). <https://doi.org/10.5201/ipol.2012.11m-ksvd>, pp. 96–133.
- [24] Wikipedia contributors. *K-SVD — Wikipedia, the free encyclopedia*. accessed 17 settembre 2025, general description of K-SVD algorithm. 2025. URL: <https://en.wikipedia.org/wiki/K-SVD>.
- [25] Quan Xiao et al. “Image Denoising via K-SVD with Primal-Dual Active Set Algorithm (K-SVD_P)”. In: *WACV. improvement over classical K-SVD for high noise levels*. 2020. URL: https://openaccess.thecvf.com/content_WACV_2020/papers/Xiao_Image_denoising_via_K-SVD_with_primal-dual_active_set_algorithm_WACV_2020_paper.pdf.

- [26] Jiechao Sheng et al. “SRNet: Sparse representation-based network for image denoising”. In: *Digital Signal Processing* 130 (2022). Embedding of sparse coding framework into CNN using unrolled optimization and multi-scale residual blocks, p. 103702. DOI: 10.1016/j.dsp.2022.103702. URL: <https://www.sciencedirect.com/science/article/abs/pii/S1051200422003190>.
- [27] Cristina Garcia-Cardona e Brendt Wohlberg. “Convolutional Dictionary Learning: A Comparative Review and New Algorithms”. In: *IEEE Transactions on Computational Imaging* 4.3 (2018). Survey and new approaches for convolutional sparse dictionary learning, pp. 366–381. DOI: 10.1109/TCI.2018.2840334. URL: <https://arxiv.org/abs/1709.02893>.
- [28] Yali Zhang et al. “Image Denoising using Convolutional Sparse Coding Network with Dry Friction”. In: *Proceedings of the Asian Conference on Computer Vision (ACCV)*. 2022, pp. 2322–2336.
- [29] Christian Marius Lillielund, Henrik Bagger Jensen e Christian Fischer Pedersen. “Cloud K-SVD for Image Denoising”. In: *SN Computer Science* 3 (2022). Dictionary learning across multiple nodes. Originally on arXiv:2303.00755, p. 151. DOI: 10.1007/s42979-022-01042-y. URL: <https://doi.org/10.1007/s42979-022-01042-y>.
- [30] Alan V. Oppenheim e Ronald W. Schaffer. *Discrete-Time Signal Processing*. 2nd. Prentice Hall, 1999.
- [31] Kai Zhang, Wangmeng Zuo, Yunjin Chen et al. “Beyond a Gaussian denoiser: Residual learning of deep CNN for image denoising”. In: *IEEE Transactions on Image Processing* 26.7 (2017), pp. 3142–3155.
- [32] Kai Zhang, Wangmeng Zuo e Lei Zhang. “FFDNet: Toward a fast and flexible solution for CNN-based image denoising”. In: *IEEE Transactions on Image Processing* 27.9 (2018), pp. 4608–4622. DOI: 10.1109/TIP.2018.2839891.
- [33] Jaakko Lehtinen et al. “Noise2Noise: Learning Image Restoration without Clean Data”. In: *Proceedings of Machine Learning Research* 80 (2018). arXiv:1803.04189, pp. 2965–2974. URL: <https://arxiv.org/abs/1803.04189>.
- [34] Saeed Anwar e Nick Barnes. “Real Image Denoising With Feature Attention”. In: *2019 IEEE/CVF International Conference on Computer Vision (ICCV)*. 2019, pp. 3155–3164. DOI: 10.1109/ICCV.2019.00325.
- [35] Leonid I. Rudin, Stanley Osher e Emad Fatemi. “Nonlinear total variation based noise removal algorithms”. In: *Physica D: Nonlinear Phenomena* 60.1-4 (nov. 1992), pp. 259–268. DOI: 10.1016/0167-2789(92)90242-F.
- [36] Anil Hore e D. Ziou. “Image Quality Metrics: PSNR vs. SSIM”. In: *International Journal of Computer Vision* 21.1 (2010), pp. 87–112. DOI: 10.1007/s00138-010-0230-4.
- [37] Zhou Wang et al. “Image Quality Assessment: From Error Visibility to Structural Similarity”. In: *IEEE Transactions on Image Processing* 13.4 (2004), pp. 600–612. DOI: 10.1109/TIP.2003.819861.
- [38] Joshua Batson e Loïc Royer. “Noise2Self: Blind Denoising by Self-Supervision”. In: *Proceedings of the 36th International Conference on Machine Learning (ICML)*. 2019, pp. 524–533. URL: <https://arxiv.org/abs/1901.11365>.
- [39] Pascal Vincent et al. “Extracting and Composing Robust Features with Denoising Autoencoders”. In: *Proceedings of the 25th International Conference on Machine Learning (ICML)*. 2008, pp. 1096–1103. DOI: 10.1145/1390156.1390294.

- [40] Abdelrahman Abdelhamed, Radu Timofte, Michael S. Brown et al. “NTIRE 2019 Challenge on Real Image Denoising: Methods and Results”. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*. 2019. DOI: 10.1109/CVPRW.2019.00320.
- [41] Kaiming He et al. “Deep Residual Learning for Image Recognition”. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2016, pp. 770–778. DOI: 10.1109/CVPR.2016.90.
- [42] Jie Hu, Li Shen e Gang Sun. “Squeeze-and-Excitation Networks”. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2018, pp. 7132–7141. DOI: 10.1109/CVPR.2018.00745.
- [43] Ian Goodfellow, Yoshua Bengio e Aaron Courville. *Deep Learning*. MIT Press, 2016. ISBN: 9780262035613. URL: <https://www.deeplearningbook.org/>.
- [44] Diederik P. Kingma e Max Welling. “Auto-Encoding Variational Bayes”. In: *arXiv preprint arXiv:1312.6114* (2013). URL: <https://arxiv.org/abs/1312.6114>.
- [45] Ian Goodfellow, Jean Pouget-Abadie et al. “Generative Adversarial Nets”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 27. 2014.
- [46] Alexey Dosovitskiy et al. “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale”. In: *International Conference on Learning Representations (ICLR)*. 2021. URL: <https://arxiv.org/abs/2010.11929>.
- [47] D. Martin et al. “A Database of Human Segmented Natural Images and its Application to Evaluating Segmentation Algorithms and Measuring Ecological Statistics”. In: *Proc. 8th Int’l Conf. Computer Vision*. Vol. 2. Lug. 2001, pp. 416–423.
- [48] Andrey Ignatov, Radu Timofte et al. “PIRM challenge on perceptual image enhancement on smartphones: report”. In: *European Conference on Computer Vision (ECCV) Workshops*. Gen. 2019.
- [49] Vladimir Bychkovsky et al. “Learning photographic global tonal adjustment with a database of input/output image pairs”. In: *CVPR 2011*. 2011, pp. 97–104. DOI: 10.1109/CVPR.2011.5995413.
- [50] Josue Anaya e Adrian Barbu. “RENOIR - A Dataset for Real Low-Light Noise Image Reduction”. In: *arXiv preprint arXiv:1409.8230* (2014).
- [51] Kai Zhang, Wangmeng Zuo, Shuhang Gu et al. “Learning a Deep CNN Denoiser Prior for Image Restoration”. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2017, pp. 3929–3938.
- [52] Radu Timofte, Eirik Agustsson e Luc Van Gool. “NTIRE 2017 Challenge on Single Image Super-Resolution: Methods and Results”. In: *CVPR Workshops*. 2017.
- [53] Konstantin Pogorelov et al. “KVASIR: A Multi-Class Image Dataset for Computer Aided Gastrointestinal Disease Detection”. In: *giu*. 2017. DOI: 10.1145/3083187.3083212.